

Department of Electrical Engineering and Computer Science

MASSACHUSETTS INSTITUTE OF TECHNOLOGY

6.097 Operating System Engineering: Fall 2002

Quiz I

All problems are open-ended questions. In order to receive credit you must answer the question as precisely as possible. You have 80 minutes to answer this quiz.

Write your name on this cover sheet AND at the bottom of each page of this booklet.

Some questions may be much harder than others. Read them all through first and attack them in the order that allows you to make the most progress. If you find a question ambiguous, be sure to write down any assumptions you make. Be neat. If we can't understand your answer, we can't give you credit!

THIS IS AN OPEN BOOK, OPEN NOTES QUIZ.

1 (xx/35)	2 (xx/20)	3 (xx/25)	4 (xx/15)	5 (xx/5)	Total (xx/100)

Name: Alyssa P. Hacker

I Traps/Interrupts

1. [15 points]: Draw a picture of the kernel stack after the following events have happened: (1) the icode program has called the exec system call; and (2) a clock interrupt handler has interrupted **exec()** at line 3027; and (3) the clock handler just finished executing line 3739? For each entry on the stack describe what that entry represents. You may ignore local variables.

(Draw high addresses high.)

The kernel stack starts off empty. A system call must be the first thing on a trap frame at this point — if any interrupts happened before, they would not have had any other process to switch to on the return path.

in trap	PSW from icode
	PC of icode's SYS instruction
	r0 of icode
	PSW after trap (br7 + 6)
	r1 of icode
	SP of icode
	masked PSW after trap (6)
	return address for call1 return (line 786)
in _trap	saved r5
	saved r4
	saved r3
	saved r2
	&cret
	&exec (an argument to trap1)
	return address in _trap (line 2772)
in _trap1	saved r5/r4/r3/r2 + &cret
	return address in _trap1 (line 2848)
in _exec	saved r5/r4/r3/r2 + &cret
CLOCK interrupt	PSW from exec
	PC from exec (line 3027)
	r0 from exec
	PSW after interrupt (br6)
	r1 from exec
	sp (same as sp now)
	masked PSW
	return address in trap (line 800)
in _clock	saved r5/r4/r3/r2 + &cret

2. [5 points]: Which kernel stack is the one you drew in response to question 1? Kernel stack 0? Kernel stack 1?

(Explain your answer briefly)

Kernel Stack 1 is used. Process 1 (icode) was executing at the time of the trap and so its kernel stack is used. If the machine is already in kernel mode when an interrupt happens, the current kernel stack is used.

3. [5 points]: Can another clock interrupt interrupt the handler at line 3739? At line 3770?

(Explain your answer briefly)

At line 3739, another clock interrupt *can not* interrupt the handler: the priority of the processor has been set to 6 already as a result of line 534 (or 535).

At line 3770, the interrupt handler has lowered the priority level to 5 so another clock interrupt (at level 6) *could* interrupt the handler at this point.

4. [10 points]: Estimate how large a kernel stack must be to handle recursive traps and interrupts correctly?

(Explain your answer briefly)

The kernel stack must be able to hold sufficient stack and trap frames for the most recursive case. This occurs if a process executes a system call, receives a priority 4 interrupt, followed by a priority 5 interrupt, followed by a priority 6 interrupt, followed by a re-entrant priority 6 interrupt.

In addition to the size of the trap frame (i.e. the arguments to **_trap**), you would need to consider stack frames generated by the deepest system call procedure call path (e.g. **open** calls **namei** which calls ...; this is probably not too deep but might change as the kernel evolved), the deepest interrupt 4 handler call path, etc.

From the figure, we can see that a system call trap requires 20 stack elements for the trap frame and stack frames for the trap dispatch code (plus some extra for locals). Similarly, a non-syscall trap requires 8 stack elements for the trap frame. Assume that the maximum call depth for a function in the kernel is 4, where each stack frame requires 6 stack elements (5 from **csav**, and an average of 1 argument). This gives a maximum depth of approximately

$$20 + 4 \times 6 + 4 \times (8 + 4 \times 6) = 172$$

. Since each stack entry is two bytes, this gives a lower bound of 344 bytes. Of course, some additional room will be needed to account for local variables, which were ignored for this problem.

II Coordinating threads

5. [5 points]: What do the statements at line 2088 and line 2092 do?

(Explain your answer briefly)

Line 2088 calls **spl6()** which sets the processor privilege level to 6. This causes all interrupts of priority less than or equal to 6 to be ignored. Line 2092 calls **spl0()**, lowering the privilege level back to 0. These calls are used to mark atomic sections.

6. [15 points]: Why are these statements needed? Give an actual sequence of events that will result in erroneous behavior if these two statements are omitted. Your sequence of events should be as concrete as possible, not merely hypothetical.

(Illustrate your answer using a time line)

Several scenarios can lead to incorrect behavior:

Deadlock:

1. **runin**++ on 1954
2. scheduler calls **sleep** on 1955 with **runin** as channel
3. executes lines 2089 setting **p_wchan** to **&runin**
4. processor is interrupted by clock
5. clock interrupt handler sets **runin** to zero (3821)
6. handler calls wakeup with **&runin** as argument (3822)
7. wakeup calls setrun, which sets **p_wchan** to zero and **p_stat** to **SRUN** (2139–2140)
8. handler returns
9. now scheduler sets **p_stat** to **SSLEEP** (2090)
10. and calls **swtch**
11. scheduler will never be woken up, because **p_stat == SSLEEP** and **p_wchan** is zero

Deadlock:

1. a user process calls indirectly **bwrite()**, which launches write on buffer.
2. then calls **iowait**,
3. which calls sleep with the address of the buffer as the channel
4. sleep sets **p_wchan** (2089)
5. disk interrupt comes in completing the write of the buffer
6. it calls **iodone**, which calls **wakeup** on the buffer address
7. calls **setrun**, which sets **p_wchan** to zero and **p_stat** to **SRUN**
8. interrupt handler completes
9. process sets **p_stat** to **SSLEEP** (2090)
10. and call **swtch**
11. process will never be woken up because **p_wchan** is zero and **p_stat SSLEEP**

III Address spaces

7. [10 points]: Draw the prototype segmentation registers with their actual content after the call to **estabur()** on line 3152 has completed. Assume that **w0** is 410, **w1** is 129, **w2** is 32, and **w3** is 128 (see lines 3076 through 3082). Explain the content of the registers briefly.

(Ignore the protection bits in the descriptor)

Converting what we are given into blocks (and hence the arguments to **estabur**), we get:

```
text size is 129 bytes (w1 = 129)          ==> nt = 3 blocks
SSIZE (stack size) = 20 blocks             ==> ns = 20 blocks
data size is 160 bytes (w2 = 32, w3 = 128) ==> nd = 3 blocks
USIZE = 16 blocks
```

That is, **estabur(nt = 3, nd = 3, ns = 20, sep = 0)**.

Since each segment (8KB) contains 128 blocks, the text, data, and stack each fit within one segment:

```
         +------------+          +------------+
PAR[7]:  |    -89     |  PDR[1]: |  108 ED RW |
         +------------+          +------------+

PAR[7] = USIZE + nd + ns - 128
       = 16 + 3 + 20 - 128
       = 39 - 128
       = -89

PDR[1] = (128 - ns) << 8 | ED | RW
       = 108<<8 | ED | RW

         +------------+          +------------+
PAR[1]:  |  USIZE     |  PDR[1]: |  2      RW |
         +------------+          +------------+

PAR[1] = USIZE = 20
PDR[1] = (nd - 1) << 8) | RW
       = 2<<8 | RW

         +------------+          +------------+
PAR[0]:  |      0     |  PDR[0]: |  2      RO |
         +------------+          +------------+

PAR[0] = 0, because at line 1675 a = 0
PDR[0] = (nt - 1) << 8) | R0
       = 2<<8 | R0
```

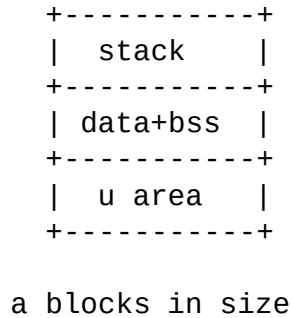
All other PARs and PDRs are cleared by lines 1694–1696.

Name: Alyssa P. Hacker

8. [10 points]: Why is "a-128" stored in the prototype segment for the stack? Explain your answer by showing how the virtual address 0177777 is translated to a physical address.

(Explain your answer briefly)

At the point where "a-128" is written into PAR[7], "a" equals the combined sizes of the u area, data, bss and stack. A contiguous region of physical memory ("a" blocks in size) is allocated to hold these components of the process.



Dividing the VA 0177777 into its constituent fields, APF:block_number:block_offset yields 7:127:63. Translation of this VA selects PAR[7] which holds "a-128". Added to this number is the block_number, 127, resulting in physical block number "a-1". The physical address refernced is therefore "a-1":63. But since the chunk of physical memory allocated was of size "a" blocks, this precisely refers to the last byte (byte 63) of the last block (block "a-1"). This is where the first byte of the stack is located.

9. [5 points]: What happens if the processor executes the instruction "tst -(sp)", and the user stack pointer falls below the stack segment? Write down the names of the functions that v6 invokes in order.

(Explain briefly)

A segmentation exception will result and the stack will grow (assuming free memory can be found).

The following calls will be made:

```
Machine generates exceptions
  => trap:
     => trap()
        => backup()
        => grow()
```

Then control returns back to the user process and it resumes execution where it left off, re-executing the "tst -(sp)" instruction. The second execution succeeds of course.

IV I/O and file systems

10. [5 points]: Why does the structure **struct file** exist? Why not have a file descriptor point directly to the incore inode structure, and store **f_offset** in it?

struct file exists so that if two separate processes open the same file, they have their own offset pointer into it.

Name: Alyssa P. Hacker

11. [10 points]: Assume the directory "d" exists, that it is the current working directory, and that it has two entries (including "." and '"..."). If an application calls **creat ("bar", 0444)**, and the power fails right after the processor completed the call **wdir** on line 7467, what is the state of the file system on disk? More specifically, does the file "bar" exist on disk? Does the file "bar" show up in the directory "d" on disk?

(Be brief)

The updated inode of directory "d" exists (because inodes are updated synchronously). The data of "d" may not be written because v6 wrote it using **bdwrite** (it is a partial block). The file doesn't exist yet, because we haven't gotten far enough yet into the **creat** call (**maknode** is called from line 5790).

End of Quiz I Solutions

MIT OpenCourseWare
http://ocw.mit.edu

6.828 Operating System Engineering
Fall 2012

For information about citing these materials or our Terms of Use, visit: http://ocw.mit.edu/terms.



Department of Electrical Engineering and Computer Science

MASSACHUSETTS INSTITUTE OF TECHNOLOGY

6.828 Operating System Engineering: Fall 2006

Quiz I

All problems are open-ended questions. In order to receive credit you must answer the question as precisely as possible. You have 80 minutes to answer this quiz.

Write your name on this cover sheet AND at the bottom of each page of this booklet.

Some questions may be much harder than others. Read them all through first and attack them in the order that allows you to make the most progress. If you find a question ambiguous, be sure to write down any assumptions you make. Be neat. If we can't understand your answer, we can't give you credit!

THIS IS AN OPEN BOOK, OPEN NOTES QUIZ.

1 (xx/20)	2 (xx/10)	3 (xx/25)	4 (xx/35)	5 (xx/10)	Total (xx/100)

Name: Solution Set

Quiz score statistics: median 65, mean 67, maximum 100.

I Processes and concurrency

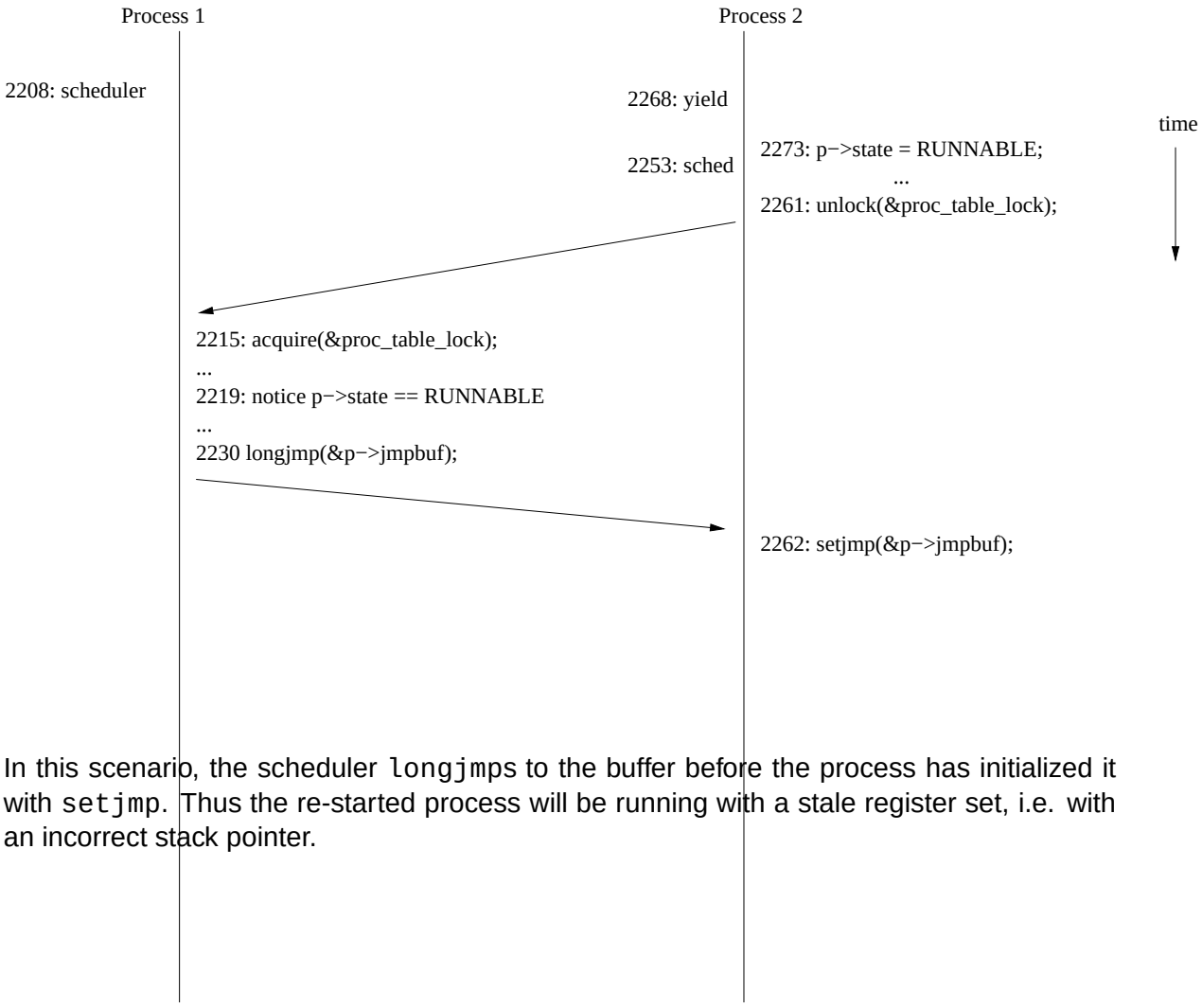
1. [10 points]: Ben Bitdiddle observes that releases and acquires of `proc_table_lock` are spread through proc.c, which makes it difficult to reason if the code is correct. He rereads the comment on line 2222, and proposes that the to-be-scheduled process doesn't release `proc_table_lock` when it runs. That is, he removes all the `acquire` and `release` calls for `proc_table_lock` except the two already in `scheduler`. He recompiles xv6 and boots it. What will Ben observe?

Only one processor at a time will be able to run a user process.

2. [10 points]: Dissatisfied with the simplification of the previous question, Ben Bitdiddle explores another altnerative. He undoes all his changes, and now he modifies the code so that during a context switch from one process to another, `proc_table_lock` isn't held. That is, he makes the following changes:

- release `proc_table_lock` before line 2262 and before line 2229;
- acquire at 2231;
- remove the releases in, for example, `yield` and `sleep`.

Ben observes a race condition between one processor running `scheduler` and a second processor running `yield` (on behalf of a user process). Give a timeline of events with 2 processes that demonstrates the race condition.



In this scenario, the scheduler `longjmp`s to the buffer before the process has initialized it with `setjmp`. Thus the re-started process will be running with a stale register set, i.e. with an incorrect stack pointer.

Name:

II System calls

3. [10 points]: If one process kills a process that is sleeping, it calls `proc_kill`, which sets `p->killed` and makes the target process runnable. In `trap`, the target process checks `p->killed` and exits. An alternative is for the target process to check `p->killed` when it comes out of sleep (e.g., right after line 2230), and call exit. Why is this alternative undesirable? That is, why wait with exiting until the return from trap? (Hint: think about system calls that invoke sleep several times, such as creating a file). Be specific in your answer by giving a concrete example.

If the process has any data structures in inconsistent states, it needs to be given a chance to undo them.

For example, at line 4024, if the process stopped executing during `ide_rw`, the block it is holding would never be `brelse`'d (would be B_BUSY forever). Other processes trying to read this block later would deadlock.

Name:

III Address spaces

4. [10 points]: JOS uses the paging support of the x86 processor, while xv6 uses only the segmentation support. Clearly using paging in JOS required you to write more lines of code than xv6 requires for using segmentation. List 3 significant ways in which JOS is better than xv6 because of its use of paging.

JOS does not require an environment's memory to be laid out in contiguous physical memory.

JOS can allow sharing of memory between environments at small granularity (individual pages). This is used, for example, to share the UENVS table with all environments.

JOS can edit memory permissions at small granularity (individual pages). This is used, for example, to implement copy-on-write fork.

(There are other correct answers as well. Some of these benefits could be realized with segmentation, but not the way xv6 uses segments, and probably not using C.)

In JOS `page_free` is called when the reference count of a page reaches 0. It adds the page to the free list of pages. Of course, the kernel can reference the page via the mapping at KERNBASE (the one used by KADDR).

5. [10 points]: To keep the kernel honest you might consider removing the KERNBASE mapping in `page_free`. Assuming no other changes outside of `page_free`, what would break as a result?

After a page is freed, accesses to the KADDR mapping would cause extra page faults once the page was re-allocated and referenced by the kernel (for example, to zero the page before reusing it).

6. [5 points]: How would you fix this problem while still removing the KERNBASE mapping in `page_free`?

Remap the page in `page_alloc`.

Name:

IV File systems

The xv6 cache implements a least recently-used cache replacement policy (i.e., it evicts the block from the cache that hasn't been used for the longest time).

7. [4 points]: What lines of code reorder the `bufhead` list to reflect the buffers' usage pattern?

4037–4042

8. [6 points]: List the code changes necessary to make xv6 implement a most-recently used policy (i.e., evict the block from the cache that has been used most recently). (Give line numbers that would need to change, along with the changes.)

Change line 3987 to traverse the list in the opposite direction.

(Or change lines 4037-4042 to insert the released block at the other end of the list.)

9. [10 points]: Identify one disk block that you expect to be in the cache frequently with the least-recently used policy. (Explain briefly.)

The superblock (block 1) is used in every call to `balloc`, `bfree`, and `ialloc`.

(There are other correct answers as well.)

In xv6 two child processes can communicate through pipes only if they have a common ancestor who created the pipe. To support communication between any two processes, some Unix operating systems provide *named pipes*, which are also called FIFO. A named pipe is durable and appears in the file system name space, and must be unlinked explicitly after use. One could use named pipes as follows. One process could do as follows:

```
mkfifo pipe       // make a named pipe
gzip < pipe > out // read from the named pipe, compress, and write to out
```

And, another process, unrelated to the first, could do as follows:

```
cat file > pipe   // write the contents to the named pipe
```

The result of opening a named pipe is a file descriptor that behaves exactly like one of the two file descriptors returned by the pipe call, depending on the open mode (for reading or for writing).

10. [15 points]: Write up how you would change xv6 to support named pipes. List the functions that would have to be added or changed, with short descriptions of the new functionality or changes.

Add a new system call `sys_mkfifo` that uses `mknod` to create an inode with the new type `T_PIPE`.

Create a new in-memory table mapping inode numbers to `struct file*` pairs.

In sys_open, at line 5285, check whether `ip->type` is `T_PIPE`. If so, look in the table for an entry for `ip->inum`. If there is not one there, allocate a new table entry and call `pipe_alloc` to get a pair of `struct file*` for the new entry. Use the corresponding `struct file*` (either the read or write end) instead of calling `filealloc` at line 5286.

Name:

V 6.828

11. [5 points]: Describe the most memorable error you have made so far in one of the labs. (Provide enough detail that we can understand your answer.)

Some memorable bugs (in summary):

Using `M-x comment-region` to comment out assembly in Emacs commented the lines with `;`, which does not actually introduce a comment in x86 assembly.

Setting the upcall function pointer to the C page fault handler instead of the assembly stub.

Pointer arithmetic problems, especially adding a `PDX` to an integer address instead of to a `pte_t*`.

Adding ANSI text coloring but not changing the default (code zero) black on black.

We would like to hear your opinions about 6.828, so please answer the following two questions. (Any answer, except no answer, will receive full credit!)

12. [3 points]: What is the best aspect of 6.828?

13. [2 points]: What is the worst aspect of 6.828?

End of Quiz I

Name:

MIT OpenCourseWare
http://ocw.mit.edu

6.828 Operating System Engineering

Fall 2012

For information about citing these materials or our Terms of Use, visit: http://ocw.mit.edu/terms.



Department of Electrical Engineering and Computer Science

MASSACHUSETTS INSTITUTE OF TECHNOLOGY

6.828 Fall 2009

Quiz I

All problems are open-ended questions. In order to receive credit you must answer the question as precisely as possible. You have 80 minutes to finish this quiz.

Write your name on this cover sheet AND at the bottom of each page of this booklet.

Some questions may be harder than others. Read them all through first and attack them in the order that allows you to make the most progress. If you find a question ambiguous, be sure to write down any assumptions you make. Be neat. If we can't understand your answer, we can't give you credit!

THIS IS AN OPEN BOOK, OPEN NOTES EXAM.

Please do not write in the boxes below.

I (xx/28)	II (xx/20)	III (xx/18)	IV (xx/28)	V (xx/6)	Total (xx/100)

Name:

I Concurrency

1. [8 points]: In xv6, the interrupt handler `ideintr` uses the lock `idelock` to ensure that it executes atomically. For example, if processor 0 runs `iderw` and processor 1 runs the interrupt handler `ideintr`, then the lock `idelock` ensures that their critical sections are appropriately ordered. Why is it important that `acquire(&idelock)` also turns off interrupts, in addition to marking the lock as acquired?

2. [8 points]: `bget` (and thus `bread`) returns a locked block to its caller. Why is it important that xv6 locks blocks? Consider `ialloc` (line 3802). What could go wrong from line 3811 through 3817 if the block returned by `bread` wasn't locked?

3. [8 points]: `bget` (line 3566) uses an integer `flags` stored in the returned block to indicate whether a thread is holding the lock on the block, instead of a spinlock. What could go wrong if we replaced the per-block `B_BUSY` flag used by `bget` with a per-block spinlock?

4. [4 points]: What ensures that setting `flags` in `bget` (line 3566) is an atomic operation?

II File systems

5. [8 points]: Ben wants to speed up the rate at which xv6 can write to the disk. He notices that `iderw` can have only one outstanding disk write per user process; xv6 puts the caller of `iderw` to sleep until the dirty bit is cleared by `ideintr`. Ben proposes to modify `iderw` to not call sleep, but instead return immediately. If a process issues multiple writes, all these writes will end up in the IDE queue, and the driver and disk controller can optimize performing a batch of writes. Ben makes all the required changes to xv6; he observes that write throughput improves for large files and that the system works correctly when there are no failures. Unfortunately, when Ben runs `fsck` after a system failure, he finds he cannot repair the disk into a consistent state anymore. What has gone wrong?

6. [6 points]: Ben obsesses about performance and proposes a design to speed up directory listings. He stores the inode for each file in the `struct dirent`, a scheme called *embedded inodes*. When a user types "`ls -l`" to get a detailed directory listing, xv6 can avoid reading the inode for each file in the directory, because it is stored with the directory. Ben makes the change, and observes that xv6 is faster for running "`ls -l`". He also notices that hard links don't work correctly any more. Give a sequence of shell commands that will work incorrectly with Ben's embedded inodes and briefly describe what will go wrong.

7. [6 points]: Suggest a solution to the link problem that achieves the performance benefits of embedded inodes without sacrificing correctness.

III Stacks

8. [8 points]: Suppose you wanted to change the system call interface in JOS so that, instead of returning the system call result in EAX, the kernel pushed the result on to the user space stack. Fill in the code below to implement this. For the purposes of this question, you can assume that the user stack pointer points to valid memory.

```
static void
trap_dispatch(struct Trapframe *tf)
{
        // ...
        if (tf->tf_trapno == T_SYSCALL) {
                r = syscall(tf->tf_regs.reg_eax, ...);
                // QUIZ: Your code here









        }
        // ...
}
```

9. [8 points]: xv6's `initcode` invokes the `exec` system call to replace itself with `/init`. `exec` allocates a stack for `/init` and carefully initializes it, starting from line 5080. Draw the stack it builds, giving the values that are on the stack and a brief explanation of each value. For pointers, simply draw an arrow or describe where they point to.

10. [2 points]: While debugging xv6, Alyssa notices the value `0x7b70` in the ESP register. Which kernel thread is running on the processor?

IV Virtual Memory

Ben Bitdiddle wants to add a debugger for user programs to his implementation of JOS. He decides to write this debugger as a regular user program, and realizes that he's going to need to add some system calls to support debugging. He starts with a system call that lets the debugger copy a page of memory between its own environment and any other environment.

Ben first implements his new syscall, `sys_copy_page`, by switching to the source environment's address space and copying the data into a scratch buffer, then switching to the destination environment's address space and copying the data back out of the scratch buffer. Ben's code is as follows:

```
static char scratch[PGSIZE];

static int
sys_copy_page(envid_t srcenvid, void *srcva,
              envid_t dstenvid, void *dstva)
{
        struct Env *srcenv, *dstenv;
        int r;

        if ((r = envid2env(srcenvid, &srcenv, 0)) < 0)
                return r;
        if ((r = envid2env(dstenvid, &dstenv, 0)) < 0)
                return r;
        if (srcenv != curenv && dstenv != curenv)
                return -E_INVAL;

        lcr3(srcenv->env_cr3);
        memmove(scratch, srcva, PGSIZE);
        lcr3(dstenv->env_cr3);
        memmove(dstva, scratch, PGSIZE);

        lcr3(curenv->env_cr3);
        return 0;
}
```

11. [8 points]: Louis Reasoner is troubled by Ben's approach. He points out that Ben is copying data in to the scratch buffer while running in one environment's address space, then expecting to read that data back *after* switching to a different address space. Ben assures Louis that this is alright. Why is it okay for Ben's implementation to switch between address spaces like this?

12. [6 points]: Having addressed Louis' concerns, Ben starts boasting about his understanding of address spaces and protection. Alyssa decides to put his kernel to the test and discovers that, not only can she crash Ben's kernel by passing the virtual address of an unmapped page to `sys_copy_page`, but she can write a user program that writes to arbitrary kernel memory. How can Ben fix these two problems?

13. [6 points]: Ben decides to change his strategy to copy data directly from the source page to the destination page without switching address spaces. His new implementation of `sys_copy_page` starts by finding the `struct Page*` of the source page and the physical address of the destination page. Why does this approach let Ben avoid switching address spaces in `sys_copy_page`?

14. [8 points]: Ben uses a helper function to copy data between the source and destination pages that he looked up in his new `sys_copy_page`. Implement this function below.

```
void
copy_physical_page(physaddr_t dst, struct Page* src)
{



}
```

V 6.828

We'd like to hear your opinions about 6.828, so please answer the following questions. (Any answer, except no answer, will receive full credit.)

15. [2 points]: This year we posted draft chapters that provide a commentary on xv6. Did you read them? If so, did you find the chapters useful? What should we do to improve them?

16. [2 points]: What is the best aspect of 6.828?

17. [2 points]: What is the worst aspect of 6.828?

End of Quiz

Name:

MIT OpenCourseWare
http://ocw.mit.edu

6.828 Operating System Engineering

Fall 2012

For information about citing these materials or our Terms of Use, visit: http://ocw.mit.edu/terms.



Department of Electrical Engineering and Computer Science

MASSACHUSETTS INSTITUTE OF TECHNOLOGY

6.828 Fall 2008

Quiz I

All problems are open-ended questions. In order to receive credit you must answer the question as precisely as possible. You have 80 minutes to finish this quiz.

Write your name on this cover sheet AND at the bottom of each page of this booklet.

Some questions may be harder than others. Read them all through first and attack them in the order that allows you to make the most progress. If you find a question ambiguous, be sure to write down any assumptions you make. Be neat. If we can't understand your answer, we can't give you credit!

THIS IS AN OPEN BOOK, OPEN NOTES EXAM.

Please do not write in the boxes below.

I (xx/12)	II (xx/18)	III (xx/16)	IV (xx/12)	V (xx/8)
VI (xx/16)	**VII (xx/8)**	**VIII (xx/4)**	**IX (xx/6)**	**Total (xx/100)**

Name:

I System Call Arguments

1. [6 points]: Given a user-provided pointer to a buffer in user memory (such as the argument of a system call), explain what checks and translations the JOS kernel must do to ensure that it can safely read or write the buffer memory. Do not assume or rely on the existence of a function in the kernel that does these checks and translations for you.

2. [6 points]: Given a user-provided pointer to a buffer in user memory (such as the argument of a system call), explain what checks and translations the xv6 kernel must do to ensure that it can safely read or write the buffer memory. Do not assume or rely on the existence of a function in the kernel that does these checks and translations for you.

II File System

3. [8 points]:

Describe two situations in which a power failure during a system call could leave different parts of the xv6 file system metadata inconsistent with each other. By "metadata" we mean the file system's on-disk data structures: the superblock, inodes, indirect blocks, the block in-use bitmap, and directory contents. A power failure halts the CPU and stops the disk after completion of the current sector read/write request (if any). Be sure to indicate the line number after which you assume the power failure occurs for each of the two situations, and the nature of the resulting inconsistencies.

4. [10 points]: Alice is worried that her xv6 kernel might panic at line 3596 because it runs out of struct bufs, of which there are only 10 (line 3529). Since she is mostly concerned with reliability, and does not care about the performance benefits of using these buffers as a cache, she decides to replace bget() and brelse() as follows:

```
static struct buf* bget(uint dev, uint sector) {
    struct buf *b = kalloc(sizeof(*b));
    memset(b, 0, sizeof(*b));
    b->flags |= B_BUSY;
    b->dev = dev;
    b->sector = sector;
    return b;
}

void brelse(struct buf *b) {
    kfree(b, sizeof(*b));
}
```

She modifies kalloc() to allow any size (rather than just multiples of 4096). Assuming that kalloc() never fails, describe a concrete correctness problem she will run into with this code.

III Paging vs. segmentation

JOS sets up the x86 page table to contain PTE entries for all kernel code and data structures (at a high virtual address, KERNBASE = 0xF00000000), even when running a user-space environment. The code and data for the user-space environment is mapped at low virtual memory addresses.

5. [4 points]: Explain why it is safe to keep sensitive kernel data structures mapped at 0xF0000000 while executing user-level code, which should not be able to access those kernel data structures.

6. [6 points]: Explain why JOS needs the kernel mapped while executing user-level code.

The xv6 kernel lives at low addresses, starting at 0x00100000. xv6 user-level code resides at memory addresses starting from zero, so a program with more than 1MB of instructions will use addresses that overlap with the kernel's addresses.

7. [6 points]: Explain how the processor allows both the xv6 kernel and user-level processes to store instructions at the same addresses.

IV Traps and interrupts

Recall that gcc calling conventions allow any function to trash the "caller-saved" registers %eax, %ecx, %edx, so that they may be different upon return.

8. [6 points]: xv6's system call stubs (sheet 68) are called by user-space C code as if they were ordinary C functions, so all callers are prepared for the stubs to not preserve caller-saved registers. The stubs invoke the INT instruction, which vectors into kernel code that shortly jumps to alltraps (line 2456). alltraps and trapret save and restore all registers to and from a struct trapframe (line 0477), including %eax, %ecx, and %edx, using pushal and popal, even though gcc does not expect any of these registers to remain the same after return from the system call stub.

Suppose we wanted to reduce the size of struct trapframe by not saving %ecx and %edx in alltraps. It turns out that this would cause problems. Why? Give an example of something that would break.

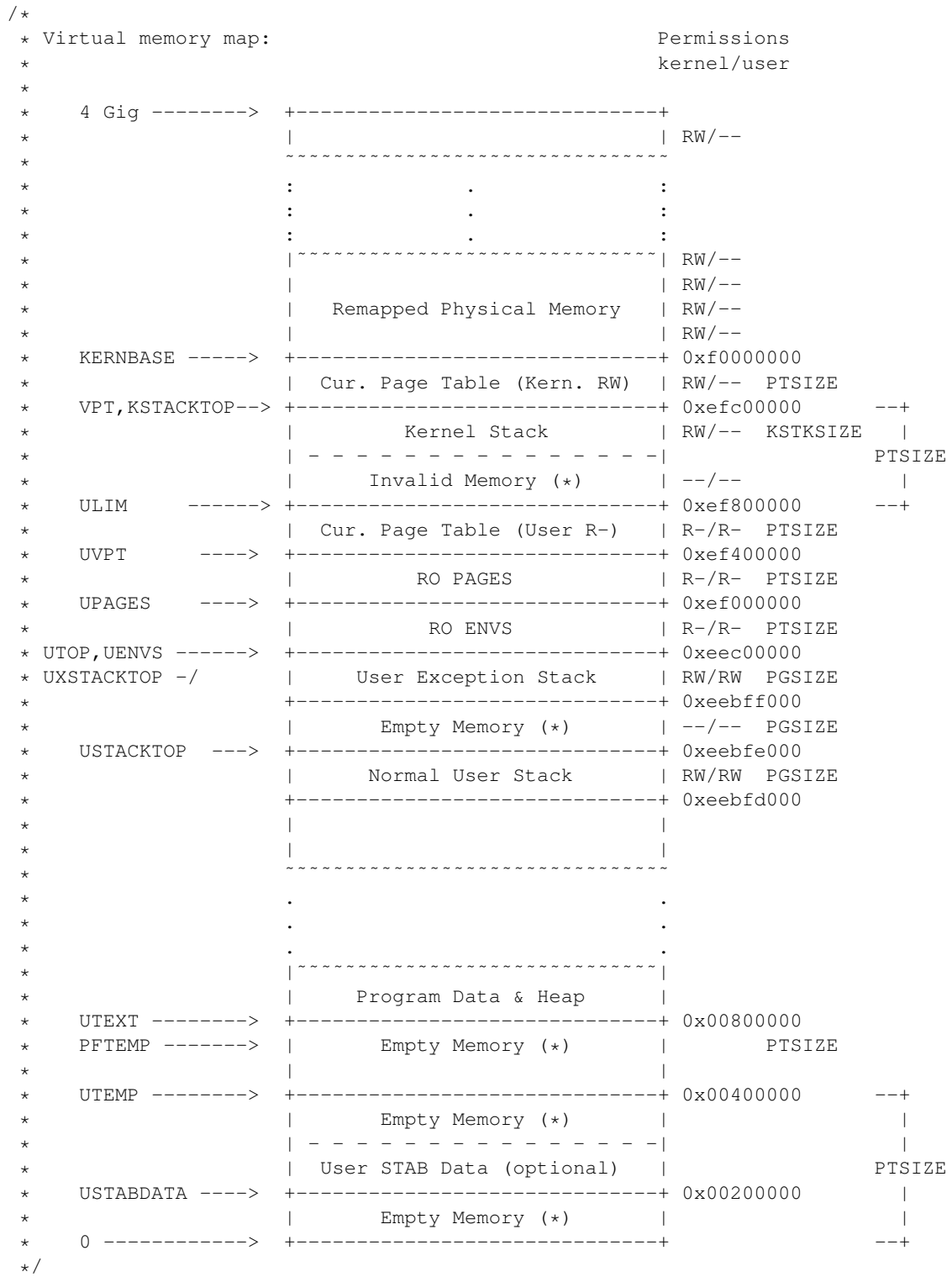
9. [6 points]: In lab 3, the JOS kernel treats user-mode page faults and errors in system calls differently. For page faults, the kernel destroys the user environment, but for system calls that encounter an error, the kernel sets an error code in the %eax register and resumes the environment at the instruction following the `INT`. Explain what would happen if the JOS kernel, when handling a page fault from user space, were to set an error code in %eax and resume the environment at the instruction following the one that caused a page fault.

V Virtual vs. physical memory

Ben wants to add support for a graphics card in JOS. The graphics card provides a 1024-by-1024-pixel display, which appears as 1MB of memory at physical address 0xF0000000, with each byte of that memory range corresponding to one pixel on the screen. The graphics card memory is not part of the physical memory detected by i386_detect_memory().

Assuming pixel value 0 corresponds to black, provide C code for clearing the screen to all-black, to be executed just after JOS turns on paging (i.e. in i386_init() just after i386_vm_init returns). The next page includes a copy of `inc/memlayout.h` for your reference.

10. [8 points]: Your code:


/*
* Virtual memory map:
Permissions
kernel/user
4 Gig
RW/--
Remapped Physical Memory
RW/--
KERNBASE
0xf0000000
Cur. Page Table (Kern. RW)
RW/-- PTSIZE
VPT,KSTACKTOP
0xefc00000
Kernel Stack
RW/-- KSTKSIZE
PTSIZE
Invalid Memory (*)
--/--
ULIM
0xef800000
Cur. Page Table (User R-)
R-/R- PTSIZE
UVPT
0xef400000
RO PAGES
R-/R- PTSIZE
UPAGES
0xef000000
RO ENVS
R-/R- PTSIZE
UTOP,UENVS
0xeec00000
UXSTACKTOP -/
User Exception Stack
RW/RW PGSIZE
0xeebff000
Empty Memory (*)
--/-- PGSIZE
USTACKTOP
0xeebfe000
Normal User Stack
RW/RW PGSIZE
0xeebfd000
Program Data & Heap
UTEXT
0x00800000
PFTEMP
Empty Memory (*)
PTSIZE
UTEMP
0x00400000
Empty Memory (*)
User STAB Data (optional)
PTSIZE
USTABDATA
0x00200000
Empty Memory (*)
0
*/

Name:

VI Kill

Here is process X running on xv6:

```
main()
{
        while(1) {
        }
}
```

As you can see, process X is in an infinite loop. Another process Y uses the kill() system call to terminate process X.

11. [8 points]: For this question, assume the xv6 system has a single CPU. Is it possible for process X to execute any more user-space instructions after process Y's kill() returns? Explain. If "yes," what will finally cause X to stop executing user-space instructions?

12. [8 points]: For this question, assume the xv6 system has two CPUs. Is it possible for process X to execute any more user-space instructions after process Y's kill() returns? Explain. If "yes," what will finally cause X to stop executing user-space instructions?

VII CLI

13. [8 points]: What would go wrong if you replaced pushcli()'s implementation (xv6 sheet 14) with just cli(), and popcli()'s implementation with just sti()?

VIII Fork

Here's a copy of sys_fork() from sheet 28:

```
int
sys_fork(void)
{
  int pid;
  struct proc *np;

  if((np = copyproc(cp)) == 0)
    return -1;
  pid = np->pid;
  np->state = RUNNABLE;
  return pid;
}
```

Explain what could go wrong if the code looked like this:

```
int
sys_fork(void)
{
  struct proc *np;

  if((np = copyproc(cp)) == 0)
    return -1;
  np->state = RUNNABLE;
  return np->pid;
}
```

14. [4 points]: Your answer:

IX 6.828

We'd like to hear your opinions about 6.828, so please answer the following questions. (Any answer, except no answer, will receive full credit.)

15. [2 points]: How could we make the ideas in the course easier to understand?

16. [2 points]: What is the best aspect of 6.828?

17. [2 points]: What is the worst aspect of 6.828?

End of Quiz

Name:

MIT OpenCourseWare
http://ocw.mit.edu

6.828 Operating System Engineering

Fall 2012

For information about citing these materials or our Terms of Use, visit: http://ocw.mit.edu/terms.



Department of Electrical Engineering and Computer Science

MASSACHUSETTS INSTITUTE OF TECHNOLOGY

6.828 Fall 2010

Quiz I

All problems are open-ended questions. In order to receive credit you must answer the question as precisely as possible. You have 80 minutes to finish this quiz.

Write your name on this cover sheet AND at the bottom of each page of this booklet.

Some questions may be harder than others. Read them all through first and attack them in the order that allows you to make the most progress. If you find a question ambiguous, be sure to write down any assumptions you make. Be neat. If we can't understand your answer, we can't give you credit!

THIS IS AN OPEN BOOK, OPEN NOTES EXAM.

Please do not write in the boxes below.

I (xx/24)	II (xx/22)	III (xx/20)	IV (xx/26)	V (xx/8)	Total (xx/100)

Name:

I Concurrency

1. [8 points]: Alyssa runs xv6 on a machine with 8 processors and 8 processes. Each process calls `sbrk` (3351) continuously, growing and shrinking its address space. Alyssa measures the number of `sbrk`s per second and notices that 8 processes achieve the same total throughput as 1 process, even though each process runs on a different processor. She profiles the xv6 kernel while running her processes and notices that most execution time is spent in `kalloc` (2426) and `kfree` (2405), though little is spent in `memset`. Why is the throughput of 8 processes the same as that of 1 process?

2. [8 points]: Does the JOS kernel have a mechanism equivalent to xv6's `swtch` (2308)? If yes, what? If not, explain why xv6 needs it but JOS does not.

3. [8 points]: xv6 enables interrupts in the kernel during system calls and device interrupts, which adds some complexity since xv6 has to carefully disable and enable interrupts when locking. In contrast, JOS (as you will find in Lab 4 Part C) only enables interrupts in user space, and arranges for the hardware to automatically disable interrupts when entering the kernel. Would anything go wrong if xv6 also disabled interrupts in the kernel? Support your claim.

II File systems

Alyssa add the statement:

```
cprintf("bwrite sector %d\n", b->sector);
```

to `bwrite` (4014), as in one of the homeworks. She then types the following two commands and observes their disk writes:

```
$ echo > x
bwrite sector 4      (inode for x)
bwrite sector 4
bwrite sector 29     (directory data for /)
$ echo ab > x
bwrite sector 28     (bitmap)
bwrite sector 508    (file data for x)
bwrite sector 4
bwrite sector 508
bwrite sector 4
bwrite sector 508
bwrite sector 4
```

Alyssa observes that her commands run slowly because each `bwrite` is synchronous (that is, it waits until the disk finishes writing). She recalls from lecture that this is necessary to help `fsck` be able to repair any inconsistencies after a crash or power failure, but naturally wonders if any of the writes could be made asynchronous. An asynchronous write updates the in-memory buffer for the block and marks it dirty, but does not send the buffer to the IDE driver. Instead, a dirty buffered block is written to the disk on a later synchronous write to that block, after a delay, or when buffer space is needed for some other block. Dirty buffered blocks may be written to disk in any order.

4. [10 points]: Assume xv6 does *no* repair of a file system after a reboot (it has no `fsck`). Alyssa starts by thinking about a single aspect of consistency: preventing the same block from being referenced by two different files; she will worry about other aspects later. Which writes during `echo ab > x` above must be synchronous to ensure this does not happen? Provide a brief explanation.

Alyssa goes on to wonder how to extend xv6 to support *named pipes*. A named pipe allows the following shell script:

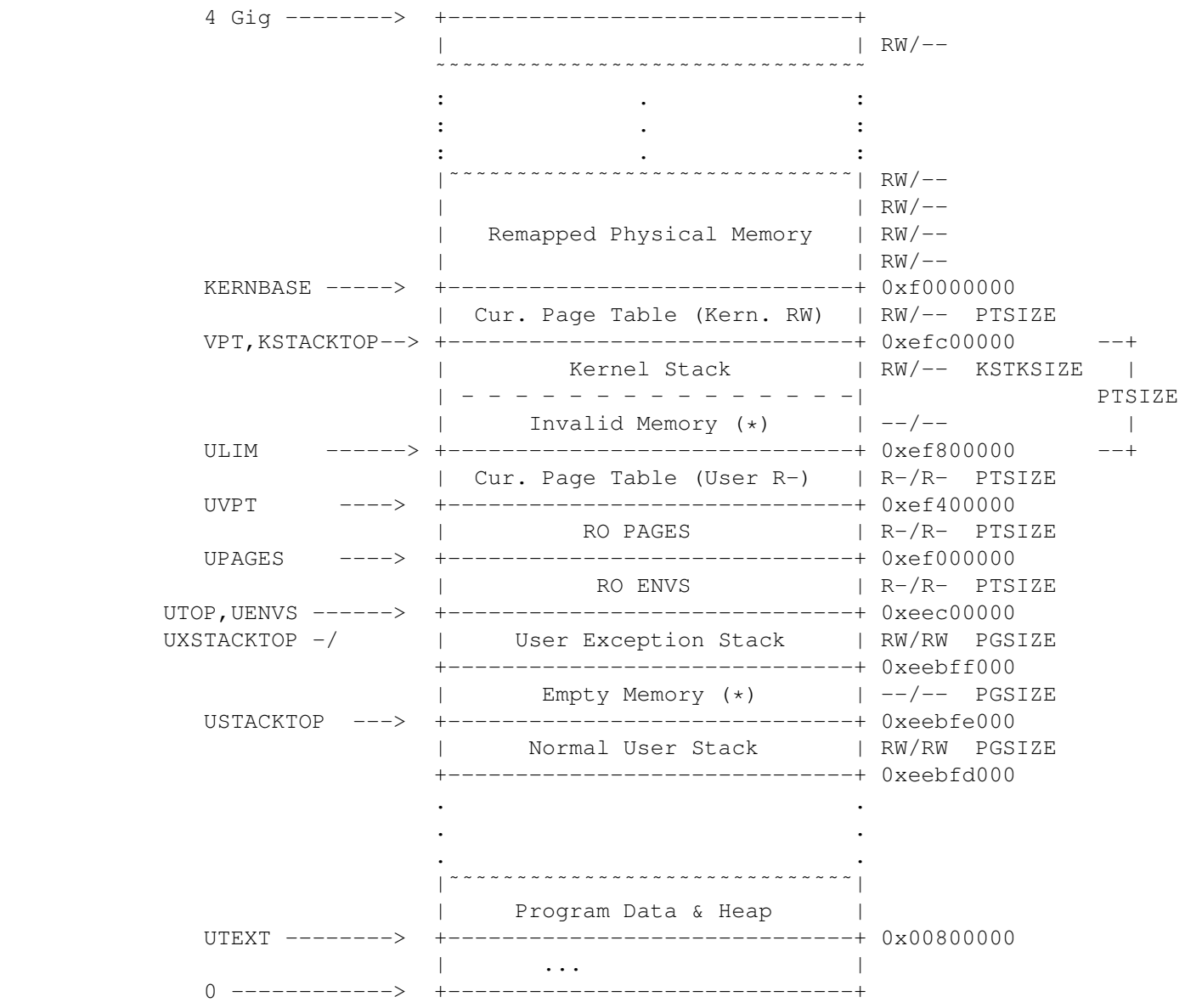
```
$ mkfifo mypipe
$ cat file > mypipe &
$ wc < mypipe
$ rm mypipe
```

This shell code behaves much like `cat file | wc`, except that the pipe is given a regular file name that is stored in the file system, even though the contents of the pipe are not. Just like a regular pipe, the kernel passes all data from the writer to the reader internally, via in-memory buffers (without writing to the disk). Unlike a regular pipe, a named pipe appears in the file system, as a "special file." As a result, the processes communicating over the pipe don't have to have a parent-child relationship, since they can open the pipe by its file name (as in the example).

5. [12 points]: Describe what modifications you would make to xv6 to support named pipes so that you can run the above shell script. You don't have to provide code; just sketch what modifications you would make in which files. (Hint: exploit the existing pipes.)

III Virtual Memory

Here's a simplified version of JOS's virtual memory map from `memlayout.h`.




```
#define KERNBASE 0xF0000000
#define VPT (KERNBASE - PTSIZE)
#define KSTACKTOP VPT
#define ULIM (KSTACKTOP - PTSIZE)
#define UVPT (ULIM - PTSIZE)
#define UPAGES (UVPT - PTSIZE)
#define UENVS (UPAGES - PTSIZE)
#define UTOP UENVS
#define USTACKTOP (UTOP - 2*PGSIZE)
```

6. [8 points]: You have a machine with 4 GB of DRAM (and 32-bit addresses). What new value should you give KERNBASE to allow a single environment to allocate and address the largest number of distinct user memory pages possible? It's okay to be off by a few dozen megabytes. Explain why a KERNBASE higher or lower than your answer would reduce the maximum allocatable environment size.

7. [12 points]: You have the following user-level code in JOS:

```
sys_page_alloc(0, 0x1000000, PTE_P|PTE_W|PTE_U);
*(int*)0x1000000 = 111;
sys_page_alloc(0, 0x2000000, PTE_P|PTE_W|PTE_U);
*(int*)0x2000000 = 222;

// your call(s) to sys_page_map() here

if(*(int*)0x1000000 == 222 &&
   *(int*)0x2000000 == 111)
  cprintf("OK\n");
```

Show how to make this code print `OK` using just one or more calls to

```
int sys_page_map(envid_t srcenvid, void *srcva,
                 envid_t dstenvid, void *dstva,
                 int perm);
```

Before the above code, nothing is mapped between 0x1000000 and the user stack.

IV JOS

Ben Bitdiddle decides to experiment with a new and potentially more flexible system call convention. Rather than passing the syscall number and arguments in registers, he instead writes the syscall number and arguments to a "syscall page" at a new, fixed location `USYSCALL` before invoking `int 0x30`. He defines a `struct SysArgs` for storing the arguments, adds `struct Page *env_syspage` to `struct Env` to record the physical location of an environment's syscall page, and adds the following code to allocate and map the syscall page for each environment:

```
static int
env_setup_vm(struct Env *e) {
  // ...
  page_alloc(&e->env_syspage);
  page_insert(e->env_pgdir, e->env_syspage,
              (void*)USYSCALL, PTE_U|PTE_W);
  return 0;
}
```

8. [10 points]: Ben begins updating his system call implementations to read their arguments from `USYSCALL`. His new convention is working great until he updates his inter-environment message passing syscalls. In his implementation, an environment sends a message by writing the message and its destination environment ID to its syscall page and invoking the kernel, which validates arguments, switches to the destination environment's address space, copies the message from `(struct SysArgs*)USYSCALL`, then switches back to the calling environment's address space. However, Ben finds that this writes the wrong data to the destination environment. Alyssa suggests that Ben should instead read the message from `(struct SysArgs*)page2kva(curenv->env_syspage)`. Explain what was wrong with Ben's approach and why Alyssa's approach fixes Ben's problem.

9. [8 points]: Both the original calling convention and Ben's new scheme use an interrupt to invoke the syscall. What prevents him from following the usual C calling convention from the user environment and simply calling the kernel function that implements the desired syscall?

10. [8 points]: Ben wants to take his syscall convention to a new level, so he adds *syscall batching*. He modifies his user code to store a *sequence* of system calls in the syscall page and modifies the `syscall` function in his kernel to iterate over this page and invoke each function in the sequence before returning to `trap_dispatch`. Louis Reasoner wants to take this for a spin, so he modifies `user/yield.c` to write its sequence of `sys_yield`, `sys_cputs`, `sys_yield`... syscalls to the syscall page before entering the kernel, as follows

```
for (i = 0; i < 5; i++) {
  batch_add(SYS_yield);
  batch_add(SYS_cputs, "Back in environment\n");
}
sys_execute_batch();
cprintf("All done.\n");
```

Much to his surprise, none of the "Back in environment" messages get printed, even though "All done" does. What happened?

V 6.828

We'd like to hear your opinions about 6.828, so please answer the following questions. (Any answer, except no answer, will receive full credit.)

11. [2 points]: This year we posted (mostly) ahead of lecture draft chapters that provide a commentary on xv6. Did you read them? If so, did you find the chapters useful? What should we do to improve them?

12. [2 points]: Are the labs too time consuming, too short, or are they about right?

13. [2 points]: What is the best aspect of 6.828?

14. [2 points]: What is the worst aspect of 6.828?

End of Quiz

Name:

MIT OpenCourseWare
http://ocw.mit.edu

6.828 Operating System Engineering

Fall 2012

For information about citing these materials or our Terms of Use, visit: http://ocw.mit.edu/terms.



Department of Electrical Engineering and Computer Science

MASSACHUSETTS INSTITUTE OF TECHNOLOGY

6.828 Fall 2011

Quiz I

All problems are open-ended questions. In order to receive credit you must answer the question as precisely as possible. You have 80 minutes to finish this quiz.

Write your name on this cover sheet AND at the bottom of each page of this booklet.

Some questions may be harder than others. Read them all through first and attack them in the order that allows you to make the most progress. If you find a question ambiguous, be sure to write down any assumptions you make. Be neat. If we can't understand your answer, we can't give you credit!

THIS IS AN OPEN BOOK, OPEN NOTES EXAM.

Please do not write in the boxes below.

I (xx/28)	II (xx/20)	III (xx/44)	IV (xx/8)	Total (xx/100)

Name:

I General questions

1. [8 points]: Unix's API is carefully designed so that programs compose easily. As an example, `write` and `read` don't take an offset as argument, but instead the kernel maintains an offset for each file descriptor. Give a shell command that illustrates how this unusual API simplifies composing programs and explain briefly why.

2. [12 points]: In xv6, `wakeup` must scan the entire `ptable` to find the processes that are sleeping on the specified channel. Ben proposes to change xv6 to use condition variables to avoid this scan. His condition variables have the following structure:

```
struct condvar {
  struct proc *waiters;
};
```

For each channel there is a separate condition variable, which holds the list of processes waiting on that condition variable. Wakeup wakes up only those processes:

```
void
cv_wakeup(struct condvar *cv)
{
  acquire(&ptable.lock);
  struct proc *p = cv->waiters;
  while (p) {
    struct proc *next = p->cv_next;
    p->cv_next = 0;
    p->state = RUNNABLE;
    p = next;
  }
  cv->waiters = 0;
  release(&ptable.lock);
}
```

(continued on next page)

Complete the implementation of `cv_sleep`. Your implementation should not call `sleep`.

```
void
cv_sleep(struct condvar *cv, struct spinlock *lk)
{
  if(proc == 0)
    panic("sleep");
  if(lk == 0)
    panic("sleep without lk");
  if (lk != &ptable.lock) {
    acquire(&ptable.lock);
    release(lk);
  }









  if (lk != &ptable.lock) {
    release(&ptable.lock);
    acquire(lk);
  }
}
```

3. [8 points]: `swtch` in xv6 doesn't explicitly save and restore all fields of **`struct`** `context`. Why is it okay that `swtch` doesn't contain any code that saves `%eip`?

II File systems

4. [10 points]: Albert has a computer running xv6. He has written his own application, which forks many processes, all of which write large files. Every once in a while, Albert's computer panics with "bget: no buffers" (line 3946). He thinks to himself that, since buffers are only used for short periods of time, it would be better for `bget()` to sleep until a buffer is free. So he changes the panic at the very end of `bget()` to:

```
sleep(&bcache, &bcache.lock); // new code
goto loop;                    // new code
```

And adds a wakeup before the release on the last line of `brelse()`:

```
wakeup(&bcache);         // new code
release(&bcache.lock);   // existing code
```

After this change, Albert's computer occasionally stops working: no panic, but no forward progress either. He calls `procdump()` from `gdb` and sees that every process (other than process 1) is waiting in his new sleep in `bget()`; the complete stack for each process is:

```
Function  Line
sleep     2525
bget      the new call to sleep
bread     3956
readsb    4282
balloc    4311
bmap      4629
writei    4769
filewrite 5176
sys_write 5285
syscall   3283
```

Process 1 is sleeping in `wait()`. Albert's computer has only one core. Albert is running a modified version of xv6 with logging disabled: `log_write()` just calls `bwrite()`, and `begin_trans()` and `end_trans()` do nothing.

Explain why the processes never wake up from the sleep in `bget()`. (Hint: look at `bmap`.)

5. [10 points]: Albert decides to take a different approach: in order to avoid running out of buffers, he'll allocate them with `kalloc()` as needed. Here's his new `bget()` and `brelse()`:

```
static struct buf*
bget(uint dev, uint sector)
{
  struct buf *b;
  b = (struct buf *) kalloc();
  b->dev = dev;
  b->sector = sector;
  b->flags = B_BUSY;
  b->prev = b->next = b->qnext = 0;
  return b;
}

void
brelse(struct buf *b)
{
  kfree((void*) b);
}
```

Again, Albert's computer has only one core, and has logging disabled.

Albert's computer seems to work for a while, but eventually panics with "freeing free block" from `bfree()` at line 4342.

Explain what is going on.

III JOS memory layout and traps

Currently, the JOS kernel reserves part of every user environment's address space for itself. Ben Bitdiddle (who fights for the user environments) decides to modify his lab 3 implementation so user environments have control over (nearly) their entire 4GB virtual address space.

6. [8 points]: In regular JOS, where the kernel and user code share the same address space, what prevents user code from reading or writing sensitive kernel data?

Ben separates the user and kernel address spaces. He constructs the kernel page directory, `kern_pgdir`, like usual, but modifies `env_setup_vm` to create a nearly empty page directory for each new environment instead of mapping the kernel in to each environment.

Ben's plan is to switch to `kern_pgdir` whenever there's a trap. Unfortunately, the x86 provides no convenient way to switch address spaces when taking a trap. Ben solves this with a *trap trampoline*. He reserves a few pages for kernel use at the top of each environment's address space. He puts all of his `trapentry.S` code (`TRAPHANDLER` declarations and `alltraps`) on one of these pages so that the CPU can jump to the trap handler entry point without switching address spaces.

7. [8 points]: Ben also maps the kernel stack in to each user environment. Why is this necessary?

Ben modifies `alltraps` (which runs from the trampoline page) to switch to the kernel page directory before calling the `trap` (up to the `lcr3`, this should be equivalent to the `alltraps` you wrote):

```
_alltraps:
  pushl %ds               # Build trap frame
  pushl %es
  pushal
  movl $GD_KD, %eax       # Load kernel segments
  movw %ax, %ds
  movw %ax, %es
  pushl %esp
  lcr3 PADDR(kern_pgdir)  # NEW: Switch to kernel address space
  mov $trap, %eax         # Call trap
  call *%eax
```

8. [10 points]: Louis Reasoner thinks doing `lcr3 PADDR(kern_pgdir)` from `alltraps` won't work because, in Ben's design, `kern_pgdir` isn't accessible in the user environment's address space. In fact, it *will* load the kernel page directory. Why?

9. [8 points]: When Ben runs his code, he finds that it executes the `lcr3`, but fails to execute the next instruction (it never sets `%eax` to the address of `trap`). Why and how can he fix this?

10. [10 points]: Ben fixes this problem and then discovers that he needs to rewrite `sys_cputs` because it can no longer read its string argument directly from the current address space. Describe how Ben can fix `sys_cputs`.

IV 6.828

We'd like to hear your opinions about 6.828, so please answer the following questions. (Any answer, except no answer, will receive full credit.)

11. [2 points]: This year we posted the complete draft of the xv6 commentary at the beginning of the semester. Did you find the chapters useful? What should we do to improve them?

12. [2 points]: This year we introduced code reviews. Did you find them useful? What should we do to improve them?

13. [2 points]: What is the best aspect of 6.828?

14. [2 points]: What is the worst aspect of 6.828?

End of Quiz

Name:

MIT OpenCourseWare
http://ocw.mit.edu

6.828 Operating System Engineering

Fall 2012

For information about citing these materials or our Terms of Use, visit: http://ocw.mit.edu/terms.



Department of Electrical Engineering and Computer Science

MASSACHUSETTS INSTITUTE OF TECHNOLOGY

6.828 Fall 2012

Quiz I

All problems are open-ended questions. In order to receive credit you must answer the question as precisely as possible. You have 80 minutes to finish this quiz.

Write your name on this cover sheet AND at the bottom of each page of this booklet.

Some questions may be harder than others. Read them all through first and attack them in the order that allows you to make the most progress. If you find a question ambiguous, be sure to write down any assumptions you make. Be neat. If we can't understand your answer, we can't give you credit!

THIS EXAM IS OPEN BOOK AND OPEN LAPTOP, but CLOSED NETWORK.

Please do not write in the boxes below.

I (xx/11)	II (xx/20)	III (xx/16)	IV (xx/24)	V (xx/8)	VI (xx/16)	VII (xx/5)	Total (xx/100)

Name:

I Shell

The xv6 shell creates new processes to run a command as follows:

```
if(fork1() == 0)
  runcmd(parsecmd(buf));
wait();
```

1. [5 points]: It is possible that the child process that runs the command exits before the parent calls `wait`. Give a scenario in which that can happen.

2. [6 points]: How does xv6 ensure that `wait` returns correctly even if the child already has exited before the parent called `wait`?

II Locks

In the in-class exercises on locking you modified a hash table to work correctly under concurrent lookup and insert by adding `acquire` and `release` statements for a global lock. The code we handed out for `put` is as follows:

```
static void
insert(int key, int value, struct entry **p, struct entry *n)
{
  struct entry *e = malloc(sizeof(struct entry));
  e->key = key;
  e->value = value;
  e->next = n;
  *p = e;
}

static
void put(int key, int value)
{
  struct entry *n, **p;
  for (p = &table[key%NBUCKET], n = table[key % NBUCKET];
                       n != 0; p = &n->next, n = n->next) {
    if (n->key > key) {
      insert(key, value, p, n);
      goto done;
    }
  }
  insert(key, value, p, n);
 done:
  return;
}
```

3. [4 points]: In your solutions for the in-class exercise, you added in the above code `acquire` and `release` calls to ensure correct operation of concurrent `put` invocations. Indicate where in the above code they should be added.

To avoid the overhead of locks, Alyssa suggests use of atomic operations, which are guaranteed to perform their multiple step operation as a single atomic operation. In particular, Alyssa suggests use of the following atomic operation:

```
bool __sync_bool_compare_and_swap (type *ptr, type oldval, type newval)
```

This builtin function performs an atomic compare and swap. If the current value of `*ptr` is `oldval`, it writes `newval` into `*ptr`, and returns `true`. Otherwise it returns `false`. The compiler turns the builtin function into the atomic instruction sequence that's appropriate for the target architecture; on the x86, it might use `lock cmpxchg`.

An example:

```
char *p;
  ...
  if(__sync_bool_compare_and_swap(&p, 0, 1))
    ....  // success: p was set to 1
  else
    ...   // failure: p was not set to 1, because p wasn't equal to 0
```

This code will set `p` to `1` only if `p` previously held `0`. If two cores execute the code at the same time, and `p` held `0`, only one of the two cores will see a `true` return value from the call.

4. [8 points]: Complete the `put` below using `__sync_bool_compare_and_swap` so that concurrent invocations will run correctly without using locks:

```
static
void put(int key, int value)
{
  struct entry *n, **p;
  struct entry *e = malloc(sizeof(struct entry));
  e->key = key;
  e->value = value;




  for (p = &table[key%NBUCKET], n = table[key % NBUCKET]; n != 0;
                                        p = &n->next, n = n->next) {



    if (n->key > key) {









    }
  }




 done:
  return;
}
```

5. [8 points]: For the lock-free implementation, give an example of a workload for which `put`'s throughput scales linearly with an increasing number of cores.

III Logical Logging

Alyssa is unhappy that xv6 logging writes an entire sector to the log even if the file system only needs to modify a few bytes in the sector. She observes that log records would be much shorter if they described the operation to be performed, rather than containing the entire modified sector. Alyssa modifies the xv6 logging system to use this idea, called "logical logging." For example, when her `balloc()` allocates a data block, it logs the setting of the bit in the block allocation bitmap with a "set bit" record. A "set bit" record contains a constant indicating the record type, and the block number that was allocated. Alyssa's recovery code for this type of log record sets the relevant bit in the block allocation bitmap on disk. The "set bit" record takes up only a few bytes in the log, rather than an entire sector.

While the "set bit" record works, it turns out that correct recovery from logical log records is not possible for some operations.

6. [8 points]: Alyssa defines a "create file" log record containing the parent directory i-number and the new file's name. Recovery for this kind of record calls a version of create() (sysfile.c) that does everything but the initial nameiparent(), and that uses bwrite for all modifications. Explain why use of this "create file" log record will incorrectly handle some failure scenarios that the current xv6 handles correctly.

7. [8 points]: Consider a "delete directory entry" log record that contains the parent i-number and the entry's name. Recovery finds the named entry in the parent directory and overwrites it with nulls. Explain if this works; if not, briefly explain why not.

IV Page Tables

Consider the following 32-bit x86 page table setup.

`%cr3` holds 0x00001000.

The Page Directory Page at physical address 0x00001000:

```
PDE 0: PPN=0x00002, PTE_P, PTE_U, PTE_W
PDE 1: PPN=0x00003, PTE_P, PTE_U, PTE_W
PDE 2: PPN=0x00002, PTE_P, PTE_U, PTE_W
... all other PDEs are zero
```

The Page Table Page at physical address 0x00002000 (which is PPN 0x00002):

```
PTE 0: PPN=0x00005, PTE_P, PTE_U, PTE_W
PTE 1: PPN=0x00006, PTE_P, PTE_U, PTE_W
... all other PTEs are zero
```

The Page Table Page at physical address 0x00003000:

```
PTE 0: PPN=0x00005, PTE_P, PTE_U, PTE_W
PTE 1: PPN=0x00005, PTE_P, PTE_U, PTE_W
... all other PTEs are zero
```

8. [8 points]: List all virtual addresses that map to physical address 0x00005555.

9. [8 points]: Given the above page table, what virtual address (if any) should the kernel store to in order to modify the mapping for virtual address 0x402000 (that's 4*1024*1024 + 8192)?

10. [8 points]: Explain exactly what the above page table should contain in order to to allow the kernel to write to the page directory page. There is more than one answer; you only have to explain one.

.

V JOS memory mapping and fork

Here is a code snippet from Ben Bitdiddle's lab 4. Ben is struggling to get user-mode `fork` to work. In his `fork`, Ben invokes `duppage` for every virtual page number (`vpn`) below `UTOP` that is present in the current environment's address space, except for the exception stack.

```
static int duppage(envid_t envid, unsigned vpn) {
    void *addr = (void *)(vpn << PGSHIFT);
    pte_t pte = uvpt[vpn];
    int r;
    // if the page is just read-only, just map it in.
    if (!(pte & (PTE_W|PTE_COW))) {
        if ((r = sys_page_map(0, addr, envid, addr, pte & PTE_SYSCALL)) < 0)
            panic("sys_page_map␣in␣child␣failed:␣%e", r);
        return 0;
    }
    if ((r = sys_page_map(0, addr, 0, addr, PTE_P|PTE_U|PTE_COW)) < 0)
        panic("sys_page_map␣in␣parent␣failed:␣%e", r);
    if ((r = sys_page_map(0, addr, envid, addr, PTE_P|PTE_U|PTE_COW)) < 0)
        panic("sys_page_map␣in␣child␣failed:␣%e", r);
    return r;
}
```

11. [8 points]: Ben Bitdiddle could not pass the `forktree` test because the above code has a bug. While debugging, he printed out the value of `uvpt[vpn]` at the end of the function, and found that the parent environment's user stack was not marked copy-on-write. Explain why this is happening, and describe how to fix the bug. You should assume that his other code is free of bugs.

VI Exokernel `exec`

Ben Bitdiddle has chosen to implement UNIX-style `exec` as his challenge exercise for lab 5. `exec` replaces the current environment's memory with the desired executable, sets up a stack, and jumps to the entry point of the executable.

Alyssa P. Hacker proposes the following design for `exec` in the user-space library operating system:

1. Unmap all pages in the current environment except for the stack and the code for `exec`.
2. Map in the pages (loaded from the ELF file on disk) for the new environment.
3. Unmap the code page for `exec`.
4. Jump to the new environment's entry point.

12. [8 points]: Ben Bitdiddle sees a problem with Alyssa's first and second steps. What might go wrong? Assume the code for `exec` fits in a single page and doesn't make any function calls (other than system calls).

13. [8 points]: There are a few other problems with Alyssa's plan. Please describe a new plan for a working `exec` in user space. Your solution shouldn't modify the kernel.

VII 6.828

We'd like to hear your opinions about 6.828, so please answer the following questions. (Any answer, except no answer, will receive full credit.)

14. [2 points]: This year we posted an improved draft of the xv6 commentary at the beginning of the semester. Did you find the chapters useful? What should we do to improve them?

15. [2 points]: This year we continued code reviews and added rebuttals. Did you find them useful? What should we do to improve them?

16. [1 points]: Did you find the in-class exercises useful? How would you suggest improving them?

End of Quiz

Name:

MIT OpenCourseWare
http://ocw.mit.edu

6.828 Operating System Engineering

Fall 2012

For information about citing these materials or our Terms of Use, visit: http://ocw.mit.edu/terms.



Department of Electrical Engineering and Computer Science

MASSACHUSETTS INSTITUTE OF TECHNOLOGY

6.097 Operating System Engineering: Fall 2002

Quiz II

All problems are open-ended questions. In order to receive credit you must answer the question as precisely as possible. You have 80 minutes to answer this quiz.

Write your name on this cover sheet AND at the bottom of each page of this booklet.

Some questions may be much harder than others. Read them all through first and attack them in the order that allows you to make the most progress. If you find a question ambiguous, be sure to write down any assumptions you make. Be neat. If we can't understand your answer, we can't give you credit!

THIS IS AN OPEN BOOK, OPEN NOTES QUIZ.

1 (xx/20)	2 (xx/20)	3 (xx/20)	4 (xx/20)	5 (xx/20)	Total (xx/100)

Name: Alyssa P. Hacker

I Fast mutual exclusion

A common way to implement concurrent inserts is using an atomic `TSL` (Test-Set-and-Lock) instruction:

```
List *list = NULL;
int insert_lock = 0;

insert(int data) {

  /* acquire the lock: */
  while(TSL(&insert_lock) != 0)
    ;

  /* critical section: */
  List *l = new List;
  l->data = data;
  l->next = list;
  list = l;

  /* release the lock: */
  insert_lock = 0;
}
```

Many processors support an atomic `cmpxchg` instruction. This instruction has the following semantics:

```
int cmpxchg(addr, v1, v2) {
  int ret = 0;
  // stop all memory activity and ignore interrupts
  if (*addr == v1) {
    *addr = v2;
    ret = 1;
  }
  // resume other memory activity and take interrupts
  return ret;
}
```

1. **[5 points]:** Give an implementation of `insert` using the `cmpxchg` instruction.

```
insert (int data) {
  List *l = new List;
  l->data = data;
  do {
    l->next = list;
  } while (!cmpxchg (&list, l->next, l));
}
```

2. **[5 points]:** Why is the implementation of `insert` with `cmpxchg` preferable over the one with `TSL`?

The `cmpxchg` version is wait free. There is no chance for a deadlock because there are no locks.

Name:

3. [5 points]: Give an implementation of `cmpxchg` using a restartable atomic sequence.

```
int cmpxchg_RAS(addr, v1, v2) {
  int ret = 0;
  BEGIN RAS
  if (*addr == v1) {
    *addr = v2;
  END RAS
    ret = 1;
  }
  return ret;
}
```

4. [5 points]: Can you extend the `insert()` routine to insert nodes into a doubly linked list using `cmpxchg` and without using locks? If so, give the code, otherwise explain why not.

```
struct List {
   List *next;
   List *prev;
   int data;
};
```

You can not just use `cmpxchg` to insert into a doubly linked list because the insert into list operations:

```
insert(int data) {
  // Prepare new element
  List *l = new List;
  l->data = data;
  l->prev = NULL;

  // Insert into list
  l->next = list;
  list->prev = l;
  list = l;
}
```

require two externaly visible memory writes. Even if `cmpxchg` is used for the first externally visible write, the process may be descheduled/interrupted before the second write. The first write will still be globally visible and would lead to an inconsistent list if another insert occurred.

Name:

II Soft updates

The pseudocode for the flusher described in the soft-updates paper is as follows (b is a block):

```
flushblock (b)
{
  lock b;
  for all dependencies that b depends on
    "remove" that dependency by undoing the change to b
    mark the dependency as "unrolled"
  write b
}

write_completed (b) {
  remove dependencies that depend on b.
  reapply "unrolled" dependencies that b depended on
  unlock b
}
```

Ben proposes another implementation. Instead of unrolling dependencies, Ben's implementation removes the dependencies by first flushing the block b' that b depends on and the blocks that b' depends on:

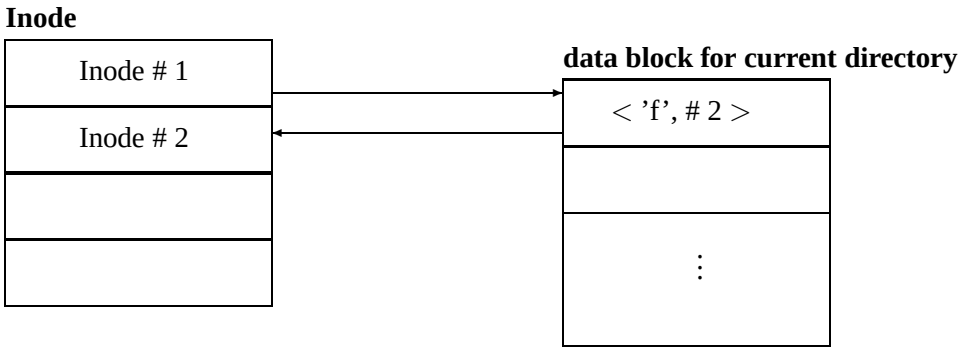
```
flushblock (b)
{
  lock b;
  for each block b' that b depends on
      flushblock (b');
  write b
}

write_completed (b) {
  unlock b
}
```

5. [5 points]: Draw a set of inode and data blocks and their dependencies that show that Ben's code is wrong. Why is Ben's code wrong?

Dependencies exist in the filesystem at a finer granularity than blocks. For example, multiple directory entries exist in a single block. As such, a circular dependency can exist between two blocks. However, since the flusher can only flush entire blocks, the presence of a circular dependency will result in an infinite loop in Ben's code.

One possible picture is taken from the paper (Figure 1). An alternative is given below:



6. [5 points]: List a sequence of system calls that can lead to the drawing that you provided as an answer to the previous question.

Assume a process executes the following calls in a single empty directory d:

```
creat('f'); // directory data block depends on file inode block
            // directory inode block depends on directory data block
```

Suppose f and d have inodes on the same block.

Name:

Ben proposes yet another implementation. This implementation removes the dependencies by first writing (instead of flushing) the block that b is depending on:

```
flushblock (b)
{
  lock b;
  for each block b' that b depends on
      write (b');
  write b
}

write_completed (b) {
  unlock b
}
```

7. [5 points]: Give a timeline of system calls and failures that results in incorrect behavior with this last implementation.

Any sequence of events where there is a circular dependency will result in such a case. Imagine:

```
user:   creat('a') // as above
kernel: flush (b)  // b is inode block of 'a' and cwd
          write b' // b' is data block of cwd
       CRASH
         write b   // never happens!
```

The block b' with its data that depends on b will be incorrectly written on disk.

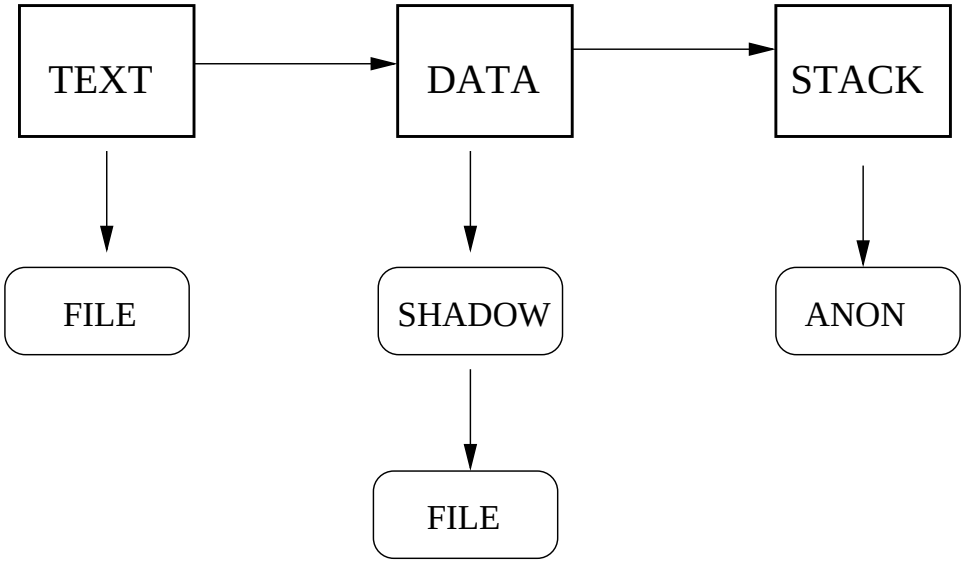
Name:

III Mach VM system

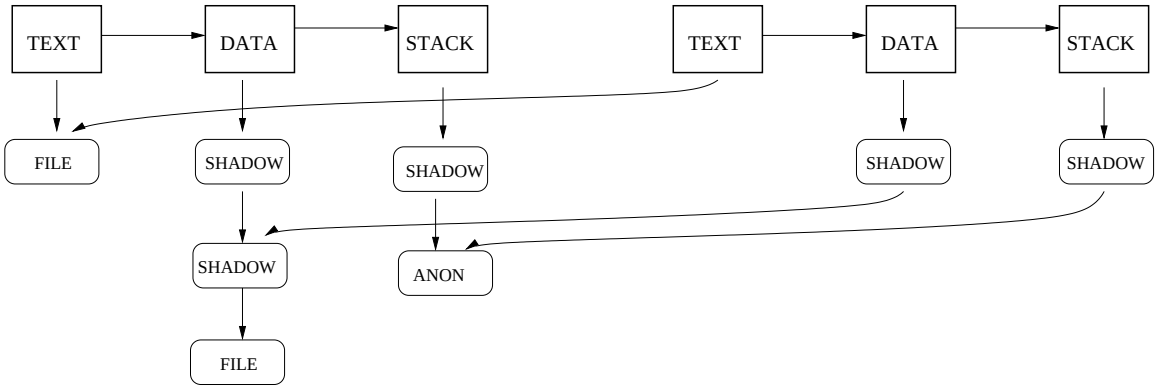
Assume a UNIX process that is running in an address space with 3 objects: one object for the read-only text of the program "a.out", one read-write, private data object (initialized from "a.out"), and one object for a read/write stack.

8. [10 points]: Draw a picture that shows the **vm_map_entries** for this process, and to which objects they are pointing. Give a brief justification for your picture.

(Hint: include the shadow objects)



9. [10 points]: Now "a.out" forks. Draw a picture that shows the **vm_map_entries** for both processes, and to which objects they are pointing. Give a brief justification for your picture.



Name:

IV Virtualizing traps and interrupts in the x86

In a hosted virtual machine monitor, a *host operating system* serves runs the VMM as an application (process) with full privileges (e.g. ring 0 on the x86). The VMM then runs the *guest operating system* inside a virtual machine. In order to do so successfully, the VMM must employ a variety of techniques to prevent the guest from discerning that it is in a virtual machine.

Ben is working on a hosted x86 virtualizer but he can't figure out how to virtualize interrupts and traps. Can you come up with a solution? **(Hint: read all questions in this section first to better understand the issues involved.)**

10. [5 points]: What IDT should the CPU use when it is executing instructions of the guest operating system? Is it the IDT of the host, the guest, or something else constructed by the VMM? Explain.

Here is one possible answer. We assume that there is no special support in the kernel besides allowing the VMM to load code that will run at ring 0. (Another possibility might be to have an exo-kernel style host operating system that allows user programs to hook into receiving notification about generic system traps.)

While the guest operating system is running, the VMM will have installed a special IDT installed that is neither the host or the guest's. This is necessary to allow the VMM to handle CPU virtualization related exceptions correctly.

11. [5 points]: What state should the VMM maintain in order to virtualize interrupts and traps? When is this state updated?

In addition to the general CPU state that must be maintained for regular CPU virtualization (e.g. general purpose registers), the VMM must maintain state about both the host operating system IDT, which it will save when it switches out the IDT before starting the guest OS, and the guest OS's desired IDT, which it will save when the guest runs `lidt`. In the case of the guest, the VMM should probably only keep a pointer to where the guest is keeping to IDT in order to avoid having to trap when the guest writes to the page where the IDT is stored.

12. [5 points]: Interrupts can be generated by hardware or by instructions that are being executed by the guest. How should the VMM handle hardware interrupts? When does the guest operating system receive an interrupt? **(Hint: The VMM relies on the host for actual device driver support.)**

Under this scheme, the VMM should immediately forward all hardware interrupts back to the host operating system. The trap frame will allow the VMM to quickly save the guest operating system's basic state, and then the other CPU state should be saved as well. Then the VMM should place the trap on the host operating system's TSS and jump the host OS's trap handler.

The guest operating system will only receive an interrupt when the VMM decides there is some sort of interrupt that the guest should get. For example, the VMM may choose to forward clock interrupts on to the guest operating system. For device I/O, the VMM will create interrupts for the guest when it receives notification from the host operating system that there is data to be provided to the guest.

Note! Calls to `outb` from the guest will result in traps and these will be emulated by the VMM with read/write calls to the host OS. The VMM should not attempt to determine if a particular interrupt is "for" the guest. This would only apply if a device were completely dedicated to the guest OS and the host OS was not allowed to use it at all.

13. [5 points]: How should the VMM handle traps that are generated by the guest (e.g. intentionally, by a system call or indirectly, as a result of the CPU virtualization)?

Errors like divide by zero or break point interrupts should be directly forwarded back into the guest operating system, using knowledge of the guest's IDT.

CPU virtualization traps and system calls will appear as GPFs since the VMM IDT is configured not to allow the `int` instruction to generate any interrupts from ring 3 or execute any privilegd instructions. The error code and instruction will allow the VMM to decide which of these two cases occurred. If the problem is the result of CPU virtualization (e.g. `lidt` executed in guest), the VMM must emulate the instruction and then continue. Otherwise, the VMM should simply forward the trap on to the guest.

End of Quiz II—-Thanks and enjoy a well-deserved break!

Name:

MIT OpenCourseWare
http://ocw.mit.edu

6.828 Operating System Engineering
Fall 2012

For information about citing these materials or our Terms of Use, visit: http://ocw.mit.edu/terms.



Department of Electrical Engineering and Computer Science

MASSACHUSETTS INSTITUTE OF TECHNOLOGY

6.828 Operating System Engineering: Fall 2006

Quiz II

All problems are open-ended questions. In order to receive credit you must answer the question as precisely as possible. You have 80 minutes to answer this quiz.

Write your name on this cover sheet AND at the bottom of each page of this booklet.

Some questions may be much harder than others. Read them all through first and attack them in the order that allows you to make the most progress. If you find a question ambiguous, be sure to write down any assumptions you make. Be neat. If we can't understand your answer, we can't give you credit!

THIS IS AN OPEN BOOK, OPEN NOTES QUIZ.

In addition to this test booklet, you should have received a copy of selected files from the JOS Lab 6 distribution, which you may use for reference while answering the quiz questions. Please do not write any answers in those pages; that packet is yours to keep.

1 (xx/10)	2 (xx/10)	3 (xx/20)	4 (xx/20)	5 (xx/20)	6 (xx/15)	7 (xx/5)	Total (xx/100)

Name: Solution Set

Quiz score statistics: median 81, mean 80, maximum 98.

I Papers

1. [10 points]: True or false?

A. T / F : According to Liedtke in "Improving IPC performance by kernel design" a designer should exploit processor-specific features to get better IPC performance, sacrificing portability.

B. T / F : Disco (as described in "Disco: Running Commodity Operating Systems on Scalable Multiprocessors") pays a high overhead for TLB faults when running IRIX because the IRIX kernel occupies more TLB entries when running on Disco instead of on the physical hardware.

C. T / F : On a symmetric multiprocessor with a small number of processors and low contention, the time for acquiring an Mellor-Crummey & Scott (MCS) lock (as described in the "Algorithms for scalable synchronization on shared-memory multiprocessors") will be lower than a test-and-set lock with exponential back-off.

D. T / F : The methods described in "Bugs as Deviant Behavior: a general approach to inferring errors in systems code" can reliably determine which line of code contains an error.

E. T / F : Adding IPv6 support to Plan 9 (as described in "Plan 9 from Bell Labs") would require recompiling all the network client programs (telnet, ssh, etc.) to be IPv6-aware.

A. True. Liedtke strongly advocates to specialize the kernel implementation to exploit features of the underling hardware, in this case the x86. One x86 feature used is the page directory for fast temporary mappings.

B. True. Irix on Disco will incur more TLB faults than Irix on the physical hardware, because under Disco kernel addresses must be translated while on the physical hardware the kernel is in an untranslated part of the memory.

C. False. The test-and-set lock takes fewer instructions on acquire, and as can been seen in the performance figures it incurs a lower delay than MCS lock under low contention.

D. False. For MAYBE errors the approach cannot identify a specific line for an error. The approach reports only that there are inconsistencies.

E. False. Because Plan 9 treats network addresses as uninterpreted strings and relies on the connection server to translate names to connection instructions, network clients do not need to be rebuilt to add a new protocol.

II Memory-mapped files

Many operating systems support memory-mapped files. This feature allows a process to map a file into its address space and read and write the file using ordinary memory operations (e.g., `mov` instructions) instead of special `read` and `write` system calls.

The prototype for `mmap` and `munmap` from BSD Unix is:

```
mmap(void *addr, size_t len, int prot, int flags, int fd, off_t off);
munmap(void *addr, size_t len);
```

`Mmap` maps the *len* bytes of pages starting at *addr* so that they correspond to the *len* bytes starting at offset *off* in the file specified by the descriptor *fd*. The additional arguments *prot* and *flags* specify permissions and additional behaviors beyond the scope of this question. For this question, you may assume that *addr*, *len*, and *off* are multiples of the hardware page size and that *prot* and *flags* are set to create a read-write mapping where changes made to the mapped memory are eventually written back to the underlying file.

`Munmap` is the inverse of `mmap`: it flushes any modified memory pages back to the underlying file and then removes the mapping from the specified address range. Subsequent references to that address range will fault.

2. [5 points]: Describe how you would implement memory-mapped files in JOS. Describe the changes, if any, that would be necessary in the library operating system, in the buffer cache, and in the file server. Sketch pseudocode for the implementation of the `mmap` and `munmap` functions. (Hint: the implementations are very simple.)

Since JOS already maps its files into memory, the changes can be implemented entirely in the library operating system by copying the existing file mapping to the desired address.

```
mmap(addr, len, prot, flags, fd, off):
        for(i=0; i<len; i+=4096)
                sys_page_map(0, INDEX2DATA(fd)+off+i, 0, addr+i,
                             PTE_P|PTE_U|PTE_W)

munmap(addr, len)
        for(a=addr; a<addr+len; a+=4096)
                if(page at a is dirty)
                        dirty the underlying file page
                sys_page_unmap(0, a)
```

Name:

3. [5 points]: Can Multics applications read and write files using memory operations? If so, how is that implemented? (Answer this question at the high level; you don't have to go into great detail why not or how.)

Multics's segments, which are its equivalent of files, are only ever accessed by using memory operations. The operating system pages in sections of disk on demand.

III IPC and scheduling (thanks to Eddie Kohler)

4. [10 points]: Ben Bitdiddle is building a multi-environment application on JOS. Environments E1–En mostly send messages to environment E0. E0 generally processes these messages right away, but it must occasionally perform a lot of computation on a message, which delays its receiving the next messages. Ben expects E0 to take the whole CPU during this computation—after all, E1–En can't do anything useful during that time, since E0 isn't blocked in `sys_ipc_recv`. But Ben notices that E0 doesn't get as much CPU as it should. Instead, E1–En keep running, trying to send messages to E0. "This reminds me of receive livelock!" muses Ben: "I need a better IPC implementation."

Ben adds a new flag to the `sys_ipc_try_send` system call. If the flag is set, and the receiving process isn't ready, the sending process will block until the message is sent.

```
struct Env { ...
        bool env_ipc_recving;           // env is blocked receiving
        envid_t env_ipc_sending_to;     // env is blocked sending to this env
        ...
};
#define ENV_IPC_TRY_SEND_BLOCKED 3

static int
sys_ipc_try_send(envid_t envid, uint32_t value, void *srcva, unsigned perm,
                 int block)
{
        int r;    struct Env *e;   ...

        if ((r = envid2env(envid, &e, 0)) < 0)
                return r;
        if (!e->env_ipc_recving) {
                if (block) {
                        curenv->env_ipc_sending_to = e->env_id;
                        curenv->env_status = ENV_IPC_TRY_SEND_BLOCKED;
                        // just in case someone wakes us up early
                        // with sys_env_set_status():
                        curenv->env_tf.tf_regs.reg_eax = -E_IPC_NOT_RECV;
                        sched_yield();  // does not return
                }
                return -E_IPC_NOT_RECV;
        }

        ... rest of code as before ...
}
```

Name:

```
static int
sys_ipc_recv(void *dstva)
{
        ... ensure that !curenv->env_ipc_recving, as before ...
        ... check dstva, as before ...

        curenv->env_ipc_recving = 1;
        curenv->env_ipc_dstva = dstva;

        // look for an environment blocked
        for (i = 0; i < NENV; i++)

                if (envs[i].env_status == ENV_IPC_TRY_SEND_BLOCKED &&
                    envs[i].env_ipc_sending_to == curenv->env_id
                    _____________________________________________) {
                        // envs[i] has something to send to us!
                        // Mark it runnable, make it send to us,
                        // and if the send succeeds,
                        // return 0 from sys_recv.
                        struct Env *save_curenv = curenv;
                        curenv = &envs[i];

                        struct trapframe *tf = &curenv->env_tf;
                        int r = sys_ipc_try_send(tf->tf_edx, tf->tf_ecx,
                                                 tf->tf_ebx, tf->tf_edi,
                                                 tf->tf_ebx);
                        tf->tf_eax = r;
                        curenv->env_status = ENV_RUNNABLE;
                        if (r < 0) {
                          curenv = save_curenv;
                          continue;
                        }

                        ____________________________________________
                        curenv = save_curenv;
                        return 0;
                }

        // Otherwise, block as before
        ... rest of code as before ...
}
```

Fill in the blanks in Ben's `sys_ipc_recv` function. *Hint:* How will you make `envs[i]` send to `curenv`? Can you reuse `sys_ipc_try_send`? Remember that the five arguments to a system call are passed in registers `%edx`, `%ecx`, `%ebx`, `%edi`, and `%esi`, respectively. (Register `%eax` is used for the system call number.)

(The blanks are filled in in italics.)

Name:

Ben tries his system and observes that his fix causes `sys_ipc_recv`'s performance to slow down linearly with the *maximum* number of environments (`NENV`). Worse, Ben's system still suffers from *starvation*: environment E1 tries to send to environment E2, which continually tries to receive, yet E1 blocks forever because E0 always sends first. (The original system also suffered from the same starvation scenario.)

5. [10 points]: Sketch an implementation in precise English that would fix these problems, replacing the bold code in the `sys_ipc` system calls above. (A full solution would need to change `sys_env_set_status` and `env_free` too, but don't worry about those.)

Maintain in each `struct Env` a linked list of environments waiting to send to that one.
When `sys_ipc_try_send` blocks, it adds the current `Env` to the end of the target `Env`'s list.
When `sys_ipc_recv` checks for waiting sends, it consults the beginning of the list.

Name:

IV Unix Security and OKWS: Chroot

The `chroot` system call on Unix allows programmers and administrators to isolate programs in the file system, denying these programs access to sensitive data such as password files, private keys, or `setuid` executables. In a Unix process, `chroot("/tmp/siberia")` will set the directory `/tmp/siberia` as the new root directory: if the chrooted process then tries to access the file `/a/b`, the kernel will rewrite the access as though the process were accessing `/tmp/siberia/a/b`. This limits the process's file system access to the files in `/tmp/siberia` and its subdirectories, making inaccessible sensitive files like `/etc/passwd` and `/bin/su`.

The prototype for `chroot` from BSD Unix is:

```
int chroot(const char *newroot);
```

Where `newroot` is a directory on the file system. It returns `0` on success and `-1` on failure. Only root can call `chroot` successfully. Consider the implementation of `chroot` for JOS.

6. [5 points]: Where should most of `chroot` be implemented? Give short arguments for or against putting it in the JOS kernel (`kern/*`), the library operating system (`lib/*`), and the file server (`fs/*`).

It would be awkward to put in the kernel, because the kernel doesn't know about file systems.

It can't go in the library operating system, because then a malicious program could just not use it.

The file server must implement the actual rewriting of paths, since it is the final authority on file system operations.

7. [5 points]: Using the best of the three choices you just analyzed, describe an implementation of `chroot` for JOS. Be sure to explain how `chroot` affects later file system interactions by the same environment.

The file server can maintain for each `env_id` a file system root path. Chroot is then an IPC call that informs the file server of the new root path (interpreted relative to the current root path).

Name:

8. [5 points]: A sophisticated attacker might try to break out of a `chroot` by calling `fork` or `spawn`, hoping that the newly created environment would not be chrooted. Describe how you would need to change your implementation to prevent this attack. (If your implementation needs no changes, explain why not.)

One solution is to define that by default, a newly created environment has *no* file system access unless explicitly granted by its parent environment. Then in `fork` and `spawn` it will be up to the library implementation to grant access to the current root. If this granting is bypassed, the new environment will be *more restricted* than usual, not less.

(An equally correct, yet less exokerneley implementation of `chroot` would be to store the file system root as a variable in the `struct Env`, maintained by the kernel, and have a `sys_chroot` system call that appends to the current root path. The file server could read the root path from `UENVS` and `sys_exofork` could simply copy the root when creating the new environment.)

9. [5 points]: Chuck Phil O'Bugz recently wrote an application on BSD Unix that runs malware executable (`0wnU`) in a controlled sandbox. Chuck plans to observe the malware as it runs but wants to protect his private data from theft and wants to protect other running processes on the system from `0wnU`'s interference.

The problem is, Chuck mistakenly ran his code through `sort` and now it doesn't work anymore. Here is the code he has, with line numbers added:

```
 1 int launch_0wnU ()
 2 {
 3   int fd;
 4   char buf[20];
 5   int uid = 2001; // unprivileged UID
 6   int gid = 2001; // unprivileged GID

 7   chroot("/tmp/siberia");
 8   copy_program("/home/chuck/0wnU", "/tmp/siberia");
 9   execl("./0wnU", "-f", buf);
10   fd = open("/var/log/malware.log");
11   mkdir("/tmp/siberia", 0777);
12   setgid(gid);
13   setuid(uid);
14   sprintf(buf, "%d", fd);

15 }
```

Lines 7 through 14 are out of order. What is the correct order for those lines? Write your answer in the form "7,8,9,10,11,12,13,14".

One correct solution is 11,8,10,7,12,13,14,9.

The important constraints are:

11 before 8
8 and 10 before 7
10 before 14
7 and 12 before 13
13 and 14 before 9

Name:

V Buffer overruns

In the Lab 6 source packet you received with this test, you will find a copy of the JOS `user/sh.c`.

For the purposes of this question, assume that any code implemented outside `user/sh.c` and any code that you filled in (the `Your code here` sections) is free of buffer overruns.

You'll note that the top of the file contains the suggestive comment:

```
#define BUFSIZ 1024 /* Find the buffer overrun bug! */
```

But is there really a buffer overrun bug?

10. [5 points]: Which buffer is the comment talking about? Identify the name of the buffer and the line number where it is declared.

`argv0buf`, line 24

11. [5 points]: How can an attacker cause that buffer to overflow?

Pass a command with a very long program name that does not start with a slash. The `strcpy` on line 132 will overflow the buffer.

Name:

12. [10 points]: If the buffer does overflow, can an attacker use that overflow to run arbitrary code? If so, explain exactly how the attack works: draw a stack diagram showing which values are overwritten and explain how the control flow changes as a result. If the attacker cannot run such an attack against `user/sh`, explain why not.

The attacker *cannot* run arbitrary code: the buffer overflow can overwrite `runcmd`'s return address, but `runcmd` never returns after executing line 132.

(The buffer overflow might also overwrite any of the other local stack values, but changing those cannot cause any damage either.)

VI File descriptors

13. [5 points]: In Unix when a parent process forks a child process, the parent and process share open file descriptors. In particular, they share the offset. An alternative plan is for the parent and child not to share the offset but each have its own instance. Explain why sharing is a more desirable plan. Use a specific shell command to illustrate your explanation.

`(date; ls) >out` works better if the offset is shared. Otherwise `ls` overwrites the output of `date` when writing to out.

14. [5 points]: In JOS, to spawn a child process, the parent process must open and read the file that contains the child's program. Explain how the implementation of `spawn` might accidentally leave this file descriptor open in the child. What is the fix?

If the file descriptor is open when `spawn` copies the file table, then the child will inherit the open file descriptor.

The simplest fix is to close the file descriptor before copying the file table.

15. [5 points]: Besides not passing the grading script, what harm is there in leaving the file descriptor open in the child?

Eventually the environments will run out of file descriptors.

VII 6.828

We would like to hear your opinions about 6.828, so please answer the following two questions. (Any answer, except no answer, will receive full credit!)

16. [1 points]: On a scale of 1 to 10, please rate how much you learned from 6.828 (1 is low, 10 is high).

17. [2 points]: If you could change one aspect of 6.828, what would it be?

18. [2 points]: Are there any topics you would like to see added to or removed from the class?

End of Quiz II. Thanks for your participation and enjoy a well-deserved break!

Name:

MIT OpenCourseWare
http://ocw.mit.edu

6.828 Operating System Engineering
Fall 2012

For information about citing these materials or our Terms of Use, visit: http://ocw.mit.edu/terms.



Department of Electrical Engineering and Computer Science

MASSACHUSETTS INSTITUTE OF TECHNOLOGY

6.828 Fall 2008

Quiz II

Most problems are open-ended questions. In order to receive credit you must answer the question as precisely as possible. You have 90 minutes to finish this quiz.

Write your name on this cover sheet.

Some questions may be harder than others. Read them all through first and attack them in the order that allows you to make the most progress. If you find a question ambiguous, be sure to write down any assumptions you make. Be neat. If we can't understand your answer, we can't give you credit!

THIS IS AN OPEN BOOK, OPEN NOTES EXAM.

Please do not write in the boxes below.

I (xx/21)	II (xx/8)	III (xx/8)	IV (xx/8)	V (xx/8)
VI (xx/11)	**VII (xx/14)**	**VIII (xx/22)**	**Total (xx/100)**	

Name:

I File System Consistency

Ben is writing software that stores data in an xsyncfs file system (see the paper *Rethink the Sync* by Nightingale *et al*). He's nervous about crash recovery. To help himself test, he adds a new system call to his operating system called crash(), which powers off his computer without doing anything else. He also writes a simple "crash" command which calls his system call. His general test strategy is to perform some operations, call crash(), restart the computer and let the file system finish recovering, and then observe what files and data are on the disk.

1. [7 points]: First, Ben types the following commands to his shell:

```
% echo hello > foo
% crash
```

After the restart, is Ben guaranteed to see a file foo with contents "hello"? Why or why not?

2. [7 points]: Now Ben runs the following UNIX program which he believes is equivalent to the above commands:

```
pid = fork();
if(pid == 0){
  fd = creat("foo", 0666); // create a file foo
  write(fd, "hello\n", 6);
  close(fd);
  exit(0);
}
wait(&status); // wait for pid to exit
crash();
```

After the restart, is Ben guaranteed to see "hello" in foo? Why or why not?

3. [7 points]: What would the answers be for the xv6 file system? Why?

II Virtualization

The paper *A Comparison of Software and Hardware Techniques for x86 Virtualization* states that one of the goals of virtualization is "Fidelity: Software on the VMM executes identically to its execution on hardware, barring timing effects."

The paper also says, at the end of Section 3.2, that the software virtualization technique outlined in Section 3 does not translate guest user-mode code. That is, when the guest operating system executes instructions that would cause a switch to CPL=3 on real hardware, the VMM stops translating code and allows the original guest instructions to execute directly on the hardware.

4. [8 points]: Explain a way that a carefully constructed guest kernel and/or user-mode program could exploit direct execution of user-mode code to discover whether it was executing on a virtual machine. You should assume the system outlined in Section 3, running on a 32-bit x86.

III Fault Tolerance

Ben is impressed with the paper *Fault Tolerance Under UNIX*, by Borg *et al.* He builds a system that is identical – with one exception. In order to get better performance, his hardware has two busses, and every machine has a connection to both busses. When a machine needs to send a message, it selects one of the two busses at random, and sends the message on that bus.

5. [8 points]: Explain why this change will cause serious problems to the correctness of the system. Give a specific example of something that will likely go wrong.

IV Bugs as Deviant Behavior

Have a look at Figure 3 of the paper *Bugs as Deviant Behavior* by Engler *et al*. Suppose first function in Figure 3 looked like:

```
ssize_t proc_mpc_write(struct file *file,
                       const char *buff){
  retval = parse_qos(buff, incoming);
}
```

That is, suppose the first seven lines of the original function were deleted.

6. [8 points]: Would the checker described in Section 7.1 emit an error for this modified function? Why or why not?

V Livelock

Answer this question in the context of the paper *Eliminating Receive Livelock in an Interrupt-driven Kernel* by Mogul *et al.*

7. [8 points]: Figure 6-3 shows that performance with polling and no quota is quite poor under high input load, but polling with a quota performs much better. Explain what happens without a quota that results in almost zero performance under high input load, and how a quota helps solve this problem.

VI KeyKOS

Answer this question in the context of the paper *The KeyKOS Nanokernel Architecture* by Bomberger *et al*.

8. [4 points]: Suppose a machine running KeyNIX loses power while creating a new file in a directory. After the system is powered back up and executes to a quiescent state (e.g. so that any recovery code is executed), what are the possible states that the file system could be in? Circle all that apply (-1 point for each wrong answer).

A. The new file is allocated but the directory does not contain the new file.

B. The new file is not allocated but the directory contains a reference to the new (unallocated) file.

C. The new file is allocated and the directory contains a reference to the new file.

D. The new file is not allocated and the directory does not contain a reference to the new file.

9. [7 points]: Most file system implementations worry a great deal about what happens after a crash, but the KeyNIX file system has no explicit file system recovery code. Explain what strategy KeyNIX and KeyKOS use to recover from crashes such as the one above. In particular, would the KeyNIX file system recover to a consistent state if a new file was created but not added to its parent directory by the time the machine was powered off (and if so, how)?

VII HiStar

Answer this question in the context of the paper *Making Information Flow Explicit in HiStar* by Zeldovich *et al*.

Recall that KeyNIX maintains a global table mapping process IDs to capabilities (keys) that allow sending a message to that process's Unix keeper. When one process running on top of KeyNIX wants to send a signal to a different process, the sender's Unix keeper must check that the sender is allowed to send a signal to the recipient process (e.g. that both processes belong to the same user), before sending a "kill" message to the recipient's Unix keeper.

HiStar implements signals in a similar fashion: the Unix library locates the signal gate for the recipient process and sends a "kill" message to that gate. (Even though the Unix library has no global process ID table, it can still find the signal gate for a given process ID by enumerating that process's container to find the signal gate, since a process ID is the ID of that process's container object.)

10. [7 points]: The HiStar Unix library is not trusted; anything the Unix library can do, ordinary program code can also do. As a result, the Unix library cannot be trusted to check that signals are only sent to processes owned by the same user. How does HiStar prevent a malicious user program from sending signals to other users' processes?

11. [7 points]: Can KeyNIX be re-designed to prevent a compromised Unix keeper (controlled by an attacker) from sending signals to other users' processes? Sketch out a design that would run on KeyKOS, or describe why it would be difficult to implement using capabilities.

VIII RCU

Answer this question in the context of the paper *Read-Copy Update* by McKenney *et al*.

12. [7 points]: Illustrate a specific problem that could occur if the reference-counted linked list search code in Figure 4 did not obtain a read lock on list_lock. Be sure to indicate what would go wrong as a result.

13. [7 points]: The RCU version of the linked list search code in Figure 8 does not use locks. Describe the steps taken by RCU to ensure that the problem seen in the above question does not occur in this case.

14. [8 points]: Suppose you want to use RCU for a linked list in the xv6 kernel. Explain how to detect quiescence in xv6. To maximize read performance, you may not use any additional code (such as locks) around read accesses to the linked list.

End of Quiz

MIT OpenCourseWare
http://ocw.mit.edu

6.828 Operating System Engineering

Fall 2012

For information about citing these materials or our Terms of Use, visit: http://ocw.mit.edu/terms.



Department of Electrical Engineering and Computer Science

MASSACHUSETTS INSTITUTE OF TECHNOLOGY

6.828 Fall 2009

Quiz II

All problems are open-ended questions. In order to receive credit you must answer the question as precisely as possible. You have 80 minutes to finish this quiz.

Write your name on this cover sheet AND at the bottom of each page of this booklet.

Some questions may be harder than others. Read them all through first and attack them in the order that allows you to make the most progress. If you find a question ambiguous, be sure to write down any assumptions you make. Be neat. If we can't understand your answer, we can't give you credit!

THIS IS AN OPEN BOOK, OPEN NOTES EXAM.

Please do not write in the boxes below.

I (xx/36)	II (xx/20)	III (xx/22)	IV (xx/16)	V (xx/6)	Total (xx/100)

Name:

I Paper questions

1. [6 points]: Many disks have a cache of disk blocks inside the disk and reorder writes to get better performance. Such reordering could cause problems for the version of ext3 as described by Tweedie in "Journaling the Linux ext2fs Filesystem;" give a scenario that results in inconsistent file system metadata structures.

2. [6 points]: How could you extend ext3 to fix this problem without disabling the disk's cache?

3. [6 points]: KeyKOS's use of capabilities is motivated by the "confused deputy" problem. Give an example of the confused deputy problem in Unix. Give an example how a KeyKOS application developer uses capabilities to avoid your example of the confused deputy problem in Unix.

4. [6 points]: To track dependencies introduced by Unix system calls, BackTracker (as described in "Backtracking Intrusions" by King and Chen) maintains objects for processes, files, and filenames. Ben thinks it's redundant to maintain both file objects, which identify files by inode numbers, and filename objects, which identify files by absolute path. He proposes modifying BackTracker to collapse the two into just filename objects, identifying files solely by their absolute path. Alyssa argues that this change will make BackTracker inaccurate (i.e., BackTracker could fail to identify an attack that influences a suspicious file or process). Give an example showing that Alyssa is correct.

5. [6 points]: Louis Reasoner proposes to extend BackTracker to recover from attacks by undoing all the attacker's actions, but not regular user actions. Using BackTracker, he can identify the starting event of an attack. He proposes to first use ReVirt to roll the system back to just before that starting event. For example, for the attack in Figure 1 of the paper, he would roll the system back to just before the "[sh,bash]" process was created. He would then *selectively* roll the system forward using ReVirt, skipping all the events that could have been influenced by the attacker, according to the dependencies that BackTracker has computed. For example, Louis' extension would not replay any operation that depended on the "[sh,bash]" node. Ben points out that this approach may fail to re-execute regular user actions. Give an example showing that Ben is right.

6. [6 points]: Figure 8 in the Klee paper ("Unassisted and automatic generation of high-coverage tests for complex systems programs") shows an example of a bug found by Klee in the `pr` utility. Specially, executing `pr -e t2.txt` will result in a buffer overflow if `t2.txt` has the content:

```
\b\b\b\b\b\b\b\t
```

Explain how Klee is able to construct this input.

II Lock-free data structures

Joe Buck is increasing the performance of the the Linux physical page allocator using a lock-free LIFO data structure to track free pages. He reasons that recently freed pages will still be in on-chip hardware caches, so allocating those pages first will improve performance. Joe Buck's code for the lock-free LIFO is below.

```
// Compare the value at addr with oldval.  If equal, store newval
// at addr and return 1, otherwise return 0.  Executes atomically.
int cmpxchg(void *addr, uint32_t oldval, uint32_t newval);

struct page {
    struct page *next;
    /* Other page metadata ... */
};
struct page *head;

void push(struct page *page) {
    while (1) {
        page->next = head;
        if (cmpxchg(&head, (uint32_t)page->next, (uint32_t)page))
            break;
    }
}

struct page * pop(void) {
    while (1) {
        struct page *page = head;
        if (page == NULL)
            return NULL;
        struct page *next = page->next;
        if (cmpxchg(&head, (uint32_t)page, (uint32_t)next))
            return page;
    }
}
```

7. [10 points]: Unfortunately Joe Buck's code has a race condition. Describe a sequence of events for two CPUs that could cause the LIFO to be in an inconsistent state (that is, an allocated page appears in the LIFO or an unallocated page does not appear in the LIFO). For your convenience, we provided the first two events of a sequence; your job is to add the others.

1. Initially, *head* → page A → page B → page C → *NULL*.
2. CPU 1 starts to `pop`; it sets `page = A` and `next = B`.

8. [10 points]: Some CPUs provide atomic instructions that make it easier to write lock-free code. For example, PowerPC CPUs provide the Load-Linked and Store-Conditional instructions. LL must be paired with a SC. LL loads a value from memory and the CPU remembers that the address is load-linked. SC stores a value to an address if the address has not been written to by any other CPU since the corresponding LL. Write a correct lock-free LIFO implementation that uses LL and SC.

```
// Return the 32-bit value at addr and remember addr is load-linked.
uint32_t ll(void *addr);
// Store val at addr if *addr has not been written to since the last ll from addr.
// Return 0 on failure, 1 on success.
int sc(uint32_t val, void *addr);

void push(struct page *page) {









}

struct page * pop(void) {









}
```

III VMM

Many x86 CPUs now have support in hardware for CPU, MMU, and IO port virtualization. The CPU virtualization features provide each guest VM with its own segment/interrupt descriptor tables, control registers, and segment registers. A virtual machine monitor (VMM) can use CPU virtualization to trap a VM when it executes an instruction or causes an event that could affect the VMM or other VMs. IO port virtualization allows VMMs to control VM access to IO ports, trapping into the VMM when the VM accesses certain IO ports.

9. [6 points]: AMD virtualization hardware allows the VMM to intercept certain CPU instructions and events by setting bits in a bitmask. As discussed in Section 4.4 of "A Comparison of Software and Hardware for x86 Virtualization" the performance of a VMM depends on the frequency of exits from the VM. To help avoid frequent exits, AMD hardware copies (or shadows) CPU state into a Virtual Machine Control Block (VMCB). Guest instructions that manipulate CPU state (e.g. `lcr0`) operate on the shadow state instead of the real CPU state. Circle the instructions that the VMM *must* intercept, because it is not possible to shadow the state they manipulate.

- Instructions that read or write control registers (e.g. `cr3`)
- Instructions that cause the computer to shutdown
- Instructions that load descriptor tables (e.g. `lgdt`)
- Instructions that clears cache contents without writing back to memory
- `pushf` and `popf`
- `iret`

10. [6 points]: You are unhappy with the performance of networking in JOS. On x86 hardware, port-based IO is not optimized and JOS's e100 driver is executing many `in` and `out` instructions, which hurt performance. The e100 exposes all of the same control registers using memory-mapped IO, which is much faster than port-based IO. Suppose we're running JOS in a virtual machine, but giving it direct access to our host's e100 card. Describe how you might use dynamic binary translation in the VMM to transparently convert JOS's e100 driver from using `in` and `out` port-based IO to using memory-mapped IO.

11. [10 points]: Modern MMU virtualization hardware works as described in Section 7.4 of "A Comparison of Software and Hardware for x86 Virtualization." As usual, guest VMs create *guest page tables* that translate from guest virtual addresses to guest physical addresses. However, with MMU virtualization, guest physical addresses are then further translated through a *nested page table*, created by the VMM, which maps guest physical addresses to host physical addresses.

When preparing a VM to run, the VMM must allocate physical pages for the guest's memory and initialize this nested page table. Write the code to do this below. Your implementation should allocate `npages` host physical pages and map them in the page table `pgdir` starting at guest physical address 0. Return 0 from the function on success, or return a negative value on an error.

```
/* Helper functions from JOS  */
int     page_insert(pde_t *pgdir, struct Page *pp, void *va, int perm);
int     page_alloc(struct Page **pp_store);
physaddr_t page2pa(struct Page *pp);

/* Guest permission bits */
#define PTE_GUEST       (PTE_P|PTE_W|PTE_U)

int
vm_init_mem(unsigned int npages, pde_t *pgdir)
{









}
```

IV High-performance JOS

12. [10 points]: Ben, a little sleep-deprived from the network lab, dreams of turning JOS into the next big, high-performance web serving platform. He took the easy route in his network lab and made his receive syscall always return immediately, even if no packet is available, which is less than ideal for CPU utilization. He starts by making his receive syscall block, much like `sys_ipc_recv`.

Ben keeps around his old non-blocking receive function for use in the new implementation. Now that he's no longer polling, he configures the e100 to generate an interrupt whenever it puts a packet in the receive ring and installs `e100_rx_intr` as the interrupt handler. Finally, since only one environment can receive at a time, he adds a few global variables to track the currently receiving environment.

```
// Copy the next packet into dst and return the length of the packet.
// If the receive ring is empty, return 0.
int e100_recv_nonblocking(void *dst);

// Environment currently blocked on receive, or NULL if none.
struct Env *e100_receiver;
// Packet buffer of the currently receiving environment.
void       *e100_receiver_dst;
```

Give pseudo-code for the new blocking syscall implementation and for `e100_rx_intr`. You can assume `dst` is a valid pointer to a sufficiently large buffer.

```
int sys_e100_recv(void *dst) {









}

void e100_rx_intr(void) {
    // Acknowledge the interrupt.  Otherwise, the e100 blocks further interrupts.
    e100_ack_rx();









}
```

13. [6 points]: Is your solution susceptible to receive livelock? If so, give a scenario that demonstrates the livelock. If not, explain why not.

V 6.828

We'd like to hear your opinions about 6.828, so please answer the following questions. (Any answer, except no answer, will receive full credit.)

14. [2 points]: The network lab took more time on average than we had intended. What would have saved you the most time? If it's test cases, which test cases would you add? If it's documentation, what would you like documentation on?

15. [2 points]: What is the best aspect of 6.828?

16. [2 points]: What is the worst aspect of 6.828?

End of Quiz

Name:

MIT OpenCourseWare
http://ocw.mit.edu

6.828 Operating System Engineering

Fall 2012

For information about citing these materials or our Terms of Use, visit: http://ocw.mit.edu/terms.



Department of Electrical Engineering and Computer Science

MASSACHUSETTS INSTITUTE OF TECHNOLOGY

6.828 Fall 2011

Quiz II

All problems are open-ended questions. In order to receive credit you must answer the question as precisely as possible. You have 80 minutes to finish this quiz.

Write your name on this cover sheet AND at the bottom of each page of this booklet.

Some questions may be harder than others. Read them all through first and attack them in the order that allows you to make the most progress. If you find a question ambiguous, be sure to write down any assumptions you make. Be neat. If we can't understand your answer, we can't give you credit!

THIS IS AN OPEN BOOK, OPEN NOTES EXAM.

Please do not write in the boxes below.

I (xx/40)	II (xx/18)	III (xx/18)	IV (xx/16)	V (xx/8)	Total (xx/100)

Name:

I Short paper questions

1. [8 points]: In Singularity ("Singularity: rethinking the software stack" by Hunt and Larus) a system call is run on the same stack as user code. As the authors point out, this has the potential to entangle user and kernel garbage collection. Why can a malicious SIP *not* trick the kernel into examining a user stack frame when the kernel performs garbage collection? (The paper doesn't give a precise answer; you'll have to deduce it as best as you can.)

2. [8 points]: The paper on Linux scalability uses a standard trick with generation numbers to avoid a scalability bottleneck in the `dentry` cache (section 4.4). The pseudo code is as follows:

```
lookup(inode, name):
    dentry = hash(inode, name)
    g = dentry.gen
    if g == 0:
        # some core is modifying dentry
        fall back to locking protocol
    # copy relevant fields
    i = dentry.inode
    n = dentry.name
    g1 = dentry.gen
    if g == g1 and inode == i and name == n:
        ok = atomic_inc_if_not_zero(&dentry.refcnt);
        if (!ok) fall back to locking protocol
    else:
        fall back to locking protocol
    return dentry
```

```
modify(dentry):
    spin lock dentry
    g = dentry.gen
    dentry.gen = 0 // DELETE THIS LINE
    dentry.inode = ...
    dentry.name = ...
    dentry.gen = g + 1
    unlock dentry
```

(continued on next page)

Give an example of something that could go wrong if you deleted the line marked "DELETE THIS LINE".

3. [8 points]: The ballooning technique described in "Memory Resource Management in VMware ESX Server" by Waldspurger avoids double paging by monitor and guests. Why does ESX use ballooning instead of a scheme in which the guest kernel explicitly tells the VMM about how it uses physical pages?

MCS locks don't suffer performance collapse like ticket spin locks do, as described in unpublished document assigned for the lecture 16 (paper #243). For reference in the following two questions, here is an implementation of MCS locks in C (which follows the description of MCS locks in the unpublished document):

```
1  // Each core has a qnode for each lock
2  struct qnode {
3      volatile struct qnode *next;
4      volatile char waiting;
5      char __pad[0] __attribute__((aligned(CACHELINE)));
6  };
7
8  typedef struct {
9      struct qnode *tail __attribute__((aligned(CACHELINE)));
10 } mcslock;
11
12 void
13 mcs_lock(mcslock *L, struct qnode *mynode)
14 {
15     mynode->next = NULL;
16     struct qnode *predecessor = fetch_and_store(&L->tail, mynode);
17     if (predecessor) {
18         mynode->waiting = 1;
19         predecessor->next = mynode;
20         while (mynode->waiting)
21             /* spin */;
22     }
23 }
24
25 void
26 mcs_unlock(mcslock *L, struct qnode *mynode)
27 {
28     if (!mynode->next) {
29         if (compare_and_swap(&L->tail, mynode, NULL))
30             return;
31         while (!mynode->next)
32             /* spin */;
33     }
34     mynode->next->waiting = 0;
35 }
```

Each core invokes lock and unlock with its own private `qnode`.

Name:

4. [8 points]: Use the above code to explain why MCS locks don't suffer performance collapse as ticket locks do when several cores are contending for the same lock.

5. [8 points]: As the paper reports, a disadvantage of MCS locks is that acquiring and releasing an uncontended MCS lock takes more time than acquiring and releasing an uncontended ticket lock (see Figure 11). Using the above MCS code and the ticket code from the paper (Figure 1), explain why MCS locks are more expensive in the non-contended case.

II Rethink the Sync

Tom the TA writes the following grading program. The idea is that per-assignment scores are stored in a set of input files, and the program's job is to compute a final course score for each student and write it to file *studentname*`.final`.

```
main():
    remove all files ending with ".final"
    for each student name s in alphabetical order:
        read assignment scores for student s
        calculate final grade
        filename = s + ".final"
        fd = creat(filename)
        write(fd, final grade, ...)
        close(fd)
        printf("done with %s\n", s)
```

Tom uses a Linux laptop with an EXT3 journaling file system in "journaled data" mode (so that the journal contains file content, in addition to metadata). He runs the following:

```
program | cat
```

Tom sees "done with x" for all students with names up through "m", and then his laptop crashes. Tom restarts his laptop.

6. [6 points]: Tom thinks he may have to re-run the grading program for some or all students. Explain what guarantees he can count on about which final grades will be on disk after the restart.

The next year, Tom does the same thing, but this time he uses the xsyncfs file system described in the Rethink the Sync paper. Again, Tom's laptop crashes just after he sees the "done with x" for all students with names up through "m", and he restarts the laptop.

7. [6 points]: Explain what guarantees Tom can count on about which final grades will be on disk after the restart.

The third year, Tom types these commands on his xsyncfs laptop:

```
program > out
cat out
```

Again, he sees "done with x" for all students with names up through "m", his laptop crashes, and he restarts it.

8. [6 points]: Explain what guarantees Tom can count on about which final grades will be on disk after the restart.

III KLEE

Consider this C program:

```
int
square(int x)
{
    int table[] = { 0, 1, 4, 9, 0xb00b00, 25, 36 };
    return table[x];
}

void
main(void)
{
    printf("%d\n", square(4));
}
```

You run KLEE (see the paper by Cadar *et al.*) on this program. Assume that the C libraries have no bugs.

9. [6 points]: Would KLEE find any bugs in the above program? If yes, describe the full set of test cases that KLEE could generate to illustrate the bug(s).

Now suppose you run KLEE on this program; `square()` is the same but `main()` is different:

```
int
square(int x)
{
    int table[] = { 0, 1, 4, 9, 0xb00b00, 25, 36 };
    return table[x];
}

void
main(void)
{
    int x = 0;
    scanf("%d", &x);
    printf("%d\n", square(x));
}
```

The call to `scanf()` reads an integer from the standard input and places it in `x`.

10. [6 points]: Would KLEE find any bugs in the above program? If yes, describe the full set of test cases that KLEE could generate to illustrate the bug(s).

Now this program; again, `square()` is the same but `main()` is different:

```
int
square(int x)
{
    int table[] = { 0, 1, 4, 9, 0xb00b00, 25, 36 };
    return table[x];
}

void
main(void)
{
    int x = 0, y;
    scanf("%d", &x);
    y = square(x);
    assert(y == x*x);
}
```

11. [6 points]: Would KLEE find any bugs in the above program? If yes, describe the full set of test cases that KLEE could generate to illustrate the bug(s).

IV E1000

Ben Bitdiddle has just finished implementing his E1000 receive function, shown below.

```
int
e1000_receive(char *buf)
{
    int tail = read_reg(RDT);

    // Check for and clear "Descriptor Done" bit
    if (!(rx_descriptors[tail].status & E1000_RXD_STAT_DD))
        return 0;
    rx_descriptors[tail].status = 0;

    // Receive the packet
    write_reg(RDT, (tail + 1) % RX_RING_SIZE);
    memmove(buf, rx_buffers[tail], rx_descriptors[tail].length);
    return rx_descriptors[tail].length;
}
```

Ben tests his receive function by sending JOS a hundred packets and, between each packet, checking that it received the right data. Confident in his work, he shows it off to Alyssa P. Hacker. When Alyssa tries sending a hundred packets in a short burst, everything falls apart; Ben's JOS drops several packets, duplicates some, and corrupts others.

12. [8 points]: Why does Ben's implementation drop, duplicate, and corrupt packets and how can he fix it?

Louis Reasoner is having similar problems receiving packets. However, he knows that his E1000 driver is correctly receiving all packets. For Louis, packets are getting dropped or duplicated before they are processed by the network server environment. He narrows the problem to his input environment, which looks like

```
void
input(envid_t ns_envid)
{
    while (1) {
        // Receive a packet
        nsipcbuf.pkt.jp_len = sys_net_rx(nsipcbuf.pkt.jp_data, 2048);
        if (nsipcbuf.pkt.jp_len > 0) {
            // Pass packet to network server environment
            ipc_send(ns_envid, NSREQ_INPUT, &nsipcbuf, PTE_P|PTE_U);
            // Wait for NS environment to process the packet
            for (int i = 0; i < 8; i++)
                sys_yield();
        } else
            sys_yield();
    }
}
```

13. [4 points]: Explain the race in Louis' implementation that causes the network server environment to see dropped and duplicated packets.

14. [4 points]: Modify Louis' code to fix this race. (You can mark up the code above if you find it easier.)

V 6.828

We'd like to hear your opinions about 6.828, so please answer the following questions. (Any answer, except no answer, will receive full credit.)

15. [2 points]: Grade 6.828 on a scale of 0 (worst) to 10 (best)?

16. [2 points]: Any suggestions for how to improve 6.828?

17. [2 points]: What is the best aspect of 6.828?

18. [2 points]: What is the worst aspect of 6.828?

End of Quiz

Name:

MIT OpenCourseWare
http://ocw.mit.edu

6.828 Operating System Engineering

Fall 2012

For information about citing these materials or our Terms of Use, visit: http://ocw.mit.edu/terms.



Department of Electrical Engineering and Computer Science

MASSACHUSETTS INSTITUTE OF TECHNOLOGY

6.828 Fall 2010

Quiz II

All problems are open-ended questions. In order to receive credit you must answer the question as precisely as possible. You have 90 minutes to finish this quiz.

Write your name on this cover sheet AND at the bottom of each page of this booklet.

Some questions may be harder than others. Read them all through first and attack them in the order that allows you to make the most progress. If you find a question ambiguous, be sure to write down any assumptions you make. Be neat. If we can't understand your answer, we can't give you credit!

THIS IS AN OPEN BOOK, OPEN NOTES EXAM.

Please do not write in the boxes below.

I (xx/30)	II (xx/46)	III (xx/16)	IV (xx/8)	Total (xx/100)

Name:

I Short Paper Questions

You are using an xsyncfs file system as described in the paper "Rethink the Sync." You run the following program with its standard output connected to your terminal. Initially, the file xyzzy does not exist.

```
int main(void)
{
  // O_CREAT means create the file if it doesn't already exist.
  // O_WRONLY means open for writing.
  int fd = open("xyzzy", O_CREAT | O_WRONLY);
  write(fd, "one", 4);
  write(fd, "two", 4);
  close(fd);
  printf("done");
  return 0;
}
```

1. [8 points]: Sadly a power failure occurs just after you start the program, and you do not see `done` on the console. You restart the system (including the file system's recovery procedure) and look at file xyzzy. Write yes or no after each line, depending on whether it is possible state for xyzzy.

- xyzzy is empty
- xyzzy contains just `one`
- xyzzy contains just `two`
- xyzzy contains `onetwo`

Suppose now that you delete xyzzy and run the same program with its output redirected to file plugh:

```
$ program > plugh
```

2. [8 points]: Again, the power fails just after you start the program, and you don't see any output on your console. After restart you see that file plugh contains `done`. Which states for file xyzzy are possible now?

- xyzzy is empty
- xyzzy contains just `one`
- xyzzy contains just `two`
- xyzzy contains `onetwo`

The paper "A Comparison of Software and Hardware Techniques for x86 Virtualization" describes techniques by which a software VMM can detect guest kernel modifications to the guest's page tables. The VMM needs to be aware of such modifications so that the VMM can reflect them in the "shadow" page table that the VMM installs in the real hardware %cr3. Indicate whether each of the following statements are true or false about the the paper's description of how the VMWare software VMM deals with PTE writes:

3. [6 points]:

True / False : The VMM uses binary translation (BT) to insert instructions before every guest kernel memory-referencing instruction to detect whether the instruction modifies a PTE.

True / False : The VMM write-protects the pages that make up the guest's page tables.

True / False : Each guest kernel modification of a PTE results in a page fault to the VMM.

True / False : The shadow page table is identical to the guest kernel's page table, except that some PTE's present or writable bits differ.

Mark each of the following statements according to whether it accurately reflects the content of the paper "Efficient System-Enforced Deterministic Parallelism."

4. [8 points]:

True / False : The output of a program on Determinator cannot be influenced by any factors other than the program code itself.

True / False : A given program on Determinator always interleaves the execution of memory read and write instructions from different CPUs in the same order.

True / False : A program run on Determinator cannot have any bugs relating to concurrency.

True / False : The Determinator file system is implemented using Puts and Gets between the file system server and its clients.

True / False : Each Get in a parent must be matched by a Ret in the corresponding child.

True / False : A parent can use Get to observe the contents of a child space's memory at any time.

II Concurrency and RCU

After taking 6.828, Ben is fascinated with concurrency, and decides to implement concurrent stacks. He starts with the following correct serial implementation of a stack:

```
1  elem_t *head;   // the top of the stack
2
3  void push(int key, int value)
4  {
5    elem_t *e = malloc(sizeof(*e));
6    e->next = head;
7    e->key = key;
8    e->value = value;
9    head = e;    // Put it on the stack
10 }
11
12 elem_t *pop(void)
13 {
14   elem_t *e = head;
15   if (e) head = e->next;
16   return e;
17 }
18
19 elem_t *search(int key)
20 {
21   for (elem_t *e = head; e; e = e->next) {
22     if (e->key == key) {
23       return e;
24     }
25   }
26   return NULL;
27 }
```

Ben wants to run this code on his multicore computer. The cores share a cache-coherent memory and Ben places `head`, `freelist`, etc. in shared memory, so that different cores can push and pop from the shared stack.

5. [8 points]: Ben is pretty sure this code cannot run correctly on a multicore computer. That is, if two threads on different cores concurrently invoke `search`, `push`, and `pop`, then there can be races. To refresh Ben's mind, please give an example of a non-benign race in Ben's code.

Ben decides to make the implementation correct using a read-write lock. Here are the locked versions of `push` and `search`:

```
struct rwlock rwlock;

void locked_push(int key, int value)
{
  acquire_write(&rwlock);
  push(key, value);
  release_write(&rwlock);
}

elem_t *locked_search(int key)
{
  acquire_read(&rwlock);
  elem_t *e = search(key);
  release_read(&rwlock);
  return e;
}
```

Ben implements read/write locks correctly: either one writer can acquire the lock in write mode, or several readers can acquire the lock in read mode. The acquire parts of his read/write lock are. You don't need to understand them in detail.

```
struct rwlock {
  spinlock_t l;
  volatile unsigned nreader;
};

void acquire_read(struct rwlock *rl)
{
  spin_lock(&rl->l);
  atomic_increment(&rl->nreader);
  spin_unlock(&rl->l);
}

void acquire_write(struct rwlock *rl)
{
   spin_lock(&rl->l);
   while (rl->nreader) /* spin */ ;
}
```

6. [6 points]: If Ben invokes many `locked_search`'s concurrently, why does the version with a read/write lock perform better than if Ben had used a regular spin lock?

7. [6 points]: Ben observes that even with a read/write lock and no concurrent `push`'s or `pop`'s, 48 concurrent `locked_search`'s run slower than 48 concurrent `search`'s. Explain briefly why.

Inspired by the RCU paper ("Read-copy update" by McKenney et al.), Ben decides to create an RCU-like list implementation, but based on time instead of quiescent states. He adds two fields to `struct elem_t` (`time` and `delayed_next`) and introduces the following RCU functions:

```
elem_t *delayed_freelist;
int safe_time[NUMCPU];

void rcu_free(elem_t *e)
{
  e->time = time_since_boot();
  e->delayed_next = delayed_freelist;
  delayed_freelist = e;
  rcu_gc();
}

void rcu_safe(void)
{
  safe_time[mycpu] = time_since_boot();
}

void rcu_gc(void) { /* .. see below .. */ }
```

When a thread wants to free an element, it calls `rcu_free` to inform RCU that it is done with the element. The element cannot be reused yet, because other threads may have a reference to it, so `rcu_free` adds the element to the `delayed_freelist`. When a thread has no references to any element, it informs RCU by calling `rcu_safe`. You can assume that `time_since_boot` is very cheap to execute. The `rcu_gc` function frees elements that are safe to free (i.e., elements that cannot be in use by any thread).

An example of how these functions are intended to be used is as follows:

```
void thread1(void) {
  elem_t *e;
  while (1) {
    acquire_write(&rwlock);
    if ((e = pop())) {
      compute1(e);
      rcu_free(e);
    }
    rcu_safe();
    release_write(&rwlock);
  }
}

void thread2(void) {
 elem_t *e;
 while (1) {
   if ((e = search(random())))
      compute2(e);
   rcu_safe();
  }
}
```

This code assume there is a list with many elements. Thread 1 repeatedly pops an element from this list, computes with it, `rcu_free`'s it, and informs RCU it has no references to any element. Thread 2 searches the list, computes with the found element, and informs RCU it has no references to any element.

8. [8 points]: Why must RCU delay free operations? Give an example scenario and an explanation of what might go wrong if RCU didn't delay free operations.

9. [10 points]: Complete pseudocode for the RCU garbage collector (`rcu_gc`) that frees all elements from the delayed free list that can safely be freed (use `free` to free the individual elements).

```
void rcu_gc(void)
{

}
```

Consider the following two variations on `thread2` (differences underlined):

```
void thread2_unlocked(void) {
 elem_t *e;
 while (1) {
   int key = random();
   if ((e = search(key))) {
      compute2(e);
   }
   // No rcu_safe();
  }
}

void thread2_locked(void) {
 elem_t *e;
 while (1) {
   int key = random();
   if ((e = locked_search(key))) {
      compute2(e);
   }
  }
}
```

10. [8 points]: Assuming no concurrent `push`'s or `pop`'s, would 48 concurrent `thread2`'s perform as fast as 48 concurrent `thread2_unlocked`'s, as slow as 48 concurrent `thread2_locked`'s, or somewhere between? Explain briefly.

III Network driver

Late one night, Ben Bitdiddle is tooling away at his E100 receive implementation for the network lab. For reference, the receive frame descriptor (RFD) format is

Command word						Status word			
EL	S	000000000	H	SF	000	C	0	OK	Status bits
Link address									
Reserved									
0	0	Size				EOF	F	Actual count	

He implements receive as follows:

```
// The number of RFD's in the receive ring
#define NUM_RFDS 16
// Pointers to the RFD's in the receive ring
static struct rfd *rfd_ring[NUM_RFDS];
// The index of the next RFD to read a packet from
static unsigned int rfd_tail;

// Receive one packet and store it in to buf.
int e100_receive(char *buf)
{
  struct rfd *rfd = rfd_ring[rfd_tail % NUM_RFDS];
  if (!(rfd->status & RFD_STATUS_C))
    // No packets in receive ring
    return 0;
  // Clear completed ("C") bit
  rfd->status = 0;
  // Copy packet data
  int actualCount = rfd->actualCount & 0x3FFF;
  memmove(buf, rfd->data, actualCount);
  // Move to the next entry in the ring
  rfd_tail++;
  return actualCount;
}
```

Ben tests his implementation using the echo server and finds that it works. Proud of his succinct implementation, he shows it off to Louis Reasoner, who spots a bug.

11. [8 points]: Louis points out to Ben that he needs to manipulate the suspend ("S") bits in the RFD's, so Ben sets the S bit on `rfd_ring[NUM_RFDS-1]` during initialization and adds the following code just after the `memmove` in `e100_receive`:

```
// Set the suspend bit on this RFD
rfd->command = RFD_COMMAND_S;
// Clear the suspend bit on the previous RFD
rfd_ring[(rfd_tail + NUM_RFDS - 1)%NUM_RFDS]->command = 0;
```

Give an example of a problem that Ben's addition prevents.

12. [8 points]: Ben again tests his code using the echo server. Now confident in his implementation, Ben shows it off to Alyssa P. Hacker, who informs Ben that, while the change he just made was important, it missed something: just after clearing the suspend bit on the previous RFD, he needs to issue an "RU resume" command. Give an example of a problem that Alyssa's addition fixes.

IV 6.828

We'd like to hear your opinions about 6.828, so please answer the following questions. (Any answer, except no answer, will receive full credit.)

13. [2 points]: In the second half of the term, we read papers instead of xv6 source code. Which papers did you like? (We read ext3 Journaling, Rethink the Sync, Eliminating Receive Livelock, KeyKOS, Singularity, Anderson Locks, RCU, Barrelfish, Determinator, Software vs Hardware Virtualization, SMP-ReVirt, and Ksplice.)

14. [2 points]: With respect to the labs since quiz 1, are those labs too time consuming, too short, or are they about right? (Lab 4 was COW fork, preemptive multitasking, and IPC; lab 5 was the file system; lab 6 was the network; and lab 7 was the shell and project.)

15. [2 points]: Now that you have completed 6.828, what is the best aspect of 6.828?

16. [2 points]: Now that you have completed 6.828, what is the worst aspect of 6.828?

End of Quiz

Name:

MIT OpenCourseWare
http://ocw.mit.edu

6.828 Operating System Engineering

Fall 2012

For information about citing these materials or our Terms of Use, visit: http://ocw.mit.edu/terms.



Department of Electrical Engineering and Computer Science

MASSACHUSETTS INSTITUTE OF TECHNOLOGY

6.828 Operating System Engineering: Fall 2003

Quiz I Solutions

All problems are open-ended questions. In order to receive credit you must answer the question as precisely as possible. You have 80 minutes to answer this quiz.

Write your name on this cover sheet AND at the bottom of each page of this booklet.

Some questions may be much harder than others. Read them all through first and attack them in the order that allows you to make the most progress. If you find a question ambiguous, be sure to write down any assumptions you make. Be neat. If we can't understand your answer, we can't give you credit!

THIS IS AN OPEN BOOK, OPEN NOTES QUIZ.

1 (xx/19)	2 (xx/15)	3 (xx/25)	4 (xx/20)	5 (xx/15)	6 (xx/6)	Total (xx/100)

Name:

I Calling conventions

In lab 2 you extended `printf`. Here is the core of `printf` relevant for this question:

```
void printk(const char *fmt, va_list ap)
{
        register char *p, *q;
        register int ch, n;
        u_quad_t uq;
        int base, lflag, qflag, tmp, width;
        char padc;

        for (;;) {
                padc = ' ';
                width = 0;
                while ((ch = *(u_char *) fmt++) != '%') {
                        if (ch == '\0')
                                return;
                        cons_putc(ch);
                }
                lflag = 0;
                qflag = 0;
        reswitch:
                switch (ch = *(u_char *) fmt++) {
                case 'd':
                        uq = getint(&ap, lflag, qflag);
                        if ((quad_t) uq < 0) {
                                cons_putc('-');
                                uq = -(quad_t) uq;
                        }
                        base = 10;
                        goto number;

                [...  other cases omitted ... not relevant to the question]

                number:
                        p = ksprintn(uq, base, &tmp);
                        if (width && (width -= tmp) > 0)
                                while (width--)
                                        cons_putc(padc);
                        while ((ch = *p--) != '\0')
                                cons_putc(ch);
                        break;
                }
        }
}
```

Name:

```
static u_quad_t
getint(va_list *ap, int lflag, int qflag)
{
        if (lflag)
                return va_arg(*ap, u_long);
        else if (qflag)
                return va_arg(*ap, u_quad_t);
        else
                return va_arg(*ap, u_int);
}


int printf(const char *fmt,...)
{
        va_list ap;

        va_start(ap, fmt);
        kprintf(fmt, ap);
        va_end(ap);
        return 0;
}
```

1. [10 points]: What values does gcc on the x86 push on the stack for the call:

`printf(''the class number is %s and used to be %d \n'', ''6828'', 6097)`

1. The number 6097, as a 4-byte integer.
2. The address of the (null-terminated) string "`6828`", as a 4-byte pointer.
3. The address of the string "`the class number ...`", as a 4-byte pointer.

Name:

2. [4 points]: gcc pushes the arguments in a particular order. What is the order and why?

gcc pushes arguments in reverse order, last argument first. Because the stack grows down on the x86 (and PDP-11), this means that the first argument (last one pushed) will have the lowest address in memory, just above the function's return address. This way, the called function can find its first argument without knowing exactly how many and what types of other arguments it was called with. Knowing the location and type of the first argument, the function can then locate the second argument (if there is one), and so on. The `printf()` function depends on this property because it uses its first argument (the format string) to determine how many and what types of additional arguments it was called with.

3. [5 points]: Explain what *va_arg* does briefly.

The `va_arg` macro is invoked with an expression of the form `va_arg(`$vl, type$`)`. The macro assumes that vl is a pointer variable of type `va_list`, which points to the "current" argument in a variable-length list of arguments that was pushed onto the stack as a result of a C function call. The `va_arg` macro further assumes that $type$ is a C-language type name describing the type of this argument. `va_arg` uses the vl pointer to read the current argument from the stack and "return" it as the result of the `va_arg(`$vl, type$`)` expression. After reading the current argument, `va_arg` also increments the vl pointer by the appropriate number of bytes, typically `sizeof(`$type$`)`, to advance vl to point to the next argument on the stack. As a result, the next invocation of the `va_arg` macro on the same vl will read the next argument in the list, and so on.

Name:

II Concurrency

Sheets 86 through 87 show the code for a simple device: the paper tape reader.

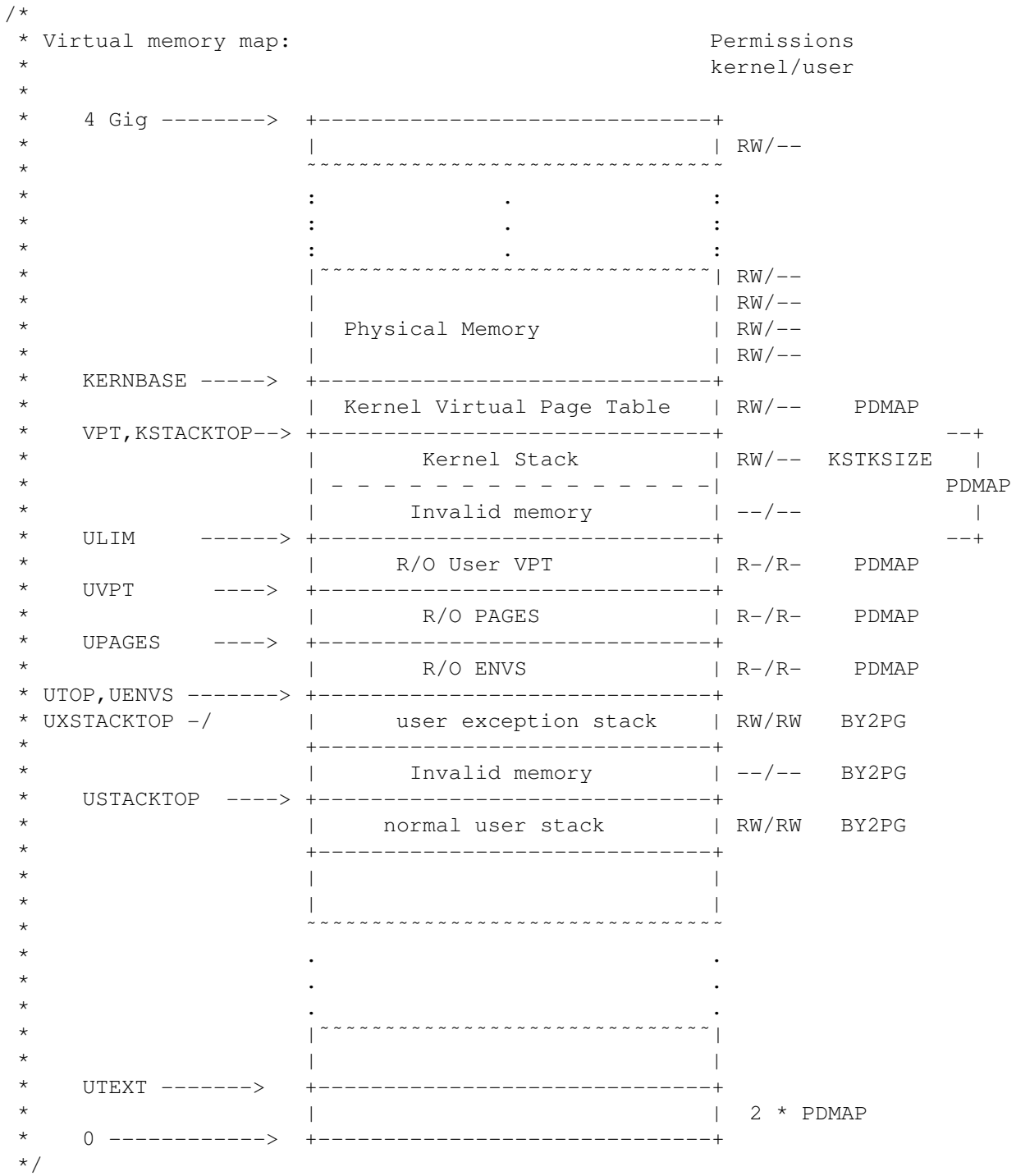
4. [15 points]: If you delete *spl4()* on line 8686, can you give a concrete sequence of events that results in deadlock? (Hint: you don't have to understand the device deeply to answer this question; focus on the interaction of *sleep* and *wakeup*.)

1. A user process opens the device, causing a call to `pcopen()` in the kernel.
2. `pcopen()` sets `pc11.pcstate` to `WAITING` (line 8657), then sleeps until an interrupt causes `pc11.pcstate` to change (lines 8658-8661).
3. A device interrupt causes `pcrint()` to set `pc11.pcstate` to `READING` (line 8724).
4. `pcopen()` wakes up and returns to the user process.
5. The user process now invokes the `read()` system call on this device, resulting in a call to `pcread()` in the kernel.
6. `pcread` reaches line 8691 after finding that the input buffer is empty (line 8688) and `pc11.pcstate` is not `EOF` (line 8689).
7. An interrupt occurs before `pcread()` executes line 8691, causing `pcrint()` to be called in the middle of the call to `pcread`.
8. Since `pc11.pcstate` is now `READING`, the `if` block at line 8726 is taken.
9. At line 8727, `pcrint()` discovers that the device has reported an error, so it sets `pc11.pcstate` to `EOF` (line 8728) and then calls `wakeup` (line 8734). This call to `wakeup` does nothing, since the (interrupted) call to `pcread()` has not yet reached its `sleep()` call on line 8693, and no other user process is currently reading the device.
10. `pcrint()` returns, and `pcread()` resumes where it left off, at line 8691.
11. `pcread()` reaches line 8693, where it calls `sleep()` to wait for more characters to arrive from the device. But notice that `pc11.pcstate` is now `EOF`, because `pcrint()` set this condition *after* `pcread()` checked for it in line 8689. Because `pc11.pcstate` is `EOF` and not `READING`, the `wakeup(&pc11.pcin)` call on line 8734 can never be reached even if further device interrupts occur, causing the user process to sleep forever.

Name:

III Virtual memory

Here is the layout of virtual memory that you set up in lab 2.



Name:

Attached to the quiz is the `i386_vm_init` that was provided to you. Assume you completed `i386_vm_init` correctly.

5. [10 points]: What entries (rows) in the page directory have been filled in after `i386_vm_init` has completed? What addresses do they map and where do they point? In other words, fill out this table as much as possible:

Entry	Base Virtual Address	Points to (logically):
1023	0xffc00000	Page table for top 4MB of phys mem
1022	0xff800000	.
.	.	.
.	.	.
.	.	.
.	.	.
960	KERNBASE (0xf0000000)	Page table for low 4MB of phys mem
959	VPT (0xefc00000)	Page directory (kernel-only, R/W)
958	ULIM (0xef800000)	Page table mapping kernel stack
957	UVPT (0xef400000)	Page directory (kernel/user, R-O)
956	UPAGES (0xef000000)	Page table mapping "pages" array
955	UTOP,UENVS (0xeec00000)	Page table mapping "envs" array
954	.	Nothing mapped
.	.	.
.	.	.
.	.	.
.	.	.
.	.	.
.	.	.
.	.	.
.	.	.
.	.	.
.	.	.
.	.	.
.	.	.
.	.	.
2	0x00800000	.
1	0x00400000	.
0	0x00000000	Nothing mapped

Name:

6. [5 points]:

Here is a section of the course staff solution to Lab 2's `i386_vm_init`. This section sets up the UPAGES mapping.

```
////////////////////////////////////////////////////////////////////
// Make 'pages' point to an array of size 'npage' of 'struct Page'.
// You must allocate this array yourself.
// Map this array read-only by the user at virtual address UPAGES
// (ie. perm = PTE_U | PTE_P)
// Permissions:
//    - pages -- kernel RW, user NONE
//    - the image mapped at UPAGES  -- kernel R, user R
// Your code goes here:
n = npage*sizeof(struct Page);
pages = alloc(n, BY2PG, 1);
boot_map_segment(pgdir, UPAGES, n, PADDR(pages), PTE_U);
```

A common mistake is to add the line:

```
boot_map_segment(pgdir, (u_int)pages, n, PADDR(pages), PTE_W);
```

This line is unnecessary, because the mapping already exists. Why does the mapping already exist? Explain exactly which other code has already provided the mapping. You may find it useful to refer to the `i386_vm_init` attached to this quiz.

The `alloc()` function, used to allocate the `pages` array, always returns a kernel virtual address in the region from `KERNBASE` to $2^{32}-1$ in which physical memory is directly and contiguously mapped into the kernel's address space. You already established a mapping for this entire region of the kernel virtual address space, in the immediately preceding section of `i386_vm_init()`, under this comment:

```
////////////////////////////////////////////////////////////////////
// Map all of physical memory at KERNBASE.
// Ie.  the VA range [KERNBASE, 2^32 - 1] should map to
//      the PA range [0, 2^32 - 1 - KERNBASE]
// We might not have that many(ie. 2^32 - 1 - KERNBASE)
// bytes of physical memory.  But we just set up the mapping anyway.
// Permissions: kernel RW, user NONE
```

Name:

7. [5 points]:

In Lab 3, `env.c` creates the user-level address space for an environment. If the code that created the address space was buggy and did not set up a mapping for the area starting at KERNBASE, when would this bug manifest itself? What specific instruction would cause the bug to "take effect" (triple-fault the processor)? (Note: you can answer this question without having completed lab 3.)

When the kernel attempts to switch to the new environment, `env_run()` will call `lcr3()` to load the new environment's page directory into the processor's page directory base register (PDBR/CR3) and flush the TLB. Since the kernel's code itself resides in the virtual address range from `KERNBASE` to $2^{32} - 1$, when the processor tries to execute the next instruction immediately after the "move to CR3" instruction, it will take a page fault because that virtual address is no longer mapped. In order to handle this fault, the processor attempts to push the old CS, EIP, and EFLAGS on the stack—but since the kernel stack is also in this virtual address region starting at `KERNBASE`, these memory accesses will also fail, causing a triple fault.

8. [5 points]: On the x86, we make the kernel's `Page` structures accessible to the user environments (in the form of the mapping at UPAGES). What specific mechanism (i.e., what register, memory address, or bit thereof) is used to keep the user environments from changing the `Page` structures?

The mapping of the `pages` array at virtual address `UPAGES` includes the permission bit `PTE_U`, allowing code running in user mode access to this mapping. This mapping does *not* include the permission bit `PTE_W`, however, which ensures that this mapping cannot be used to write to the `pages` array (by code running in either user or kernel mode).

Name:

IV System calls

9. [5 points]: Draw the kernel stack after v6's `icode` called its first instruction and the kernel just entered `trap` in trap.c (sheet 26).

Old PSW
Old PC
Old r0
PS in trap
Old r1
User SP
PS & 037 = error code (`dev`)
Address of line 0785

Name:

10. [10 points]: What are the values of the arguments to trap? (Fill out the following table.)

dev	6 - the low 4 bits of the PSW the processor loads from the trap vector defined at line 0518.
sp	0 - the "top" of the user stack, and the top of virtual address space - i.e., 2^16 wrapped to 0.
r1	irrelevant
nps	030346 - PM=user, CM=kernel, spl7, dev=6
r0	irrelevant
pc	2 - just after syscall instruction in icode
ps	170000 - PM=user, CM=user

11. [5 points]: Briefly describe the point of the statement on line 3188.

Sets the saved user-mode PC to zero in order to begin execution of the newly loaded program at virtual address zero.

Name:

V Thread switching

A process in UNIX v6 switches to another process using `retu`, which is called on line 2228.

12. [10 points]: Annotate every line of assembly of `_retu` (reproduced from sheet 07). What does the statement do and why?

```
_retu:
   bis     $340, PS     # Disable interrupts (set SPL=7 in PSW) to
                        # prevent interference while switching stacks.

   mov     (sp)+, r1    # Pop return address of retu's caller
                        # off of old stack, save in r1

   mov     (sp), KISA6  # Switch kernel's u-area (page 6) mapping to
                        # the _new_ process's u-area (including stack)

   mov     $_u, r0      # Load address of bottom of u-area (struct user)
                        # into r0

1:
   mov     (r0)+, sp    # Load new process's saved sp (r6) from u_rsav[0]


   mov     (r0)+, r5    # Load new process's saved r5 from u_rsav[1]


   bic     $340, PS     # Re-enable interrupts


   jmp     (r1)         # Return to the code that called retu,
                        # except executing on the new process's stack!
```

13. [5 points]: What does the stack pointer point to at line 2229 in the first call to swtch, after the kernel booted?

The stack pointer points into process 1's kernel stack at this point, specifically at the stack frame established by `newproc()` and saved by the `savu()` call on line 1889. The call from `main()` to `newproc()` on line 1627 originally set up this stack frame in the context of process 0, but then `newproc()` copied process 0's kernel stack (lines 1897-1916) in order to initialize process 1's kernel stack. It is this snapshot of process 0's kernel stack that the first call to `swtch()` switches to.

Name:

VI Feedback

Since 6.828 is a new subject, we would appreciate receiving some feedback on how we are doing so that we can make corrections. (Any answer, except no answer, will receive full credit!)

14. [2 points]: What is the best aspect of 6.828?

15. [2 points]: What is the worst aspect of 6.828?

16. [2 points]: If there is one thing that you would like to see changed in 6.828, what would it be?

End of Quiz I

Name:

Here is `i386_vm_init` from Lab 2.

```
// Set up a two-level page table:
//    boot_pgdir is its virtual address of the root
//    boot_cr3 is the physical adresss of the root
// Then turn on paging.  Then effectively turn off segmentation.
// (i.e., the segment base addrs are set to zero).
//
// This function only sets up the kernel part of the address space
// (ie. addresses >= UTOP).  The user part of the address space
// will be setup later.
//
// From UTOP to ULIM, the user is allowed to read but not write.
// Above ULIM the user cannot read (or write).
void
i386_vm_init(void)
{
  Pde *pgdir;
  u_int cr0, n;

  panic("i386_vm_init: This function is not finished\n");

  //////////////////////////////////////////////////////////////////////
  // create initial page directory.
  pgdir = alloc(BY2PG, BY2PG, 1);
  boot_pgdir = pgdir;
  boot_cr3 = PADDR(pgdir);

  //////////////////////////////////////////////////////////////////////
  // Recursively insert PD in itself as a page table, to form
  // a virtual page table at virtual address VPT.
  // (For now, you don't have understand the greater purpose of the
  // following two lines.)

  // Permissions: kernel RW, user NONE
  pgdir[PDX(VPT)] = PADDR(pgdir)|PTE_W|PTE_P;

  // same for UVPT
  // Permissions: kernel R, user R
  pgdir[PDX(UVPT)] = PADDR(pgdir)|PTE_U|PTE_P;

  //////////////////////////////////////////////////////////////////////
  // Map the kernel stack (symbol name "bootstack"):
  //   [KSTACKTOP-PDMAP, KSTACKTOP)  -- the complete VA range of the stack
  //     * [KSTACKTOP-KSTKSIZE, KSTACKTOP) -- backed by physical memory
  //     * [KSTACKTOP-PDMAP, KSTACKTOP-KSTKSIZE) -- not backed => faults
  //   Permissions: kernel RW, user NONE
  // Your code goes here:

  //////////////////////////////////////////////////////////////////////
  // Map all of physical memory at KERNBASE.
  // Ie.  the VA range [KERNBASE, 2^32 - 1] should map to
  //      the PA range [0, 2^32 - 1 - KERNBASE]
```

Name:

```
// We might not have that many(ie. 2^32 - 1 - KERNBASE)
// bytes of physical memory.  But we just set up the mapping anyway.
// Permissions: kernel RW, user NONE
// Your code goes here:

////////////////////////////////////////////////////////////////////
// Make 'pages' point to an array of size 'npage' of 'struct Page'.
// You must allocate this array yourself.
// Map this array read-only by the user at virtual address UPAGES
// (ie. perm = PTE_U | PTE_P)
// Permissions:
//    - pages -- kernel RW, user NONE
//    - the image mapped at UPAGES  -- kernel R, user R
// Your code goes here:

////////////////////////////////////////////////////////////////////
// Make 'envs' point to an array of size 'NENV' of 'struct Env'.
// You must allocate this array yourself.
// Map this array read-only by the user at virtual address UENVS
// (ie. perm = PTE_U | PTE_P)
// Permissions:
//    - envs itself -- kernel RW, user NONE
//    - the image of envs mapped at UENVS  -- kernel R, user R
// Your code goes here:

check_boot_pgdir();

////////////////////////////////////////////////////////////////////
// On x86, segmentation maps a VA to a LA (linear addr) and
// paging maps the LA to a PA.  I.e. VA => LA => PA.  If paging is
// turned off the LA is used as the PA.  Note: there is no way to
// turn off segmentation.  The closest thing is to set the base
// address to 0, so the VA => LA mapping is the identity.

// Current mapping: VA KERNBASE+x => PA x.
//     (segmentation base=-KERNBASE and paging is off)

// From here on down we must maintain this VA KERNBASE + x => PA x
// mapping, even though we are turning on paging and reconfiguring
// segmentation.

// Map VA 0:4MB same as VA KERNBASE, i.e. to PA 0:4MB.
// (Limits our kernel to <4MB)
pgdir[0] = pgdir[PDX(KERNBASE)];

// Install page table.
lcr3(boot_cr3);

// Turn on paging.
cr0 = rcr0();
cr0 |= CR0_PE|CR0_PG|CR0_AM|CR0_WP|CR0_NE|CR0_TS|CR0_EM|CR0_MP;
cr0 &= ~(CR0_TS|CR0_EM);
lcr0(cr0);
```

Name:

```
  // Current mapping: KERNBASE+x => x => x.
  // (x < 4MB so uses paging pgdir[0])

  // Reload all segment registers.
  asm volatile("lgdt _gdt_pd+2");
  asm volatile("movw %%ax,%%gs" :: "a" (GD_UD|3));
  asm volatile("movw %%ax,%%fs" :: "a" (GD_UD|3));
  asm volatile("movw %%ax,%%es" :: "a" (GD_KD));
  asm volatile("movw %%ax,%%ds" :: "a" (GD_KD));
  asm volatile("movw %%ax,%%ss" :: "a" (GD_KD));
  asm volatile("ljmp %0,$1f\n 1:\n" :: "i" (GD_KT));  // reload cs
  asm volatile("lldt %0" :: "m" (0));

  // Final mapping: KERNBASE+x => KERNBASE+x => x.

  // This mapping was only used after paging was turned on but
  // before the segment registers were reloaded.
  pgdir[0] = 0;

  // Flush the TLB for good measure, to kill the pgdir[0] mapping.
  lcr3(boot_cr3);
}
```

Name:

MIT OpenCourseWare
http://ocw.mit.edu

6.828 Operating System Engineering
Fall 2012

For information about citing these materials or our Terms of Use, visit: http://ocw.mit.edu/terms.

I Processes, stacks, and concurrency

1. [5 points]: Explain briefly one benefit of having one kernel stack per process (as in v6) rather than a single kernel stack (as in JOS).

Having a per-process stack allows the kernel to release the processor to another process when a system call needs to wait, for example while reading from the disk.

2. [5 points]: In v6's `getblk`, lines 4953 through line 4958 are protected by `spl6()`. Describe something that could go wrong if this protection was eliminated.

These lines handle the case where the requested block is not in the buffer cache, and there are no free buffers in `bfreelist` in which `getblk()` can store the block read from disk. `getblk()` sets `B_WANTED` to indicate it is waiting for a free buffer, and sleeps on `&bfreelist`.

If the `spl6()` were not there, a disk interrupt might occur after the check on line 4953 but before the sleep on line 4955. The interrupt might call `iodone()`, which might call `brelse()` to put a buffer on the free list. In that case, `brelse()` would call `wakeup(&bfreelist)` and clear `B_WANTED` before the interrupted process called `sleep()`. As a result the process would not wake up even though there is a free buffer.

3. [5 points]: Two v6 processes call `read()` at about the same time for different blocks that are not in the cache. There are many blocks on `bfreelist`, and no other active processes. As a result, there are two calls to `getblk()`, both of which allocate a free buffer in lines 4960 through 4975 at about the same time. Could the two processes end up taking the same buffer from the free list? If yes, how could that happen? If no, what prevents that from happening?

No, this could not happen. In V6, context switches between processes executing in the kernel can occur only during `sleep()` or just before a process returns to user space. As long as no interrupt-time code removes a buffer from `bfreelist` (and none does) there will no problem.

4. [5 points]: Suppose the B_BUSY flag were eliminated from the v6 source, so that for example lines 4941 through 4946 were deleted. Explain a scenario in which incorrect behavior would result.

In vague terms, the B_BUSY flag indicates that a process is actively using this buffer, for example because it is waiting for the result of a disk read to be put there. Eliminating the flag could result in kernel data structures being put in an inconsistent state.

For example, suppose a process calls `read()`, and the resulting `getblk()` call from `bread()` does not find the block in the cache. `getblk()` will put the buffer on the device's buffer list with the desired block number and device, but (so far) without correct contents. Then `bread()` will call `rkstrategy()`, which (among other things) sets the `av_forw` pointer in the `struct buf` to 0 and then waits for the disk read to complete. If, meanwhile, another process calls `getblk()` for the same block number, it will call `notavail()` from line 4948, which will crash because `av_forw` is zero.

5. [10 points]: It turns out that `savu` doesn't really need to save both `r5` and `sp`, as long as `retu` cooperates. Below you'll see a new version of `savu` that saves only `r5`. The `retu` code below is copied exactly from the v6 source; please modify it so that it works with the new `savu`. `retu` is called only by `swtch` (sheet 21) and `expand` (sheet 22).

```
/* the new savu: */
_savu:
        bis     $340,PS
        mov     (sp)+,r1
        mov     (sp),r0
        /* DELETED: mov  sp,(r0)+ */
        mov     r5,(r0)+
        bic     $340,PS
        jmp     (r1)
```

```
/* the new retu: */
_retu:
    bis     $340,PS
    mov     (sp)+,r1
    mov     (sp),KISA6
    mov     $_u,r0
1:
    /* DELETED: mov (r0)+,sp */
    mov     (r0)+,r5
    mov     r5, sp  /* ADDED */
    sub     $6, sp  /* ADDED */
    bic     $340,PS
    jmp     (r1)
```

The `sub $6, sp` preserves the three registers saved by the `csv` at the entry to the function that called `savu`. They must be preserved since they are callee-saved registers which will be needed when we return to the function that called the function that called `savu`.

II Address spaces

6. [10 points]: A process calls `exec()` to run an executable whose first four 16-bit values (in octal) are 410, 1010, 200, 100. In decimal: 264, 520, 128, 64. Write down the content of the prototype segmentation registers after the call to **estabur()** on line 3152 has completed. Please write the addresses and lengths in decimal.

	length	bits		address
PDR 0	8	RO	PAR 0	0
PDR 1	2	RW	PAR 1	16 (USIZE)
PDR 2	0	0	PAR 2	0
PDR 3	0	0	PAR 3	0
PDR 4	0	0	PAR 4	0
PDR 5	0	0	PAR 5	0
PDR 6	0	0	PAR 6	0
PDR 7	108	RW|ED	PAR 7	16+3+20−128 = −89

7. [5 points]: What is the main benefit of keeping the text area separate from the user and data area? (Explain briefly)

It allows the kernel to keep the text section read-only, which may catch some accidental stores to incorrect addresses. It also allows different processes executing the same program to share the physical memory that holds the program's instructions.

8. [5 points]: Why is it convenient for the user stack's physical memory to start just above the user data? What does the kernel have to do if the user program wishes to use more data memory? (Explain briefly)

The kernel can describe a process's physical memory with just two numbers, the start address and the length.

If a program needs to grow its stack or data, in general the kernel will need to copy the user area, data, and stack to a new contiguous region of physical memory that's large enough to hold the sum of the new sizes.

III JOS

The staff solution for the user-level `pgfault` in `lib/fork.c` starts as follows:

```
static void
pgfault(struct UTrapframe *utf)
{
   int r;
   void *addr = (void*)utf->utf_fault_va;
   uint32_t err = utf->utf_err;

   if (debug)
       cprintf("fault %08x %08x %d from %08x\n", addr,
                      %&vpt[VPN(addr)], err & 7, (&addr)[4]);

   if ((vpt[VPN(addr)] & (PTE_P|PTE_U|PTE_W|PTE_COW)) != (PTE_P|PTE_U|PTE_COW))
          panic("fault at %x with pte %x from %08x, not copy-on-write",
             addr, vpt[PPN(addr)], (&addr)[4]);

        // rest of the pgfault
        .....
}
```

9. [10 points]: What is `vpt`? How did the kernel set up an environment's virtual address space to make this code fragment work correctly? (Explain briefly)

`vpt` refers to a region of virtual address space into which the kernel has mapped the current process's page table. The kernel sets the page directory entry for the address `vpt` to point to the physical address of the page directory itself.

The staff solution for kernel-level `page_fault_handler` in `kern/trap.c` contains the following fragment:

```
page_fault_handler(struct Trapframe *tf)
{
    uint32_t fault_va;
    struct UTrapframe *utf;

    // Read processor's CR2 register to find the faulting address
    fault_va = rcr2();

    // some code for earlier labs
    ....

    // lab 4 code
    if (curenv->env_pgfault_upcall == 0) {
        cprintf("[%08x] user fault va %08x ip %08x\n",
            curenv->env_id, fault_va, tf->tf_eip);
        print_trapframe(tf);
        env_destroy(curenv);
    }

    // Second if statement:
    if (tf->tf_esp >= UXSTACKTOP - PGSIZE &&
        tf->tf_esp < UXSTACKTOP) {
        utf = (struct UTrapframe*)(tf->tf_esp
                    - sizeof(struct UTrapframe)
                    - 4);
    } else {
        utf = (struct UTrapframe*)(UXSTACKTOP
                    - sizeof(struct UTrapframe));
    }
    utf->utf_fault_va = fault_va;
    utf->utf_err = tf->tf_err;
    utf->utf_regs = tf->tf_regs;
    utf->utf_eip = tf->tf_eip;
    utf->utf_eflags = tf->tf_eflags;
    utf->utf_esp = tf->tf_esp;

    tf->tf_esp = (uintptr_t) utf;
    tf->tf_eip = (uintptr_t) curenv->env_pgfault_upcall;
    page_fault_mode = PFM_NONE;

    env_run(curenv);
}
```

10. [10 points]: What is the purpose of the second `if` statement (labeled "second if statement" in the code)? (Explain briefly)

The second `if` detects a page fault in the user fault handler. In that case, the kernel pushes the new `UTrapframe` onto the user exception stack, rather than at `UXSTACKTOP`.

IV File systems

11. [5 points]: The file system can be viewed as graph with i-nodes as nodes and directory entries as links. Using `link` can a program create cycles in the v6 i-node graph? If so, show a sequence of commands that creates a cycle. If not, how does v6 prevent cycles? (Explain briefly)

V6 prevents cycles by disallowing links to directories, except for the superuser. The superuser can create a cycle with the following commands:

```
# mkdir foo
# chdir foo
# link ../foo bar
```

12. [5 points]: Suppose that because of kernel bug a v6 process were able to get a file descriptor that referred to a free i-node (that is, `i_mode` is zero). Explain briefly why this would be undesirable, giving an explicit example of a bad outcome.

Suppose process A has such a file descriptor. If process B then creates a new file and is allocated that i-node, then A will be able to read B's data. That would be bad. If both processes wrote to the file, that would also be bad.

13. [10 points]: Explain how a v6 process can acquire a file descriptor that references a free i-node. Your explanation is not allowed to involve crashes, reboots, or any other external source of file system corruption. It must only involve ordinary file-oriented system calls, such as `creat`, `open`, `close`, and `unlink`. This question is difficult. Look at lines 7663 through 7664.

Two processes are required to exploit the bug. Suppose one process calls `open("a", 0)` for a file that exists at the time of the call. `open()` calls `namei()`, and `namei()` calls `iget()` on line 7664 to fetch `a`'s i-node from the disk. If `iget()` blocks waiting for the disk, then the other process could run and call `unlink("a")`. Then `iget()` in the first process will fetch a free i-node; since there's no check for that error, `open()` would then return a file descriptor to that i-node.

The following code provides evidence of the bug.

```
/*
 * exercise bug at line 7663/7664 of v6 namei.
 *
 * http://pdos.csail.mit.edu/6.828/2005/inode-thing.c
 *
 * first, mkdir d
 *
 * one process keeps creating d/a over and over, writing "d/a" into it.
 * the other process keeps deleting d/a and creating a different file,
 * containing "d/XXXX" (for some number XXXX).
 * so if the first process ever reads d/a but sees anything other than
 * "d/a" in the file, the bug occured.
 *
 * I think this output is evidence of the problem:
 * # cc x.c
 * # ./a.out
 * starting
 * read d/a failed, ret 0, errno 0
 * done
 */

extern errno;
char junk[20*512];

cr(s)
     char *s;
{
  int fd, n;

  fd = creat(s, 0666);
  if(fd < 0){
    printf("creat %s failed\n", s);
    exit(1);
  }
  if(write(fd, s, strlen(s)) != strlen(s)){
    printf("small write %s failed\n", s);
    exit(1);
  }
  close(fd);
}

main()
{
  int pid, fd, n, i;
  char name[10];

  for(i = 0; i < 1000; i++){
```

```
    name[0] = 'd';
    name[1] = '/';
    name[2] = '0' + (i / 1000) % 10;
    name[3] = '0' + (i / 100) % 10;
    name[4] = '0' + (i / 10) % 10;
    name[5] = '0' + (i / 1) % 10;
    name[6] = '\0';
    unlink(name);
  }

  printf("starting\n");

  pid = fork();
  if(pid < 0){
    perror("fork");
    exit(1);
  }

  if(pid == 0){
    while(1){
      cr("d/a");

      fd = open("d/a", 2);
      if(fd >= 0){
        errno = 0;
        n = read(fd, junk, sizeof(junk));
        if(n < 3){
          printf("read d/a failed, ret %d, errno %d\n",
                 n, errno);
          sleep(2);
          errno = 0;
          if((n = write(fd, "deadbeef\n", 9)) < 0){
            printf("write haha to bad d/a, ret %d, errno %d\n",
                   n, errno);
          }
          exit(1);
        }
        if(junk[2] != 'a'){
          printf("read %d, did not start with a\n", n);
          exit(1);
        }
        close(fd);
      }
    }
  } else {
    for(i = 0; i < 1000; i++){
      unlink("d/a");
    name[0] = 'd';
    name[1] = '/';
    name[2] = '0' + (i / 1000) % 10;
    name[3] = '0' + (i / 100) % 10;
    name[4] = '0' + (i / 10) % 10;
    name[5] = '0' + (i / 1) % 10;
    name[6] = '\0';
      cr(name);
    }
    printf("done\n");
    kill(pid, 9);
  }
}
```

MIT OpenCourseWare
http://ocw.mit.edu

6.828 Operating System Engineering
Fall 2012

For information about citing these materials or our Terms of Use, visit: http://ocw.mit.edu/terms.



Department of Electrical Engineering and Computer Science

MASSACHUSETTS INSTITUTE OF TECHNOLOGY

6.828 Operating System Engineering: Fall 2004

Quiz I Solutions

All problems are open-ended questions. In order to receive credit you must answer the question as precisely as possible. You have 80 minutes to answer this quiz.

Write your name on this cover sheet AND at the bottom of each page of this booklet.

Some questions may be much harder than others. Read them all through first and attack them in the order that allows you to make the most progress. If you find a question ambiguous, be sure to write down any assumptions you make. Be neat. If we can't understand your answer, we can't give you credit!

THIS IS AN OPEN BOOK, OPEN NOTES QUIZ.

1 (xx/40)	2 (xx/25)	3 (xx/25)	4 (xx/10)	Total (xx/100)

Name:

I Virtual memory

1. [5 points]: The PDP-11 uses two bits in the Processor Status Word (PSW) to determine whether it is currently running in kernel or user mode. Which bits in which register on the x86 determine whether the processor is running in kernel or user mode, and what x86 instruction does the JOS kernel use to enter user mode? (Explain briefly)

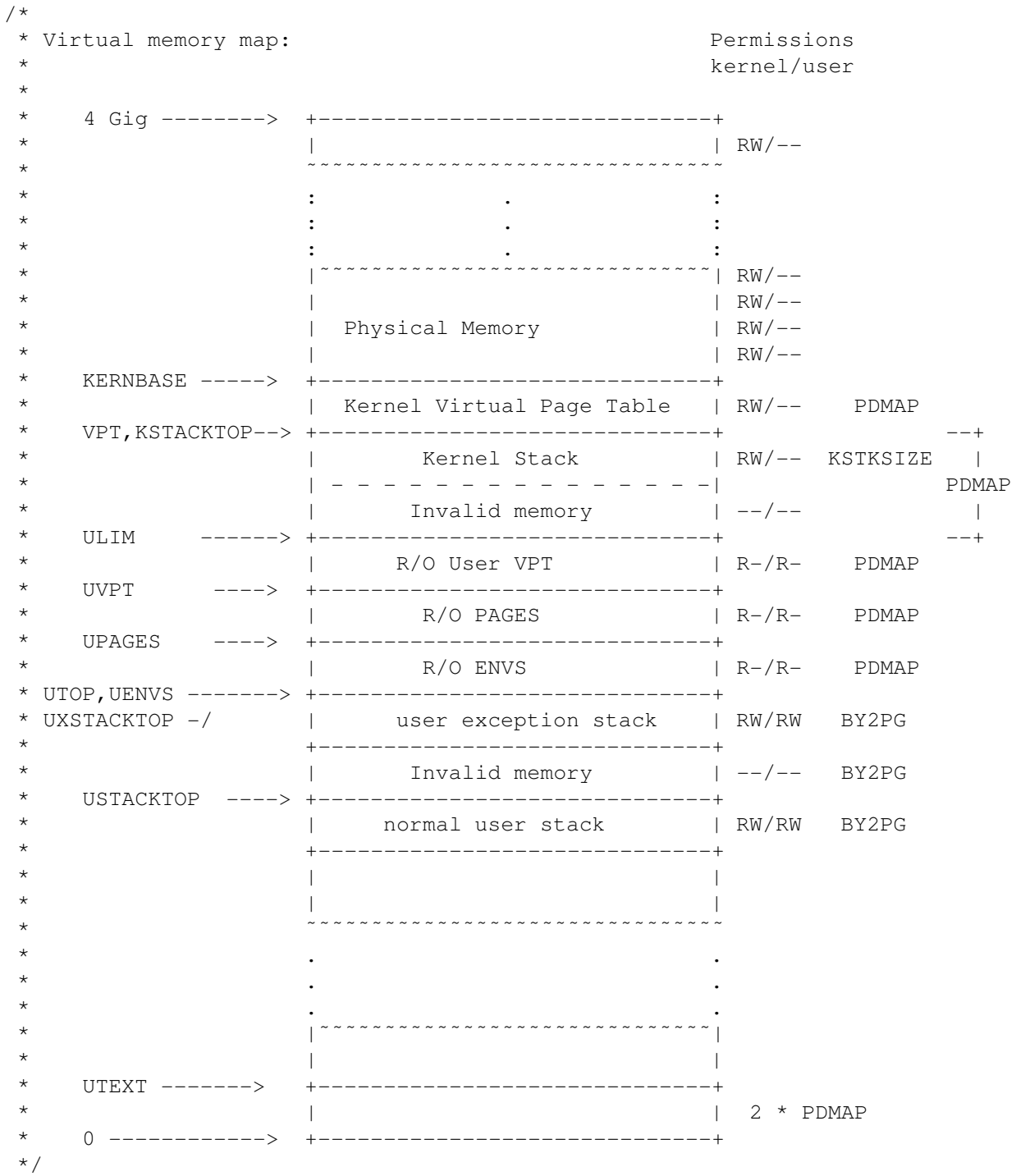
The low two bits of the Code Segment (CS) register *represent the x86 processor's Current Privilege Level (CPL): 0 is the most privileged level ("kernel mode"), 3 is the least privileged level ("user mode"), and the other two values represent intermediate privilege levels that most operating systems do not use.*

The JOS kernel uses **the IRET instruction** *to enter (or "return to") user mode. IRET loads the EIP, CS, EFLAGS, ESP, and SS registers from the kernel stack, in effect changing to the user privilege level, switching to the user's stack, and jumping to the appropriate user code all in one atomic processor operation.*

2. [5 points]: The v6 kernel must copy values from user space with the special instruction `mfpi` (push onto the current stack the value of the designated word in the "previous" address space). What instructions does the JOS kernel use to access user-level values? (Explain briefly)

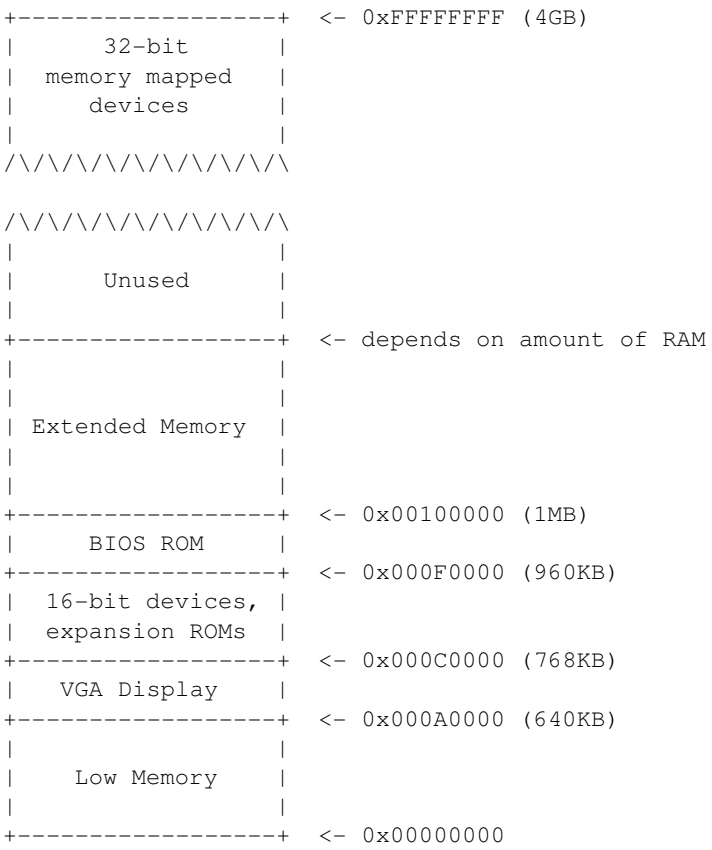
No special instructions are required *for the JOS kernel to access the user address space, because JOS maps the kernel's private virtual address region identically into every user environment's virtual address space, protecting the kernel-private mappings from user-mode access via the PTE_U bit in the appropriate page directory and page table entries. Whenever the kernel is executing in the context of (on behalf of) a particular user environment whose address space is active (loaded into the processor's CR3 register), as long as the kernel takes appropriate precautions to protect itself from invalid pointers passed by the user,* **the kernel can access the user's address space with ordinary instructions such as MOV**, *using the same user virtual addresses that the user environment itself would use to access its own memory.*

Here is the layout of virtual memory that you set up in lab 2.



Name:

Also, here is the layout of physical memory in a PC:



3. [5 points]: Why does the JOS kernel arrange that the virtual addresses from `KERNBASE` and higher have read-write access in kernel mode and no access in user mode? (Explain briefly.)

The JOS kernel needs to reserve part of the virtual address space for its own use because privilege level changes on the x86 (i.e., transitions in the CPL field of the CS register) do not cause the processor to make an automatic address space transition, as the PDP-11 does when switching privilege levels. On the x86, the kernel therefore needs "a place to stand" in the user's address space during transitions between kernel and user mode, and simply mapping all of the physical memory that the kernel makes use of into every user environment's address space is the easiest way to solve this problem.

In order to protect the kernel from buggy or malicious user environments, and to protect different user environments from each other, *the kernel must ensure that this special kernel-private portion of each environment's address space remains completely inaccessible to user environments while they are running in user mode. Allowing user environments to write into the virtual address region above KERNBASE would enable users to modify arbitrary kernel data structures and thereby compromise the integrity of the entire system, while allowing user environments to read in this region would allow any user environment to "spy on" all other running environments and find sensitive information such as passwords or cryptographic keys.*

Name:

You completed the following function in lab 2:

```
//
// Initialize page structure and memory free list.
//
page_init(void)
{
        // The exaple code here marks all pages as free.
        // However this is not truly the case.  What memory is free?
        //  1) Mark page 0 as in use(for good luck)
        //  2) Mark the rest of base memory as free.
        //  3) Then comes the IO hole [IOPHYSMEM, EXTPHYSMEM) => mark it as in u
se
        //     So that it can never be allocated.
        //  4) Then extended memory(ie. >= EXTPHYSMEM):
        //     ==> some of it's in use some is free. Where is the kernel?
        //     Which pages are used for page tables and other data structures?

        //
        // Change the code to reflect this.
        int i;
        LIST_INIT (&page_free_list);
        for (i = 0; i < npage; i++) {
                pages[i].pp_ref = 0;
                LIST_INSERT_HEAD(&page_free_list, &pages[i], pp_link);
        }
}
```

Name:

4. [10 points]: Which pages in extended memory should `page_init` mark as in use? (Explain briefly and feel free to refer to the virtual memory map.)

The kernel's `page_init` *function must mark all of the extended memory* **from 1MB to the final value of the** `freemem` **variable** *as in use. The kernel's code, data, and bss itself is located in extended memory starting at physical address 0x100000 (1MB), between the magic linker-defined symbols* `start` *and* `end`*. Immediately after the kernel executable is the physical memory the kernel has already allocated before* `page_init` *time, via the boot-time* `alloc` *function, for various critical kernel data structures such as the kernel's page tables, the* `pages` *array, and the* `envs` *array. The* `page_init` *function must make sure that the physical pages used to hold the kernel's program itself and these critical boot-time-allocated structures do not get reused for other purposes during subsequent memory allocations.*

5. [5 points]: Why should `page_init` mark page 0 as in use for good luck? (Explain briefly.)

The real-mode IDT and certain important BIOS data structures are historically located in physical page 0. *To be specific, the real-mode IDT is from 0x000 through 0x3ff, and the BIOS data area is from 0x400 through 0x4ff. If JOS ever were need to switch back to real mode and make calls back to the system's built-in BIOS, for example to support Advanced Power Management (APM) or to detect physical memory beyond the 64MB that can be registered in the system's CMOS RAM, then these data structures would need to be preserved. JOS currently does not do any BIOS callbacks, however, which is why preserving page 0 is at the moment merely "for good luck."*

We included this admittedly obscure question not because it has any particularly critical importance to OS design or implementation principles, but as a reward for those students who have been faithfully coming and paying attention to lectures. :)

Name:

In lab 3 you wrote the following code to setup the IDT for system calls in `idt_init`:

```
SETGATE(idt[T_SYSCALL], 1, GD_KT, handler_SYSCALL, 3);
```

6. [5 points]: Why must the descriptor protection level be set to 3 instead of 0 (as with most of the other entries in the IDT)? (Explain briefly.)

The processor checks the descriptor privilege level (DPL) in the IDT gate descriptor against the current privilege level (CPL) in the CS register to determine whether to allow a software interrupt (`int`* instruction) to execute using that interrupt vector. Therefore,* **the DPL for the system call gate must be set to 3 in order for the processor to permit system calls from user mode via an** `int $0x30` **instruction.** *(*`T_SYSCALL` *is 0x30.)*

7. [5 points]: If the descriptor protection level is set to 0, what would happen if the processor executes an "int 30" instruction? (Explain briefly.)

The processor takes a **General Protection Fault (GPF)** *instead of using the interrupt vector requested by the* `int` *instruction.*

Name:

II Proccesses

8. [5 points]: In which structure does v6 allocate the kernel stack? (Explain briefly.)

The v6 kernel stack for a given process is located **in the u-area** *for that process. A process's u-area consists of the kernel stack and the* `user` *structure.*

9. [5 points]: How many of these structures does v6 allocate? (Explain briefly.)

The v6 kernel allocates **one kernel stack per process***. The v6 kernel is designed this way so that when kernel code running on behalf of a given process needs block waiting for some event, the kernel can retain state on the blocked process's kernel stack even as other processes are allowed to continue running independently. Retaining the state of blocked processes on separate kernel stacks in this way makes it easier to implement complex, high-level functionality such as file systems within the kernel.*

10. [5 points]: The structure (and thus the kernel stack) is stored at the *virtual* address 140000 and up. What must happen when the kernel switches to a different process? (Explain briefly.)

Whenever the v6 kernel switches from one process to another, **it must change the value in KPAR6, the Kernel Page Address Register for this "magic" region of the kernel's virtual address space, to refer to the new process's u-area.** *In this way, regardless of which process is currently running, the current process's u-area always appears at virtual address 140000 in the kernel's address space.*

11. [10 points]: In v6, the `init` process, pid 1, inherits processes whose parent have exited. Sketch out the required changes to the kernel so that children that have no parent anymore clean themselves up on `exit` (sheet 32).

The trickiest part of modifying the v6 kernel is to decide how to deal with deallocating the orphan process's stack. Two possible approaches are: either modifying `swtch` *to free memory; or by adding a reaper process to the kernel (although, depending on how it is implemented, the later is probably not too much different from the way the* `init` *process is used to clean up orphans right now). We cannot simply free the u-area in* `exit` *since, the u-area contains the specific kernel stack being used to execute the* `exit` *code. To free all the memory being used by a process, we need to switch to a different kernel stack.*

Some other points: processes need a way to tell if they are orphans (e.g. storing `-1` *in* `p_ppid`*); zombie processes and the cleanup code in* `wait` *will still be needed for the case of non-orphan processes;* `exit` *may need to be modified to clean up zombie children that haven't been* `wait`*'d on when an orphan* `exit`*'s;* `exit` *needs to make orphans of children who aren't zombies instead of making them children of the init process; some of the code from* `wait` *to dealloc/reinit memory may be added to* `exit` *(e.g. the* `mfree` *and the code to reset the proc table entry).*

Answers mentioning some of the above points, within a coherent kernel modification plan, were given credit. Students had to realize the complication with the kernel stack to score full points on this quesiton.

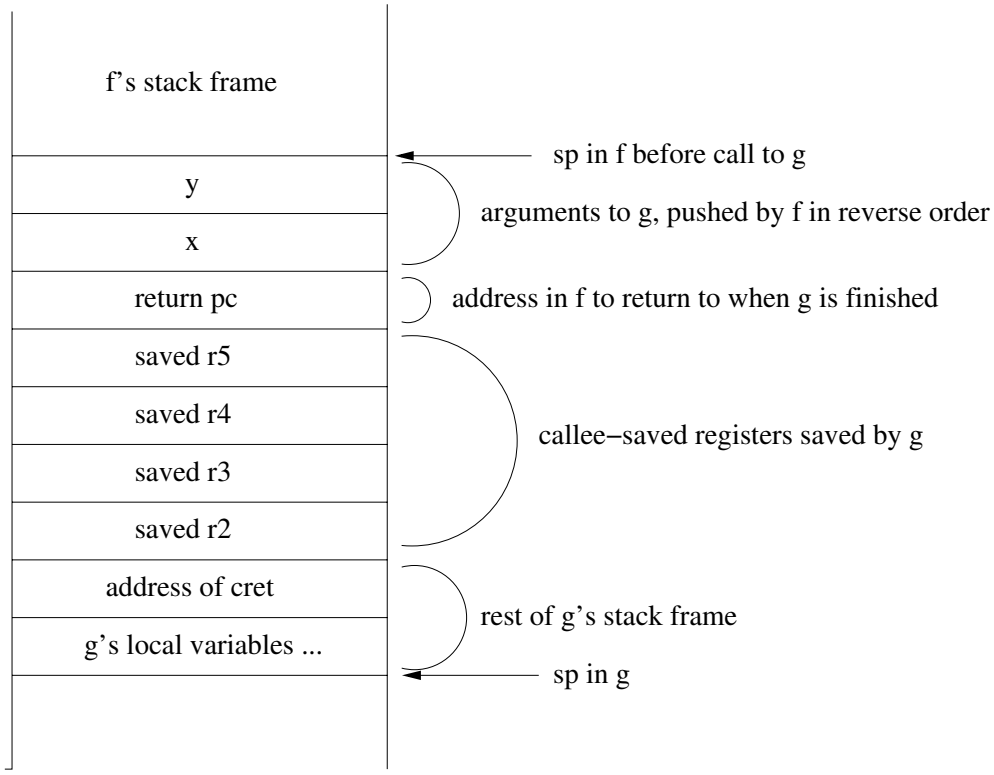
Students who answered this question assuming that we were asking you to modify the kernel so that a parent would kill off all of its children when `exit` *was called misinterpreted the question. Such answers scored low because they showed a lack of understanding of the unix process model and the purpose of having zombie processes.*

III Calling conventions

12. [5 points]: Assume a function `f` calls a function `g` using the C calling conventions of v6 on the PDP-11. The callee function `g` takes two arguments, as follows:

```
void g(int x, int y) {
...
}
```

Below is the stack at the point that `f` is about to call `g`. So far `f`'s own local variables and other private state are already on the stack, but nothing related to its call to `g`. Draw the stack at the point just before the processor executes the first instruction of `g` that does "real" work, after running `g`'s function prologue. (The information on page 10-3 of the commentary might be helpful.)



We gave full credit to any solution that mentions and lists in the correct order: the arguments y and x, the return address, and callee-saved registers.

13. [5 points]: Annotate your picture by providing for each value a brief explanation why the value is pushed on the stack.

Name:

The following PDP-11 assembly function appears (with a different name) in the v6 C library:

```
_mystery:
        mov     r5,-(sp)
        mov     2(r5),r1                / pc of caller of caller
        mov     sp,r5
        clr     r0
        cmp     -4(r1),jsrsd
        bne     8f
        mov     $2,r0
8:
        cmp     (r1),tsti
        bne     1f
        add     $2,r0
        br      2f
1:
        cmp     (r1),cmpi
        bne     1f
        add     $4,r0
        br      2f
1:
        cmp     (r1),addi
        bne     1f
        add     2(r1),r0
        br      2f
1:
        cmp     (r1),jmpi
        bne     1f
        add     2(r1),r1
        add     $4,r1
        br      8b
1:
        cmpb    1(r1),bri+1
        bne     2f
        mov     r0,-(sp)
        mov     (r1),r0
        swab    r0
        ash     $-7,r0
        add     r0,r1
        add     $2,r1
        mov     (sp)+,r0
        br      8b
2:
        asr     r0
        mov     (sp)+,r5
        rts     pc

.data
jsrsd:  jsr     pc,*$0
tsti:   tst     (sp)+
cmpi:   cmp     (sp)+,(sp)+
addi:   add     $0,sp
jmpi:   jmp     0
bri:    br      .
```

Name:

For your convenience, Russ Cox has translated it into some equivalent pseudocode:

```
mystery:
        cpc = *(r5+2)   /* cpc = pc of caller of caller */
        n = 0
        if inst[cpc-4] == "jsr pc, *$x"
                n = 2
again:
        if inst[cpc] == "tst (sp)+"
                n += 2
        else if inst[cpc] == "cmp (sp)+,(sp)+"
                n += 4
        else if inst[cpc] == "add $x, sp"
                n += x
        else if inst[cpc] == "jmp x"
                cpc += x
                goto again
        else if inst[cpc] == "br .+x"
                cpc += x
                goto again
        return n/2
```

14. [15 points]: What the does the `mystery` function compute? (Please, give a brief explanation.)

This mystery function determines **the number of 2-byte arguments** *that were passed to the function that called it. For example, suppose we have a function* `f` *that takes a variable number of arguments, like* `printf`*, and it is called from another function* `g`*:*

```
void f(int x, ...) {
  printf("I was called with %d arguments!\n", mystery());
}

void g() {
  f(1, 2, 3, 4);
}
```

Then in this case `f` *will report that it was called with 4 arguments. The "mystery function" determines this number by scanning the code in* `g` *that will be run immediately after* `f` *returns, and counting the number of 2-byte words that* `g` *pops off the stack immediately after calling* `f`*. Since the return address to* `g` *is popped off by* `f` *itself while returning, and* `g`*'s local variables and callee-saved registers will be popped off later by* `g`*'s epilogue code, the stack pop instructions that the mystery function counts consists of exactly the argument words pushed onto the stack by* `g` *for* `f`*.*

The correct functioning of this mystery function of course depends on the authors of the function knowing exactly which instructions the compiler might use to pop off arguments after calling a function. Kernighan and Ritchie, having written the C compiler along with the operating system, could be much more confident in this knowledge than most OS designers today would be.

Name:

IV 6.828

15. [6 points]: Describe the most memorable error you have made so far in one of the labs. (Provide enough detail that we can understand your answer.)

We love to know what your experience is in 6.828. Please, answers the following two questions. (Any answer, except no answer, will receive full credit!)

16. [2 points]: What is the best aspect of 6.828?

17. [2 points]: What is the worst aspect of 6.828?

End of Quiz I

Name:

MIT OpenCourseWare
http://ocw.mit.edu

6.828 Operating System Engineering
Fall 2012

For information about citing these materials or our Terms of Use, visit: http://ocw.mit.edu/terms.



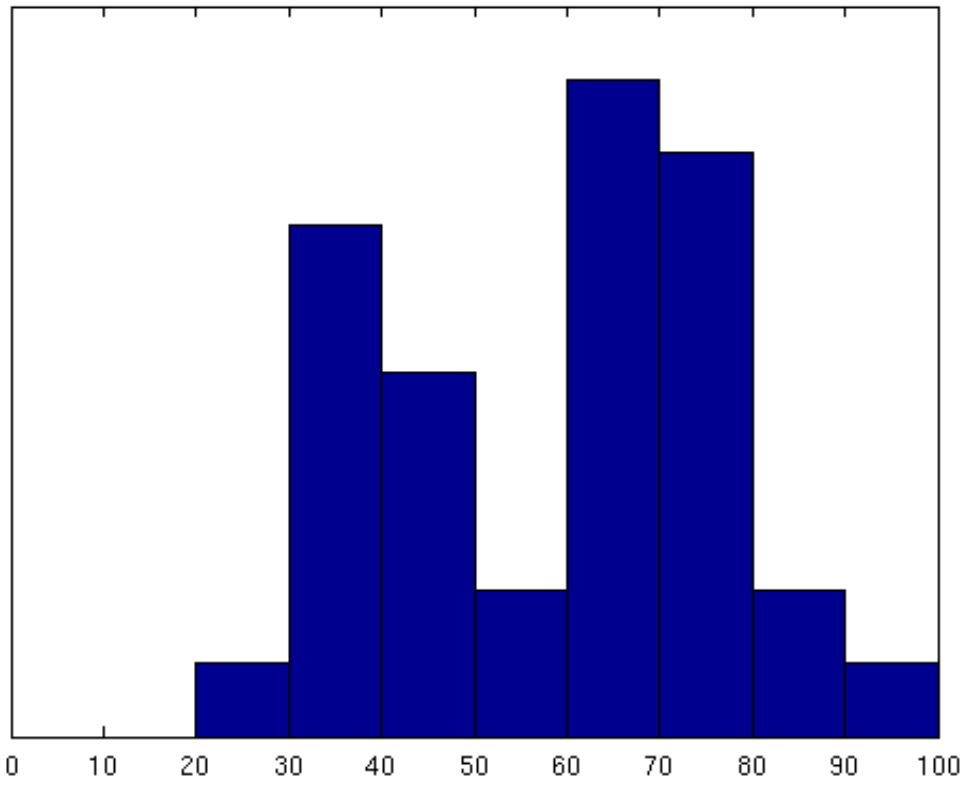
Department of Electrical Engineering and Computer Science

MASSACHUSETTS INSTITUTE OF TECHNOLOGY

6.828 Fall 2007

Quiz I Solutions

The mean score was 59, and the median was 64.



I Boot

1. [4 points]: Explain how xv6 ensures that the boot loader (bootasm.S and bootmain.c) copies the kernel from disk to memory in a way that avoids overwriting the boot loader's own instructions or stack.

Answer: The xv6 kernel is linked at 0x100000, so the ELF header information will cause bootmain() to only overwrite memory at 0x100000 and above. The boot loader's instructions are at 0x7c00, and its stack is just below that, so they won't be overwritten by the kernel.

2. [5 points]: Bugs Bunny, after reading xv6's `userinit()` (sheet 17), claims that the start-up of the first process could be made more efficient by arranging for `swtch()` to directly enter the process. Specifically he suggests changing line 1774 to read

```
p->context.eip = (uint) p->mem; // swtch() directly to start of initcode
```

Why is this change a bad idea?

Answer: The change affects what swtch() will do when scheduler() eventually calls it: swtch() will cause the CPU to execute initcode's instructions, but with the kernel's privileges and in the kernel's address space. Perhaps the most immediate problem will be that proc_table_lock won't be released (ordinarily that swtch() jumps to forkret(), which releases the lock).

II Processes and Address Spaces

3. [5 points]: JOS has a single kernel stack, while xv6 has one kernel stack per process. Explain how this restricts what the JOS kernel can do, and mention a specific feature of xv6 that would be difficult in the JOS kernel because it has only the one stack.

Answer: xv6 can have system calls that block in the middle to wait for I/O, for example to wait for the various disk reads required to look up a file pathname. The process's kernel stack saves the intermediate state of the system call while it blocks. A JOS system call cannot block in the middle, it can only block at the very end.

The Intel x86 allows the stack segment (SS) and data segment (DS) to differ. This separation allows an operating system to permit a process to expand both segments independently as needed, without worrying about the stack running into the heap. xv6, however, always makes the two segments the same (for example, see `userinit()` on sheet 17). As a result, an xv6 process's stack cannot expand beyond the memory initially allocated for it.

4. [5 points]: Explain why it would be awkward for xv6 to give a process different data and stack segments (i.e. have DS and SS refer to descriptors with different BASE fields).

Answer: It's common for C code to create pointers to data on the stack, as well as pointers to data in the heap. Functions such as strlen() that operate on pointers would need to know whether to use the pointers with DS or SS. So either the compiler would have to be modified to use 48-bit pointers (32 bits of offset plus a segment selector), or the programmer would have to keep track of this information by hand for each pointer.

III Kernel Entry

5. [5 points]: When a user-space process executes INT, INT switches to a stack specified in the TSS. JOS sets up the TSS to point to a stack in kernel memory. What might go wrong in JOS if the INT instruction *didn't* change stacks?

Answer: The environment might set its ESP to an invalid address before the INT, which would cause the kernel to fault. The environment might set its ESP to point to a protected kernel data structure, so that the INT would save EFLAGS &c on the kernel's data. If JOS had system calls like read() that copied data to user space, then the environment might be able to trick the kernel into executing arbitrary code by read()ing data onto the stack: the data could overwrite one of the kernel's saved return EIPs with a pointer to instructions in the environment's memory.

6. [12 points]: Explain why the `pushl` at xv6 line 6610 is needed.

Answer: Ordinarily that space on the stack would hold the return program counter from the call to the C library system call routine. The kernel expects something to be there on the stack when it is calculating the address from which to fetch an argument, for example on line 2696. If the pushl were deleted, argint() would return the wrong argument.

7. [12 points]: xv6 defines two structures that hold saved registers for a process: `struct trapframe` on sheet 05, and `struct context` on sheet 15. Explain a situation in which a single process will have *three* sets of saved registers.

Answer: A process makes a system call (saving user registers in a trapframe), a clock interrupt happens just as trap() is returning (saving kernel registers for the same process in another trapframe lower on the same stack), and this second call to trap() calls yield() and thus swtch() (saving kernel registers for the same process in a struct context).

IV Locking

8. [5 points]: xv6's `acquire()` turns off interrupts at line 1431 only if `nlock` is zero. `release()` turns them back on only if `nlock` (after decrementing) is zero (at line 1466). What could go wrong if `acquire()` *always* turned off interrupts, and `release()` *always* turned interrupts on?

Answer: If a process acquires two locks, and then releases one of them, this modification would cause interrupts to be turned on. In that case there is a risk that an interrupt routine (such as ide_intr()) could try to acquire the lock that is still held. That would produce a deadlock (actually a panic in acquire()).

Suppose you swapped xv6 lines 1869 and 1870, so that `yield()` looked like this:

```
void
yield(void)
{
  cp->state = RUNNABLE;
  acquire(&proc_table_lock);
  sched();
  release(&proc_table_lock);
}
```

9. [8 points]: Would anything go wrong? Explain why or why not.

Answer: The scheduler() on a different CPU might see that the process is RUNNABLE and run it. Then two CPUs would be executing on the process's kernel stack, overwriting each others' variables and return EIPs, which is likely to cause trouble.

V Context Switch

When one xv6 process switches to another, two calls to `swtch()` are involved: one from `sched()` to `scheduler()`, and one from `scheduler()` to `sched()`. Bugs Bunny is obsessed with `swtch()` and suggests eliminating one of the calls: `sched()` could call a variant of `scheduler()` directly. The new code:

```
// caller must hold proc_table_lock.
void
sched(void)
{
  scheduler2();
}

// caller must hold proc_table_lock.
void
scheduler2(void)
{
  struct proc *p;
  struct proc *from = cp; // remember who we are
  int i;

  for(;;){
    for(i = 0; i < NPROC; i++){
      p = &proc[i];
      if(p->state != RUNNABLE)
        continue;

      cp = p;
      setupsegs(p);
      p->state = RUNNING;
      swtch(&from->context, &p->context);

      // a return from swtch() means some other call to
      // scheduler2() decided to run us.
      cp = from;
      setupsegs(cp);
      return;
    }

    release(&proc_table_lock);
    acquire(&proc_table_lock);
  }
}
```

10. [12 points]: This new `swtch()`-less `sched()` works most of the time, but not always. What's likely to go wrong?

Answer: Suppose a process sleep()s waiting for an IDE interrupt. The CPU on which it was running will use the process's kernel stack to execute scheduler2(). When the IDE interrupt occurs and marks the process RUNNABLE, the scheduler on a different CPU may run the process. Then there will be two CPUs using the same kernel stack, which is sure to cause trouble.

11. [5 points]: What's the reason for the `release()` and immediate re-`acquire()` at the end of Bugs's `scheduler2()`?

Answer: Without the release() and acquire(), interrupts would always be off when there are no RUNNABLE processes, permanently preventing delivery of (for example) keyboard and IDE interrupts. Another problem is that, if there is just one process RUNNING, it may spin forever next time it tries to acquire process_table_lock since the lock may be permanently held by an idle CPU.

VI Sleep/Wakeup

In `exit()` (sheet 20), suppose you swapped lines 2023 and 2026 and changed the `wakeup1()` to `wakeup()`, so that the resulting code looked like:

```
// Parent might be sleeping in proc_wait.
wakeup(cp->parent);
acquire(&proc_table_lock);
```

12. [8 points]: What is likely to go wrong as a result of this change to `exit()`?

Answer: Suppose the parent is in wait() just before the sleep(), and the child's exit() calls wakeup(). Then the parent will miss the wakeup and sleep forever.

VII File System

The policy that `bget()` line 3576 and `brelse()` enforce is that at most one process at a time can use a buffer. One reason is to prevent other processes from using a buffer between the time it is allocated in `bget()` and when the IDE driver finishes reading valid data from the disk into the buffer (and sets `B_VALID`).

13. [8 points]: Suppose `bget()` and `brelse()` were modified to let multiple processes use a `B_VALID` buffer at the same time, while ensuring that a buffer is only re-used for a different sector when the buffer is not in use. Please describe a specific problem that would arise.

Answer: The file system code uses the one-at-a-time rule to provide mutual exclusion for code that reads and then updates the file system's own on-disk data structures. For example, if two processes call balloc() at the same time and bget() allowed them to use the same buffer at the same time, they might both allocate the same free block. It would be a serious error for the same disk block to be used by two different files.

VIII 6.828

We'd like to hear your opinions about 6.828, so please answer the following questions. (Any answer, except no answer, will receive full credit.)

14. [2 points]: What ideas in the course have been the hardest to understand?

Answer: The difference between virtual and physical memory.

15. [2 points]: What is the best aspect of 6.828?

Answer: The labs.

16. [2 points]: What is the worst aspect of 6.828?

Answer: Debugging the labs.

End of Quiz

MIT OpenCourseWare
http://ocw.mit.edu

6.828 Operating System Engineering

Fall 2012

For information about citing these materials or our Terms of Use, visit: http://ocw.mit.edu/terms.



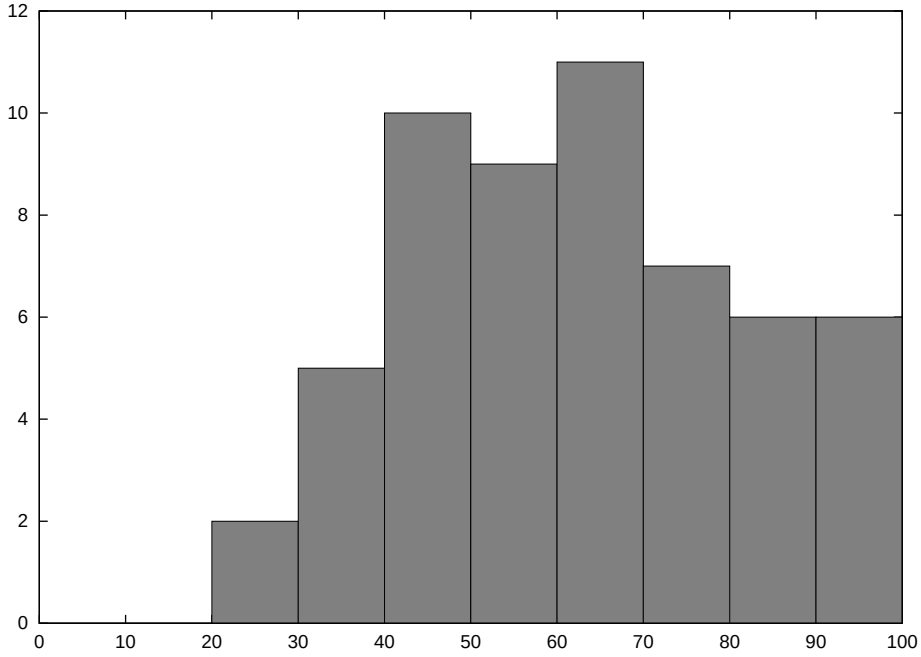
Department of Electrical Engineering and Computer Science

MASSACHUSETTS INSTITUTE OF TECHNOLOGY

6.828 Fall 2008

Quiz I Solutions

The mean score was 62, median was 61, and standard deviation 18.8.



I System Call Arguments

1. [6 points]: Given a user-provided pointer to a buffer in user memory (such as the argument of a system call), explain what checks and translations the JOS kernel must do to ensure that it can safely read or write the buffer memory. Do not assume or rely on the existence of a function in the kernel that does these checks and translations for you.

Answer: The JOS kernel does not need to translate the address, since the environment's memory is mapped into the kernel at the same addresses that the environment itself uses. JOS needs to make four safety checks before it can dereference a pointer supplied by the environment without fear of incurring a page fault or reading or writing outside the environment's memory. 1) The PTE_U flag must be set in the corresponding PTE (equivalently, the address must be less than UTOP). 2) The PTE_P bit must be set in the PTE. 3) If JOS needs to write through the pointer, the PTE_W bit must be set. 4) The above three conditions need to hold for each page of the environment's buffer.

2. [6 points]: Given a user-provided pointer to a buffer in user memory (such as the argument of a system call), explain what checks and translations the xv6 kernel must do to ensure that it can safely read or write the buffer memory. Do not assume or rely on the existence of a function in the kernel that does these checks and translations for you.

Answer: xv6 must check that the address a of each byte in the buffer is inside the process's memory, by checking that $0 \leq a < $ `p->sz`. xv6 must translate the address by adding `p->mem`.

II File System

3. [8 points]:

Describe two situations in which a power failure during a system call could leave different parts of the xv6 file system metadata inconsistent with each other. By "metadata" we mean the file system's on-disk data structures: the superblock, inodes, indirect blocks, the block in-use bitmap, and directory contents. A power failure halts the CPU and stops the disk after completion of the current sector read/write request (if any). Be sure to indicate the line number after which you assume the power failure occurs for each of the two situations, and the nature of the resulting inconsistencies.

Answer: There are many possibilities. Some lead to a block or inode not being marked free, but also not being used. Some lead to user-visible incorrectness.

Suppose a process is writing to a file, and `bmap()` decides it must allocate a new block to hold the data (line 4019), and calls `balloc()`. If a crash were to occur after line 3718 in `balloc()`, the block would be marked as in-use in the on-disk bitmap, but no inode would refer to it. This would not cause any immediate damage if the file system were to be used after a reboot, but it would decrease the amount of usable space in the file system.

As a second example, suppose a process is creating a new directory with the `mkdir()` system call, which calls `create()`, and the system crashed after line 4836. The new directory would be visible in its parent directory (due to the `dirlink()` at line 4831) but the new directory would not have "." or ".." entries. If the system rebooted and a user process looked at the new directory it would look odd.

4. [10 points]: Alice is worried that her xv6 kernel might panic at line 3596 because it runs out of struct bufs, of which there are only 10 (line 3529). Since she is mostly concerned with reliability, and does not care about the performance benefits of using these buffers as a cache, she decides to replace bget() and brelse() as follows:

```
static struct buf* bget(uint dev, uint sector) {
    struct buf *b = kalloc(sizeof(*b));
    memset(b, 0, sizeof(*b));
    b->flags |= B_BUSY;
    b->dev = dev;
    b->sector = sector;
    return b;
}

void brelse(struct buf *b) {
    kfree(b, sizeof(*b));
}
```

She modifies kalloc() to allow any size (rather than just multiples of 4096). Assuming that kalloc() never fails, describe a concrete correctness problem she will run into with this code.

Answer: This code allows the kernel threads of multiple processes to acquire a `struct buf` for the same disk block at the same time via `bread()`. Then, for example, if `balloc()` were invoked concurrently from two threads, `bread()` would return copies of the same bitmap block to both `balloc()`s, which would return the same free block to both threads. This could result in two different files using the same disk block for their data. The real `bget()` only lets one thread at a time use any given disk block, which serializes modifications to each block. Many functions rely on this serialization to avoid races or to avoid overwriting unrelated modifications to the same block: `balloc()`, `bfree()`, `ialloc()`, `ilock()`, `iupdate()`, and perhaps others.

III Paging vs. segmentation

JOS sets up the x86 page table to contain PTE entries for all kernel code and data structures (at a high virtual address, KERNBASE = 0xF00000000), even when running a user-space environment. The code and data for the user-space environment is mapped at low virtual memory addresses.

5. [4 points]: Explain why it is safe to keep sensitive kernel data structures mapped at 0xF0000000 while executing user-level code, which should not be able to access those kernel data structures.

Answer: JOS only sets the PTE_U bit in the PTEs of pages that the current environment is allowed to use, and in particular, does not set PTE_U for kernel memory mapped above 0xF0000000. The x86 paging hardware doesn't allow access to pages without PTE_U when CPL is 3.

6. [6 points]: Explain why JOS needs the kernel mapped while executing user-level code.

Answer: Both user and kernel memory must be mapped in the page table in order to cross the user/kernel boundary in either direction. For example, in order for user-level code to make a system call with INT, the IDT, GDT, task state segment, kernel stack, and kernel trap handlers must be at addresses that are present in the page table.

The xv6 kernel lives at low addresses, starting at 0x00100000. xv6 user-level code resides at memory addresses starting from zero, so a program with more than 1MB of instructions will use addresses that overlap with the kernel's addresses.

7. [6 points]: Explain how the processor allows both the xv6 kernel and user-level processes to store instructions at the same addresses.

Answer: The kernel and user-level processes use different segments in xv6, and the x86 hardware switches CS and SS segments automatically when crossing between user and kernel (and the kernel trap code switches DS). The user and kernel segments have different bases (`p->mem` and 0, respectively), so a given address refers to different physical memory depending on when user or kernel code is executing.

IV Traps and interrupts

Recall that gcc calling conventions allow any function to trash the "caller-saved" registers %eax, %ecx, %edx, so that they may be different upon return.

8. [6 points]: xv6's system call stubs (sheet 68) are called by user-space C code as if they were ordinary C functions, so all callers are prepared for the stubs to not preserve caller-saved registers. The stubs invoke the INT instruction, which vectors into kernel code that shortly jumps to alltraps (line 2456). alltraps and trapret save and restore all registers to and from a struct trapframe (line 0477), including %eax, %ecx, and %edx, using pushal and popal, even though gcc does not expect any of these registers to remain the same after return from the system call stub.

Suppose we wanted to reduce the size of struct trapframe by not saving %ecx and %edx in alltraps. It turns out that this would cause problems. Why? Give an example of something that would break.

Answer: Interrupts go through alltraps. If alltraps did not save %ecx and %edx, then the interrupted code would see unexpected changes in those registers.

9. [6 points]: In lab 3, the JOS kernel treats user-mode page faults and errors in system calls differently. For page faults, the kernel destroys the user environment, but for system calls that encounter an error, the kernel sets an error code in the %eax register and resumes the environment at the instruction following the `INT`. Explain what would happen if the JOS kernel, when handling a page fault from user space, were to set an error code in %eax and resume the environment at the instruction following the one that caused a page fault.

Answer: One problem is that user-level instructions that load memory into registers would sometimes not actually do the load. Another problem is that any user-level memory reference instruction might have the side effect of modifying %eax. `gcc` generates code that assumes neither of these things ever happens.

V Virtual vs. physical memory

Ben wants to add support for a graphics card in JOS. The graphics card provides a 1024-by-1024-pixel display, which appears as 1MB of memory at physical address 0xF0000000, with each byte of that memory range corresponding to one pixel on the screen. The graphics card memory is not part of the physical memory detected by i386_detect_memory().

Assuming pixel value 0 corresponds to black, provide C code for clearing the screen to all-black, to be executed just after JOS turns on paging (i.e. in i386_init() just after i386_vm_init returns). The next page includes a copy of `inc/memlayout.h` for your reference.

10. [8 points]:

Answer: The idea is to map the display memory at an unused part of the virtual address space, write zeroes, and then unmap. This code zeroes the display memory one page at a time, mapping each page at UTEMP:

```
pte_t *pte = pgdir_walk(pgdir, (void *) UTEMP, 1);
for(pa = 0xf0000000; pa < 0xf0000000 + 1024*1024; pa += PGSIZE){
  *pte = pa | PTE_P | PTE_W;
  tlb_invalidate(boot_pgdir, UTEMP);
  memset(UTEMP, 0, PGSIZE);
}
*pte = 0;
```

```
/*
 * Virtual memory map:                                Permissions
 *                                                    kernel/user
 *
 *    4 Gig --------->  +------------------------------+
 *                      |                              | RW/--
 *                      ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
 *                      :              .               :
 *                      :              .               :
 *                      :              .               :
 *                      |~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~| RW/--
 *                      |                              | RW/--
 *                      |   Remapped Physical Memory   | RW/--
 *                      |                              | RW/--
 *    KERNBASE ------>  +------------------------------+ 0xf0000000
 *                      |  Cur. Page Table (Kern. RW)  | RW/--  PTSIZE
 *    VPT,KSTACKTOP--> +------------------------------+ 0xefc00000      --+
 *                      |         Kernel Stack         | RW/--  KSTKSIZE   |
 *                      | - - - - - - - - - - - - - - -|                 PTSIZE
 *                      |      Invalid Memory (*)      | --/--             |
 *    ULIM     ------> +------------------------------+ 0xef800000      --+
 *                      |  Cur. Page Table (User R-)   | R-/R-  PTSIZE
 *    UVPT      ----->  +------------------------------+ 0xef400000
 *                      |          RO PAGES            | R-/R-  PTSIZE
 *    UPAGES    ----->  +------------------------------+ 0xef000000
 *                      |           RO ENVS            | R-/R-  PTSIZE
 * UTOP,UENVS ------>  +------------------------------+ 0xeec00000
 * UXSTACKTOP -/       |     User Exception Stack     | RW/RW  PGSIZE
 *                     +------------------------------+ 0xeebff000
 *                     |       Empty Memory (*)       | --/--  PGSIZE
 *    USTACKTOP  --->  +------------------------------+ 0xeebfe000
 *                     |      Normal User Stack       | RW/RW  PGSIZE
 *                     +------------------------------+ 0xeebfd000
 *                     |                              |
 *                     |                              |
 *                     ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
 *                     .                              .
 *                     .                              .
 *                     .                              .
 *                     |~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~|
 *                     |     Program Data & Heap      |
 *    UTEXT -------->  +------------------------------+ 0x00800000
 *    PFTEMP ------->  |       Empty Memory (*)       |        PTSIZE
 *                     |                              |
 *    UTEMP -------->  +------------------------------+ 0x00400000      --+
 *                     |       Empty Memory (*)       |                   |
 *                     | - - - - - - - - - - - - - - -|                   |
 *                     |  User STAB Data (optional)   |                 PTSIZE
 *    USTABDATA ---->  +------------------------------+ 0x00200000        |
 *                     |       Empty Memory (*)       |                   |
 *    0 ------------>  +------------------------------+                 --+
 */
```

VI Kill

Here is process X running on xv6:

```
main()
{
        while(1) {
        }
}
```

As you can see, process X is in an infinite loop. Another process Y uses the kill() system call to terminate process X.

11. [8 points]: For this question, assume the xv6 system has a single CPU. Is it possible for process X to execute any more user-space instructions after process Y's kill() returns? Explain. If "yes," what will finally cause X to stop executing user-space instructions?

Answer: Yes. If process Y is running, process X cannot be running. The only way process X can not be running is if it were interrupted by a timer interrupt, causing `trap()` to call `yield()` at line 2592. When `kill()` marks process X `RUNNABLE`, X's `yield()` will return, `trap()` will return to user space, and X will execute in user space. Eventually another interrupt (like the timer interrupt) will occur and `trap()` will see that X's `p->killable` flag is set and call `exit()` at 2587.

12. [8 points]: For this question, assume the xv6 system has two CPUs. Is it possible for process X to execute any more user-space instructions after process Y's kill() returns? Explain. If "yes," what will finally cause X to stop executing user-space instructions?

Answer: Yes. Process X is running in user space on a different CPU than Y, and will continue to execute until the next hardware interrupt (like the timer interrupt) on its CPU.

VII CLI

13. [8 points]: What would go wrong if you replaced pushcli()'s implementation (xv6 sheet 14) with just cli(), and popcli()'s implementation with just sti()?

Answer: If a kernel thread acquires two locks and then releases one, `release()` would cause interrupts to be turned on. Then an interrupt could occur, and if the interrupt handling code tried to acquire the one lock that is still held, `acquire()` would panic.

VIII Fork

14. [4 points]: Here's a copy of sys_fork() from sheet 28:

```
int
sys_fork(void)
{
  int pid;
  struct proc *np;

  if((np = copyproc(cp)) == 0)
    return -1;
  pid = np->pid;
  np->state = RUNNABLE;
  return pid;
}
```

Explain what could go wrong if the code looked like this:

```
int
sys_fork(void)
{
  struct proc *np;

  if((np = copyproc(cp)) == 0)
    return -1;
  np->state = RUNNABLE;
  return np->pid;
}
```

Answer: Nothing would go wrong. The `scheduler()` on a different CPU could run the new process just after `sys_fork()` marks the new process `RUNNABLE`, and the new process might immediately call `exit()`. It turns out this does no harm, because the process table slot won't be re-used (i.e. `np->pid` won't be overwritten) until the process calling `sys_fork()` calls `wait()`.

IX 6.828

We'd like to hear your opinions about 6.828, so please answer the following questions. (Any answer, except no answer, will receive full credit.)

15. [2 points]: How could we make the ideas in the course easier to understand?

16. [2 points]: What is the best aspect of 6.828?

17. [2 points]: What is the worst aspect of 6.828?

End of Quiz

MIT OpenCourseWare
http://ocw.mit.edu

6.828 Operating System Engineering

Fall 2012

For information about citing these materials or our Terms of Use, visit: http://ocw.mit.edu/terms.



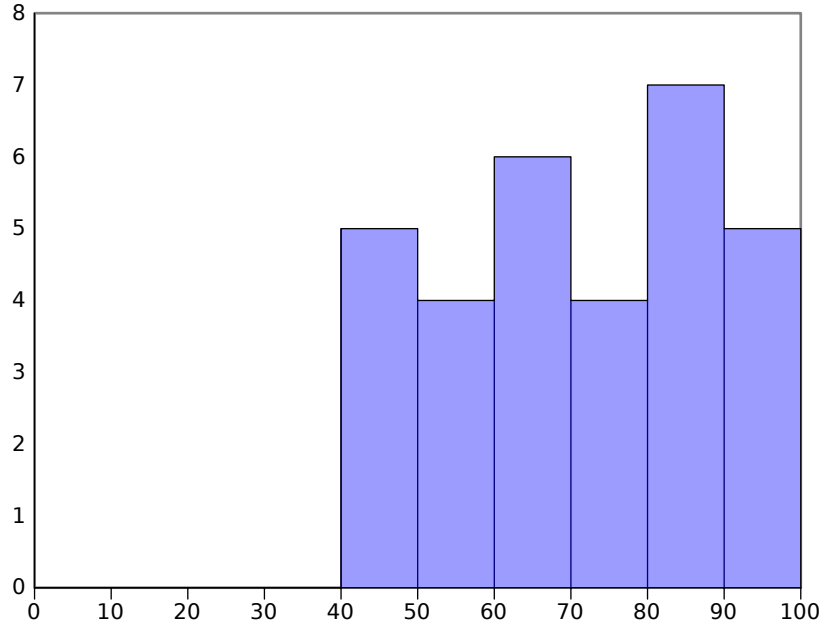
Department of Electrical Engineering and Computer Science

MASSACHUSETTS INSTITUTE OF TECHNOLOGY

6.828 Fall 2009

Quiz I Solutions

Mean 70.6 Median 73 Std. dev. 17.9



I Concurrency

1. [8 points]: In xv6, the interrupt handler `ideintr` uses the lock `idelock` to ensure that it executes atomically. For example, if processor 0 runs `iderw` and processor 1 runs the interrupt handler `ideintr`, then the lock `idelock` ensures that their critical sections are appropriately ordered. Why is it important that `acquire(&idelock)` also turns off interrupts, in addition to marking the lock as acquired?

Answer: Suppose an IDE interrupt occurred on processor 0 while `iderw` held the `idelock` on processor 0. If `acquire` did not disable interrupts, this interrupt would be serviced immediately, resulting in a call to `ideintr`, which would try to acquire the `idelock`. This would panic because the lock is already held on processor 0. (If `acquire` did not check for this situation, it would simply deadlock, waiting for the lock to be released while blocking the very processor holding the lock.)

2. [8 points]: `bget` (and thus `bread`) returns a locked block to its caller. Why is it important that xv6 locks blocks? Consider `ialloc` (line 3802). What could go wrong from line 3811 through 3817 if the block returned by `bread` wasn't locked?

Answer: Without this, simultaneous calls to `ialloc` might allocate the same inode for two different files.

3. [8 points]: `bget` (line 3566) uses an integer `flags` stored in the returned block to indicate whether a thread is holding the lock on the block, instead of a spinlock. What could go wrong if we replaced the per-block `B_BUSY` flag used by `bget` with a per-block spinlock?

Answer: `sched` would panic. Specifically, when `bread` calls `iderw`, `iderw` puts the I/O in the IDE queue and puts the current process to sleep while waiting for an IDE completion interrupt. However, holding the block's spinlock disables interrupts, so it will never receive this interrupt. `sched` (called by `sleep`), will panic if any spinlocks are held, precisely to catch such mistakes.
Note that `bget` can check whether or not a per-block spinlock is held just as readily as it can check the `B_BUSY` flag, so it is *not* the case that `bget` will block while trying to allocate a fresh block.

4. [4 points]: What ensures that setting `flags` in `bget` (line 3566) is an atomic operation?

Answer: `bget` holds the buffer cache lock, `bcache.lock`, which protects modifications to `flags`.

II File systems

5. [8 points]: Ben wants to speed up the rate at which xv6 can write to the disk. He notices that `iderw` can have only one outstanding disk write per user process; xv6 puts the caller of `iderw` to sleep until the dirty bit is cleared by `ideintr`. Ben proposes to modify `iderw` to not call sleep, but instead return immediately. If a process issues multiple writes, all these writes will end up in the IDE queue, and the driver and disk controller can optimize performing a batch of writes. Ben makes all the required changes to xv6; he observes that write throughput improves for large files and that the system works correctly when there are no failures. Unfortunately, when Ben runs `fsck` after a system failure, he finds he cannot repair the disk into a consistent state anymore. What has gone wrong?

Answer: Ben's change allows metadata writes to be reordered. The consistency of the file system's on-disk structures depends on specific ordering of metadata writes. For example, suppose you deleted a file and then created a new file. If the new file's inode is written to disk before the removal is written to disk, and the system failed between these two writes, then the two files could potentially share the same data blocks.

6. [6 points]: Ben obsesses about performance and proposes a design to speed up directory listings. He stores the inode for each file in the `struct dirent`, a scheme called *embedded inodes*. When a user types "`ls -l`" to get a detailed directory listing, xv6 can avoid reading the inode for each file in the directory, because it is stored with the directory. Ben makes the change, and observes that xv6 is faster for running "`ls -l`". He also notices that hard links don't work correctly any more. Give a sequence of shell commands that will work incorrectly with Ben's embedded inodes and briefly describe what will go wrong.

Answer: Here's one of many possible solutions

```
echo a > a
ln a b
echo bbb > b
ls -l
```

will yield a directory listing where `a` has size 2 and `b` has size 4 (including the newlines produced by `echo`), even though they should be the same file and thus the same size.

7. [6 points]: Suggest a solution to the link problem that achieves the performance benefits of embedded inodes without sacrificing correctness.

Answer: If a file has a link count greater than 1, don't embed the inode. Alternative solutions include tracking the entire set of directories where a given file is linked and updating all copies of the inode when any one of them changes, but this would be very slow and poses serious challenges for file system recovery. Simply using symbol links instead of hard links is *not* a solution, since the semantics of hard links and symbolic links differ significantly.

III Stacks

8. [8 points]: Suppose you wanted to change the system call interface in JOS so that, instead of returning the system call result in EAX, the kernel pushed the result on to the user space stack. Fill in the code below to implement this. For the purposes of this question, you can assume that the user stack pointer points to valid memory.

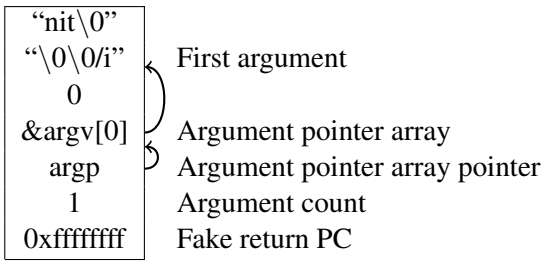
```
static void
trap_dispatch(struct Trapframe *tf)
{
        // ...
        if (tf->tf_trapno == T_SYSCALL) {
                r = syscall(tf->tf_regs.reg_eax, ...);
                // QUIZ: Your code here

        Answer: tf->tf_esp -= 4;
                *(int*)tf->tf_esp = r;

        }
        // ...
}
```

9. [8 points]: xv6's `initcode` invokes the `exec` system call to replace itself with `/init`. `exec` allocates a stack for `/init` and carefully initializes it, starting from line 5080. Draw the stack it builds, giving the values that are on the stack and a brief explanation of each value. For pointers, simply draw an arrow or describe where they point to.

Answer:



10. [2 points]: While debugging xv6, Alyssa notices the value `0x7b70` in the ESP register. Which kernel thread is running on the processor?

Answer: The scheduler thread (on processor 0, to be exact).

IV Virtual Memory

Ben Bitdiddle wants to add a debugger for user programs to his implementation of JOS. He decides to write this debugger as a regular user program, and realizes that he's going to need to add some system calls to support debugging. He starts with a system call that lets the debugger copy a page of memory between its own environment and any other environment.

Ben first implements his new syscall, `sys_copy_page`, by switching to the source environment's address space and copying the data into a scratch buffer, then switching to the destination environment's address space and copying the data back out of the scratch buffer. Ben's code is as follows:

```
static char scratch[PGSIZE];

static int
sys_copy_page(envid_t srcenvid, void *srcva,
              envid_t dstenvid, void *dstva)
{
        struct Env *srcenv, *dstenv;
        int r;

        if ((r = envid2env(srcenvid, &srcenv, 0)) < 0)
                return r;
        if ((r = envid2env(dstenvid, &dstenv, 0)) < 0)
                return r;
        if (srcenv != curenv && dstenv != curenv)
                return -E_INVAL;

        lcr3(srcenv->env_cr3);
        memmove(scratch, srcva, PGSIZE);
        lcr3(dstenv->env_cr3);
        memmove(dstva, scratch, PGSIZE);

        lcr3(curenv->env_cr3);
        return 0;
}
```

11. [8 points]: Louis Reasoner is troubled by Ben's approach. He points out that Ben is copying data in to the scratch buffer while running in one environment's address space, then expecting to read that data back *after* switching to a different address space. Ben assures Louis that this is alright. Why is it okay for Ben's implementation to switch between address spaces like this?

Answer: `scratch` is stored in the kernel BSS, and thus above KERNBASE. All mappings above ULIM are identical in every environment's address space, so switching address spaces does not affect the mapping of `scratch`.

12. [6 points]: Having addressed Louis' concerns, Ben starts boasting about his understanding of address spaces and protection. Alyssa decides to put his kernel to the test and discovers that, not only can she crash Ben's kernel by passing the virtual address of an unmapped page to `sys_copy_page`, but she can write a user program that writes to arbitrary kernel memory. How can Ben fix these two problems?

Answer: Ben should check that `srcva` is mapped with permissions `PTE_P|PTE_U` in `srcenv->env_pgdir` and that `dstva` is mapped with permissions `PTE_P|PTE_U|PTE_W` in `dstenv->env_pgdir`. A convenient way to do this is with `user_mem_assert`:

```
user_mem_assert(srcenv, srcva, PGSIZE, 0);
user_mem_assert(dstenv, dstva, PGSIZE, PTE_W);
```

13. [6 points]: Ben decides to change his strategy to copy data directly from the source page to the destination page without switching address spaces. His new implementation of `sys_copy_page` starts by finding the `struct Page*` of the source page and the physical address of the destination page. Why does this approach let Ben avoid switching address spaces in `sys_copy_page`?

Answer: All physical memory is directly mapped starting at KERNBASE in every environment's address space. Thus, the kernel can convert a physical address to a virtual address that is valid regardless of the current address space.

14. [8 points]: Ben uses a helper function to copy data between the source and destination pages that he looked up in his new `sys_copy_page`. Implement this function below.

```
void
copy_physical_page(physaddr_t dst, struct Page* src)
{
```

Answer:

```
memmove(KADDR(dst), page2kva(src), PGSIZE);
```

```
}
```

V 6.828

We'd like to hear your opinions about 6.828, so please answer the following questions. (Any answer, except no answer, will receive full credit.)

15. [2 points]: This year we posted draft chapters that provide a commentary on xv6. Did you read them? If so, did you find the chapters useful? What should we do to improve them?

16. [2 points]: What is the best aspect of 6.828?

17. [2 points]: What is the worst aspect of 6.828?

End of Quiz

MIT OpenCourseWare
http://ocw.mit.edu

6.828 Operating System Engineering

Fall 2012

For information about citing these materials or our Terms of Use, visit: http://ocw.mit.edu/terms.



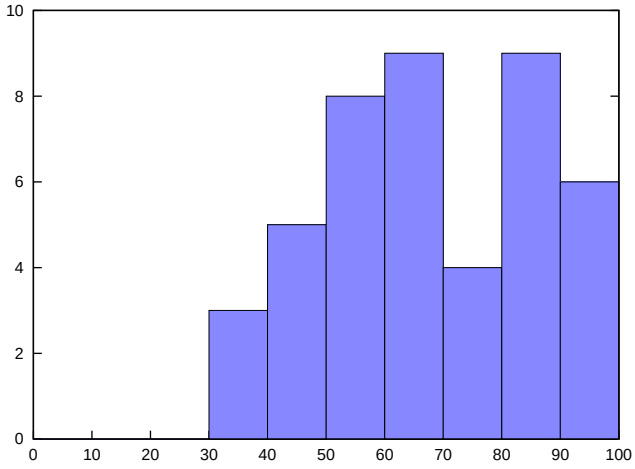
Department of Electrical Engineering and Computer Science

MASSACHUSETTS INSTITUTE OF TECHNOLOGY

6.828 Fall 2010

Quiz I Solutions

Mean 67 Median 66 Std. dev. 18.3



I Concurrency

1. [8 points]: Alyssa runs xv6 on a machine with 8 processors and 8 processes. Each process calls `sbrk` (3351) continuously, growing and shrinking its address space. Alyssa measures the number of `sbrk`s per second and notices that 8 processes achieve the same total throughput as 1 process, even though each process runs on a different processor. She profiles the xv6 kernel while running her processes and notices that most execution time is spent in `kalloc` (2426) and `kfree` (2405), though little is spent in `memset`. Why is the throughput of 8 processes the same as that of 1 process?

Answer: Both `kalloc` and `kfree` hold `kmem.lock` during their whole execution, which will serialize all of the processes so that only one is making progress at a time.

2. [8 points]: Does the JOS kernel have a mechanism equivalent to xv6's `swtch` (2308)? If yes, what? If not, explain why xv6 needs it but JOS does not.

Answer: No. xv6 must switch between the stacks of kernel threads, while JOS has only one kernel stack.

3. [8 points]: xv6 enables interrupts in the kernel during system calls and device interrupts, which adds some complexity since xv6 has to carefully disable and enable interrupts when locking. In contrast, JOS (as you will find in Lab 4 Part C) only enables interrupts in user space, and arranges for the hardware to automatically disable interrupts when entering the kernel. Would anything go wrong if xv6 also disabled interrupts in the kernel? Support your claim.

Answer: Yes. Suppose there is only one running or runnable process and it performs a disk read. The kernel will issue the IDE request and that kernel thread will go to sleep to wait for the IDE interrupt. Since that was the only runnable process, the kernel will enter the idle loop and never wake up because it will never receive the IDE interrupt.
(It is *not* true that blocking system calls simply stop working. A blocking system call will still go to sleep, which will schedule another runnable user process and return to user space, where interrupts will be delivered. The danger is if there are *no* runnable user processes, in which case you stay in the kernel's idle loop with interrupts disabled.)

II File systems

Alyssa add the statement:

```
cprintf("bwrite sector %d\n", b->sector);
```

to `bwrite` (4014), as in one of the homeworks. She then types the following two commands and observes their disk writes:

```
$ echo > x
bwrite sector 4      (inode for x)
bwrite sector 4
bwrite sector 29     (directory data for /)
$ echo ab > x
bwrite sector 28     (bitmap)
bwrite sector 508    (file data for x)
bwrite sector 4
bwrite sector 508
bwrite sector 4
bwrite sector 508
bwrite sector 4
```

Alyssa observes that her commands run slowly because each `bwrite` is synchronous (that is, it waits until the disk finishes writing). She recalls from lecture that this is necessary to help `fsck` be able to repair any inconsistencies after a crash or power failure, but naturally wonders if any of the writes could be made asynchronous. An asynchronous write updates the in-memory buffer for the block and marks it dirty, but does not send the buffer to the IDE driver. Instead, a dirty buffered block is written to the disk on a later synchronous write to that block, after a delay, or when buffer space is needed for some other block. Dirty buffered blocks may be written to disk in any order.

4. [10 points]: Assume xv6 does *no* repair of a file system after a reboot (it has no `fsck`). Alyssa starts by thinking about a single aspect of consistency: preventing the same block from being referenced by two different files; she will worry about other aspects later. Which writes during `echo ab > x` above must be synchronous to ensure this does not happen? Provide a brief explanation.

Answer: Only the write of sector 28, which marks the file data block in-use, must be synchronous. This ensures that the block allocation write cannot occur after the first write to the inode, which creates the reference to the block. If the inode write were to happen first, followed by a crash, then after the system rebooted and read the bitmap back in, it could allocate that block for another purpose, even though it's actually in use. The remaining writes simply update file data and the file length.

Alyssa goes on to wonder how to extend xv6 to support *named pipes*. A named pipe allows the following shell script:

```
$ mkfifo mypipe
$ cat file > mypipe &
$ wc < mypipe
$ rm mypipe
```

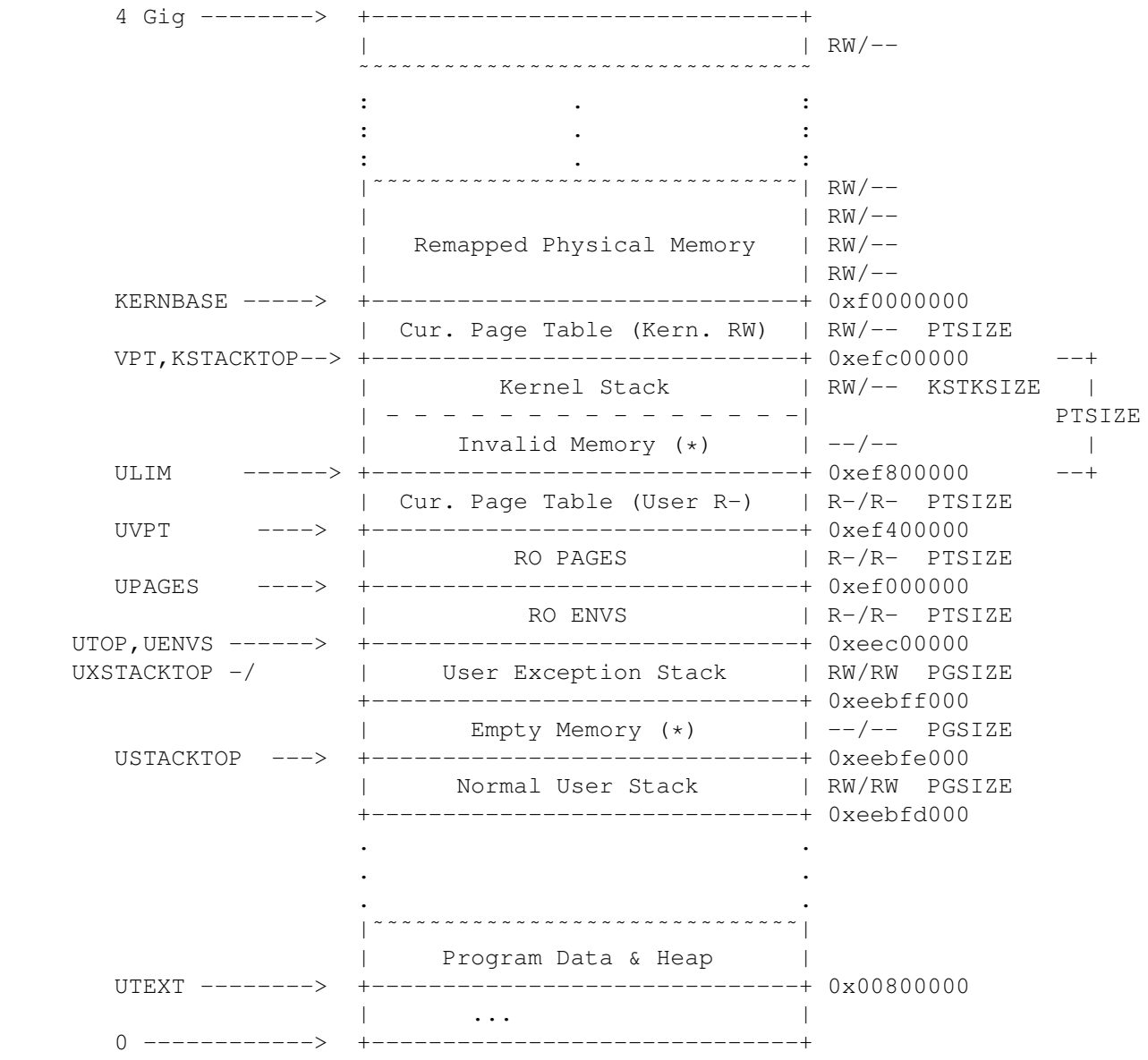
This shell code behaves much like `cat file | wc`, except that the pipe is given a regular file name that is stored in the file system, even though the contents of the pipe are not. Just like a regular pipe, the kernel passes all data from the writer to the reader internally, via in-memory buffers (without writing to the disk). Unlike a regular pipe, a named pipe appears in the file system, as a "special file." As a result, the processes communicating over the pipe don't have to have a parent-child relationship, since they can open the pipe by its file name (as in the example).

5. [12 points]: Describe what modifications you would make to xv6 to support named pipes so that you can run the above shell script. You don't have to provide code; just sketch what modifications you would make in which files. (Hint: exploit the existing pipes.)

Answer: mkfifo would create a file with the new inode type `T_PIPE`. The first open of such a file should create a `struct pipe` using `pipealloc`, record the resulting pair of `struct file`'s in the `struct inode`, and associate the `struct file` with the returned FD. From then on `read()`s and `write()`s would go to the `pipe.c` code. Later opens should simply fetch the appropriate `struct file` from the inode. Various other changes are also necessary to add an argument to `sys_mknod`, handle closing of named pipes, etc.

III Virtual Memory

Here's a simplified version of JOS's virtual memory map from `memlayout.h`.




```
#define KERNBASE 0xF0000000
#define VPT (KERNBASE - PTSIZE)
#define KSTACKTOP VPT
#define ULIM (KSTACKTOP - PTSIZE)
#define UVPT (ULIM - PTSIZE)
#define UPAGES (UVPT - PTSIZE)
#define UENVS (UPAGES - PTSIZE)
#define UTOP UENVS
#define USTACKTOP (UTOP - 2*PGSIZE)
```

6. [8 points]: You have a machine with 4 GB of DRAM (and 32-bit addresses). What new value should you give KERNBASE to allow a single environment to allocate and address the largest number of distinct user memory pages possible? It's okay to be off by a few dozen megabytes. Explain why a KERNBASE higher or lower than your answer would reduce the maximum allocatable environment size.

Answer: About 0x80000000 (2 gigabytes). A higher KERNBASE would reduce the available physical memory; JOS can only use physical pages that it can access above KERNBASE. A lower KERNBASE would reduce user virtual address space available to the environment.

7. [12 points]: You have the following user-level code in JOS:

```
sys_page_alloc(0, 0x1000000, PTE_P|PTE_W|PTE_U);
*(int*)0x1000000 = 111;
sys_page_alloc(0, 0x2000000, PTE_P|PTE_W|PTE_U);
*(int*)0x2000000 = 222;

// your call(s) to sys_page_map() here

if(*(int*)0x1000000 == 222 &&
   *(int*)0x2000000 == 111)
  cprintf("OK\n");
```

Show how to make this code print `OK` using just one or more calls to

```
int sys_page_map(envid_t srcenvid, void *srcva,
                 envid_t dstenvid, void *dstva,
                 int perm);
```

Before the above code, nothing is mapped between 0x1000000 and the user stack.

Answer:

```
sys_page_map(0, 0x1000000, 0, UTEMP, PTE_P|PTE_W|PTE_U);
sys_page_map(0, 0x2000000, 0, 0x10000000, PTE_P|PTE_W|PTE_U);
sys_page_map(0, UTEMP,     0, 0x20000000, PTE_P|PTE_W|PTE_U);
// (the temporary address doesn't have to be UTEMP)
```

IV JOS

Ben Bitdiddle decides to experiment with a new and potentially more flexible system call convention. Rather than passing the syscall number and arguments in registers, he instead writes the syscall number and arguments to a "syscall page" at a new, fixed location `USYSCALL` before invoking `int 0x30`. He defines a `struct SysArgs` for storing the arguments, adds `struct Page *env_syspage` to `struct Env` to record the physical location of an environment's syscall page, and adds the following code to allocate and map the syscall page for each environment:

```
static int
env_setup_vm(struct Env *e) {
  // ...
  page_alloc(&e->env_syspage);
  page_insert(e->env_pgdir, e->env_syspage,
              (void*)USYSCALL, PTE_U|PTE_W);
  return 0;
}
```

8. [10 points]: Ben begins updating his system call implementations to read their arguments from `USYSCALL`. His new convention is working great until he updates his inter-environment message passing syscalls. In his implementation, an environment sends a message by writing the message and its destination environment ID to its syscall page and invoking the kernel, which validates arguments, switches to the destination environment's address space, copies the message from `(struct SysArgs*)USYSCALL`, then switches back to the calling environment's address space. However, Ben finds that this writes the wrong data to the destination environment. Alyssa suggests that Ben should instead read the message from `(struct SysArgs*)page2kva(curenv->env_syspage)`. Explain what was wrong with Ben's approach and why Alyssa's approach fixes Ben's problem.

Answer: `USYSCALL` is a virtual address that will map to a different phyiscal page in each environment's page table. Ben's code is reading the message data from the *destination* environment's address space, instead of the source environment's, which contains the data he wants to copy. Alyssa's approach works because it reads the message via the source environment's physical `USYSCALL` page.

9. [8 points]: Both the original calling convention and Ben's new scheme use an interrupt to invoke the syscall. What prevents him from following the usual C calling convention from the user environment and simply calling the kernel function that implements the desired syscall?

Answer: Because the kernel text is mapped without the `PTE_U` bit set, the CPU will page fault when it tries to read an instruction from a function in the kernel text. (Alternatively, the kernel must execute at a higher privilege level than user code, and a simple function call does not switch privilege levels on the x86.)

10. [8 points]: Ben wants to take his syscall convention to a new level, so he adds *syscall batching*. He modifies his user code to store a *sequence* of system calls in the syscall page and modifies the `syscall` function in his kernel to iterate over this page and invoke each function in the sequence before returning to `trap_dispatch`. Louis Reasoner wants to take this for a spin, so he modifies `user/yield.c` to write its sequence of `sys_yield`, `sys_cputs`, `sys_yield`... syscalls to the syscall page before entering the kernel, as follows

```
for (i = 0; i < 5; i++) {
  batch_add(SYS_yield);
  batch_add(SYS_cputs, "Back in environment\n");
}
sys_execute_batch();
cprintf("All done.\n");
```

Much to his surprise, none of the "Back in environment" messages get printed, even though "All done" does. What happened?

Answer: JOS's `sys_yield` enters the scheduler. When the yielding environment gets scheduled again, JOS will resume execution from the user-space EIP in the environment's saved trap frame, which will be the EIP of the instruction following the `sys_execute_batch` `int 0x30` (roughly, the "All done." `cprintf`). Abstractly, the position in the syscall batch is transient state kept on the kernel stack by the loop in syscall, which is lost when switching environments because JOS only has one kernel stack.

V 6.828

We'd like to hear your opinions about 6.828, so please answer the following questions. (Any answer, except no answer, will receive full credit.)

11. [2 points]: This year we posted (mostly) ahead of lecture draft chapters that provide a commentary on xv6. Did you read them? If so, did you find the chapters useful? What should we do to improve them?

Answer: They were fantastic, except that the line numbers didn't match the code (please don't wait until the quiz feedback to point out bugs like this to us!)

12. [2 points]: Are the labs too time consuming, too short, or are they about right?

Answer: About right.

13. [2 points]: What is the best aspect of 6.828?

Answer: The students.

14. [2 points]: What is the worst aspect of 6.828?

Answer: Debugging little mistakes.

End of Quiz

MIT OpenCourseWare
http://ocw.mit.edu

6.828 Operating System Engineering

Fall 2012

For information about citing these materials or our Terms of Use, visit: http://ocw.mit.edu/terms.



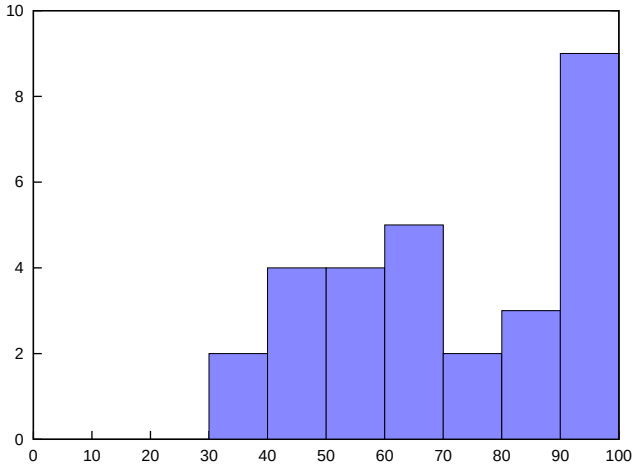
Department of Electrical Engineering and Computer Science

MASSACHUSETTS INSTITUTE OF TECHNOLOGY

6.828 Fall 2011

Quiz I Solutions

Mean 71 Median 69 Std. dev. 20.2



I General questions

1. [8 points]: Unix's API is carefully designed so that programs compose easily. As an example, `write` and `read` don't take an offset as argument, but instead the kernel maintains an offset for each file descriptor. Give a shell command that illustrates how this unusual API simplifies composing programs and explain briefly why.

Answer: `(echo Your files: ; ls) > out`
The `ls` program does not have to do any work to figure out where in file `out` to write; it would if `write()` took an offset argument.

2. [12 points]: In xv6, `wakeup` must scan the entire `ptable` to find the processes that are sleeping on the specified channel. Ben proposes to change xv6 to use condition variables to avoid this scan. His condition variables have the following structure:

```
struct condvar {
  struct proc *waiters;
};
```

For each channel there is a separate condition variable, which holds the list of processes waiting on that condition variable. Wakeup wakes up only those processes:

```
void
cv_wakeup(struct condvar *cv)
{
  acquire(&ptable.lock);
  struct proc *p = cv->waiters;
  while (p) {
    struct proc *next = p->cv_next;
    p->cv_next = 0;
    p->state = RUNNABLE;
    p = next;
  }
  cv->waiters = 0;
  release(&ptable.lock);
}
```

(continued on next page)

Complete the implementation of `cv_sleep`. Your implementation should not call `sleep`.

```
void
cv_sleep(struct condvar *cv, struct spinlock *lk)
{
  if(proc == 0)
    panic("sleep");
  if(lk == 0)
    panic("sleep without lk");
  if (lk != &ptable.lock) {
    acquire(&ptable.lock);
    release(lk);
  }

  proc->cv_next = cv->waiters;
  cv->waiters = proc;
  proc->state = SLEEPING;
  sched();

  if (lk != &ptable.lock) {
    release(&ptable.lock);
    acquire(lk);
  }
}
```

3. [8 points]: `swtch` in xv6 doesn't explicitly save and restore all fields of **`struct`** `context`. Why is it okay that `swtch` doesn't contain any code that saves `%eip`?

Answer: The call to `swtch()` saves `%eip` on the stack, which is constructed in a way such that it follows the layout of **`struct`** `context`. The return from `swtch()` restores `%eip`.

II File systems

4. [10 points]: Albert has a computer running xv6. He has written his own application, which forks many processes, all of which write large files. Every once in a while, Albert's computer panics with "bget: no buffers" (line 3946). He thinks to himself that, since buffers are only used for short periods of time, it would be better for `bget()` to sleep until a buffer is free. So he changes the panic at the very end of `bget()` to:

```
sleep(&bcache, &bcache.lock); // new code
goto loop;                    // new code
```

And adds a wakeup before the release on the last line of `brelse()`:

```
wakeup(&bcache);         // new code
release(&bcache.lock);   // existing code
```

After this change, Albert's computer occasionally stops working: no panic, but no forward progress either. He calls `procdump()` from `gdb` and sees that every process (other than process 1) is waiting in his new sleep in `bget()`; the complete stack for each process is:

```
Function  Line
sleep     2525
bget      the new call to sleep
bread     3956
readsb    4282
balloc    4311
bmap      4629
writei    4769
filewrite 5176
sys_write 5285
syscall   3283
```

Process 1 is sleeping in `wait()`. Albert's computer has only one core. Albert is running a modified version of xv6 with logging disabled: `log_write()` just calls `bwrite()`, and `begin_trans()` and `end_trans()` do nothing.

Explain why the processes never wake up from the sleep in `bget()`. (Hint: look at `bmap`.)

Answer:
`bmap()` needs to use two buffers at the same time if it is adding a new block to a large file, one to hold the file's indirect block and one (in `balloc()`) to hold the superblock while allocating a new block. Suppose there are only 10 slots in the buffer cache, and ten processes (writing ten distinct files) have each read their file's indirect block. At that point there is no free buffer into which any of them can read the superblock, so they will all wait in Albert's new sleep.

5. [10 points]: Albert decides to take a different approach: in order to avoid running out of buffers, he'll allocate them with `kalloc()` as needed. Here's his new `bget()` and `brelse()`:

```
static struct buf*
bget(uint dev, uint sector)
{
  struct buf *b;
  b = (struct buf *) kalloc();
  b->dev = dev;
  b->sector = sector;
  b->flags = B_BUSY;
  b->prev = b->next = b->qnext = 0;
  return b;
}

void
brelse(struct buf *b)
{
  kfree((void*) b);
}
```

Again, Albert's computer has only one core, and has logging disabled.

Albert's computer seems to work for a while, but eventually panics with "freeing free block" from `bfree()` at line 4342.

Explain what is going on.

Answer:
Albert's new scheme lets multiple processes use the same block at the same time (it has nothing equivalent to `B_BUSY`). This allows two processes to allocate the same block or the same inode. If two processes allocate the same block for different files, xv6 will first notice the problem when both are deleted: the second delete will free a freed block.

III JOS memory layout and traps

Currently, the JOS kernel reserves part of every user environment's address space for itself. Ben Bitdiddle (who fights for the user environments) decides to modify his lab 3 implementation so user environments have control over (nearly) their entire 4GB virtual address space.

6. [8 points]: In regular JOS, where the kernel and user code share the same address space, what prevents user code from reading or writing sensitive kernel data?

Answer: The page table entries for the kernel portion of the address space do not have the 'U' bit set, so the paging hardware disallows access to these pages when the CPU is running user code.

Ben separates the user and kernel address spaces. He constructs the kernel page directory, `kern_pgdir`, like usual, but modifies `env_setup_vm` to create a nearly empty page directory for each new environment instead of mapping the kernel in to each environment.

Ben's plan is to switch to `kern_pgdir` whenever there's a trap. Unfortunately, the x86 provides no convenient way to switch address spaces when taking a trap. Ben solves this with a *trap trampoline*. He reserves a few pages for kernel use at the top of each environment's address space. He puts all of his `trapentry.S` code (`TRAPHANDLER` declarations and `alltraps`) on one of these pages so that the CPU can jump to the trap handler entry point without switching address spaces.

7. [8 points]: Ben also maps the kernel stack in to each user environment. Why is this necessary?

Answer: The CPU needs somewhere to push the trap-time state (EIP, CS, etc) when it takes the interrupt. This happens before Ben has a chance to switch to the kernel address space, hence the kernel stack needs to be mapped in to each user environment.

Ben modifies `alltraps` (which runs from the trampoline page) to switch to the kernel page directory before calling the `trap` (up to the `lcr3`, this should be equivalent to the `alltraps` you wrote):

```
_alltraps:
  pushl %ds               # Build trap frame
  pushl %es
  pushal
  movl $GD_KD, %eax       # Load kernel segments
  movw %ax, %ds
  movw %ax, %es
  pushl %esp
  lcr3 PADDR(kern_pgdir)  # NEW: Switch to kernel address space
  mov $trap, %eax         # Call trap
  call *%eax
```

8. [10 points]: Louis Reasoner thinks doing `lcr3 PADDR(kern_pgdir)` from `alltraps` won't work because, in Ben's design, `kern_pgdir` isn't accessible in the user environment's address space. In fact, it *will* load the kernel page directory. Why?

Answer: `lcr3` takes a physical address so it doesn't matter what the current address space is.

9. [8 points]: When Ben runs his code, he finds that it executes the `lcr3`, but fails to execute the next instruction (it never sets `%eax` to the address of `trap`). Why and how can he fix this?

Answer: Ben didn't map the trampoline page in the kernel address space, so as soon as he switches to `kern_pgdir`, the instruction at the EIP will no longer be the `mov` at the end of `alltraps`. Ben can fix this by mapping the trampoline pages at the same location in `kern_pgdir`.

10. [10 points]: Ben fixes this problem and then discovers that he needs to rewrite `sys_cputs` because it can no longer read its string argument directly from the current address space. Describe how Ben can fix `sys_cputs`.

Answer: For each page that the argument string spans, `sys_cputs` needs to translate that page to a physical address using the user environment's page directory, and then read from the page that maps that physical page in the kernel's address space.

IV 6.828

We'd like to hear your opinions about 6.828, so please answer the following questions. (Any answer, except no answer, will receive full credit.)

11. [2 points]: This year we posted the complete draft of the xv6 commentary at the beginning of the semester. Did you find the chapters useful? What should we do to improve them?

Answer: Yes, they were very useful. They could use more figures.

12. [2 points]: This year we introduced code reviews. Did you find them useful? What should we do to improve them?

Answer: Yes. Reading other people's code was as helpful as getting reviews.

13. [2 points]: What is the best aspect of 6.828?

Answer: Labs.

14. [2 points]: What is the worst aspect of 6.828?

Answer: Bugs. Too much code is given.

End of Quiz

MIT OpenCourseWare
http://ocw.mit.edu

6.828 Operating System Engineering

Fall 2012

For information about citing these materials or our Terms of Use, visit: http://ocw.mit.edu/terms.



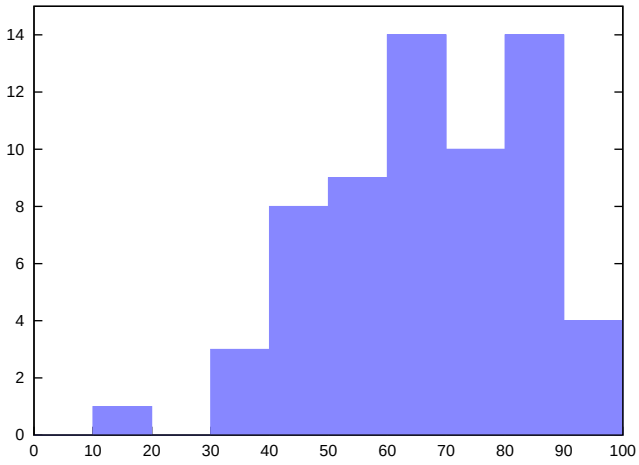
Department of Electrical Engineering and Computer Science

MASSACHUSETTS INSTITUTE OF TECHNOLOGY

6.828 Fall 2012

Quiz I Solutions

Mean 65 Median 66 Std. dev 17.5214



I Shell

The xv6 shell creates new processes to run a command as follows:

```
if(fork1() == 0)
  runcmd(parsecmd(buf));
wait();
```

1. [5 points]: It is possible that the child process that runs the command exits before the parent calls `wait`. Give a scenario in which that can happen.

Answer: Yes. After the parent forked the child, the kernel may decide to run the child first. It may execute the command and exit before the kernel runs the parent again.

2. [6 points]: How does xv6 ensure that `wait` returns correctly even if the child already has exited before the parent called `wait`?

Answer: The kernel keeps track of processes that have exited but for which the parent hasn't called `wait`. Such processes are in the zombie state.

II Locks

In the in-class exercises on locking you modified a hash table to work correctly under concurrent lookup and insert by adding `acquire` and `release` statements for a global lock. The code we handed out for `put` is as follows:

```
static void
insert(int key, int value, struct entry **p, struct entry *n)
{
  struct entry *e = malloc(sizeof(struct entry));
  e->key = key;
  e->value = value;
  e->next = n;
  *p = e;
}

static
void put(int key, int value)
{
  struct entry *n, **p;
  for (p = &table[key%NBUCKET], n = table[key % NBUCKET];
                       n != 0; p = &n->next, n = n->next) {
    if (n->key > key) {
      insert(key, value, p, n);
      goto done;
    }
  }
  insert(key, value, p, n);
 done:
  return;
}
```

3. [4 points]: In your solutions for the in-class exercise, you added in the above code `acquire` and `release` calls to ensure correct operation of concurrent `put` invocations. Indicate where in the above code they should be added.

Answer: Put an acquire for a lock before the for loop and put the corresponding release after done but before the return statement.

To avoid the overhead of locks, Alyssa suggests use of atomic operations, which are guaranteed to perform their multiple step operation as a single atomic operation. In particular, Alyssa suggests use of the following atomic operation:

```
bool __sync_bool_compare_and_swap (type *ptr, type oldval, type newval)
```

This builtin function performs an atomic compare and swap. If the current value of `*ptr` is `oldval`, it writes `newval` into `*ptr`, and returns `true`. Otherwise it returns `false`. The compiler turns the builtin function into the atomic instruction sequence that's appropriate for the target architecture; on the x86, it might use `lock cmpxchg`.

An example:

```
char *p;
  ...
  if(__sync_bool_compare_and_swap(&p, 0, 1))
    ....  // success: p was set to 1
  else
    ...   // failure: p was not set to 1, because p wasn't equal to 0
```

This code will set `p` to `1` only if `p` previously held `0`. If two cores execute the code at the same time, and `p` held `0`, only one of the two cores will see a `true` return value from the call.

4. [8 points]: Complete the `put` below using `__sync_bool_compare_and_swap` so that concurrent invocations will run correctly without using locks:

```
static
void put(int key, int value)
{
  struct entry *n, **p;
  struct entry *e = malloc(sizeof(struct entry));
  e->key = key;
  e->value = value;




  for (p = &table[key%NBUCKET], n = table[key % NBUCKET]; n != 0;
                                        p = &n->next, n = n->next) {



    if (n->key > key) {









    }
  }







 done:
  return;
}
```

Answer: Replace the above code as follows:

```
again:
  for(p = &table...){
    if(n->key > key){
      e->next = n;
      if(__sync_bool_compare_and_swap(p, n, e) == 0)
        goto again;
      else
        goto done;
    }
  }
  e->next = 0;
  if(__sync_bool_compare_and_swap(p, 0, e) == 0)
    goto again;
```

5. [8 points]: For the lock-free implementation, give an example of a workload for which `put`'s throughput scales linearly with an increasing number of cores.

Answer: A workload in which puts use keys that hash to different buckets. These puts can run concurrently. In the lock-based implementation they will be serialized.

III Logical Logging

Alyssa is unhappy that xv6 logging writes an entire sector to the log even if the file system only needs to modify a few bytes in the sector. She observes that log records would be much shorter if they described the operation to be performed, rather than containing the entire modified sector. Alyssa modifies the xv6 logging system to use this idea, called "logical logging." For example, when her `balloc()` allocates a data block, it logs the setting of the bit in the block allocation bitmap with a "set bit" record. A "set bit" record contains a constant indicating the record type, and the block number that was allocated. Alyssa's recovery code for this type of log record sets the relevant bit in the block allocation bitmap on disk. The "set bit" record takes up only a few bytes in the log, rather than an entire sector.

While the "set bit" record works, it turns out that correct recovery from logical log records is not possible for some operations.

6. [8 points]: Alyssa defines a "create file" log record containing the parent directory i-number and the new file's name. Recovery for this kind of record calls a version of create() (sysfile.c) that does everything but the initial nameiparent(), and that uses bwrite for all modifications. Explain why use of this "create file" log record will incorrectly handle some failure scenarios that the current xv6 handles correctly.

Answer: One problem: if the system crashes during recovery for a "create file" log record, an i-node may be left allocated but not used. When the system re-starts and recovery is run again, recovery will allocate a different i-node, and the first one will never be freed.

7. [8 points]: Consider a "delete directory entry" log record that contains the parent i-number and the entry's name. Recovery finds the named entry in the parent directory and overwrites it with nulls. Explain if this works; if not, briefly explain why not.

Answer: This works. If there's a crash during recovery, it happens either before or after the disk write of the zeroed directory entry. If before, the re-run of recovery after system re-start will zero out the directory entry again, which is OK. If after, the entry will be missing, and recovery will not change the directory entry, which is also OK. This argument depends on directory entries not spanning sector boundaries, which is the case in xv6.

IV Page Tables

Consider the following 32-bit x86 page table setup.

`%cr3` holds 0x00001000.

The Page Directory Page at physical address 0x00001000:

```
PDE 0: PPN=0x00002, PTE_P, PTE_U, PTE_W
PDE 1: PPN=0x00003, PTE_P, PTE_U, PTE_W
PDE 2: PPN=0x00002, PTE_P, PTE_U, PTE_W
... all other PDEs are zero
```

The Page Table Page at physical address 0x00002000 (which is PPN 0x00002):

```
PTE 0: PPN=0x00005, PTE_P, PTE_U, PTE_W
PTE 1: PPN=0x00006, PTE_P, PTE_U, PTE_W
... all other PTEs are zero
```

The Page Table Page at physical address 0x00003000:

```
PTE 0: PPN=0x00005, PTE_P, PTE_U, PTE_W
PTE 1: PPN=0x00005, PTE_P, PTE_U, PTE_W
... all other PTEs are zero
```

8. [8 points]: List all virtual addresses that map to physical address 0x00005555.

Answer: 0x00000555, 0x00400555, 0x00401555, 0x00800555

9. [8 points]: Given the above page table, what virtual address (if any) should the kernel store to in order to modify the mapping for virtual address 0x402000 (that's 4*1024*1024 + 8192)?

Answer: There is no address to which the kernel can store in order to modify the page table, because the PPNs of the page directory and page table pages are not mentioned in any PTE. Loads and stores can only refer to virtual addresses, not physical addresses, so with this page table's mappings there's no way to refer to the memory storing the page table.

10. [8 points]: Explain exactly what the above page table should contain in order to to allow the kernel to write to the page directory page. There is more than one answer; you only have to explain one.

Answer: One possibility: PTE number 2 of the page table page at 0x00002000 should contain `PPN=0x00001, PTE_P, PTE_W`

V JOS memory mapping and fork

Here is a code snippet from Ben Bitdiddle's lab 4. Ben is struggling to get user-mode `fork` to work. In his `fork`, Ben invokes `duppage` for every virtual page number (`vpn`) below `UTOP` that is present in the current environment's address space, except for the exception stack.

```
static int duppage(envid_t envid, unsigned vpn) {
    void *addr = (void *)(vpn << PGSHIFT);
    pte_t pte = uvpt[vpn];
    int r;
    // if the page is just read-only, just map it in.
    if (!(pte & (PTE_W|PTE_COW))) {
        if ((r = sys_page_map(0, addr, envid, addr, pte & PTE_SYSCALL)) < 0)
            panic("sys_page_map in child failed: %e", r);
        return 0;
    }
    if ((r = sys_page_map(0, addr, 0, addr, PTE_P|PTE_U|PTE_COW)) < 0)
        panic("sys_page_map in parent failed: %e", r);
    if ((r = sys_page_map(0, addr, envid, addr, PTE_P|PTE_U|PTE_COW)) < 0)
        panic("sys_page_map in child failed: %e", r);
    return r;
}
```

11. [8 points]: Ben Bitdiddle could not pass the `forktree` test because the above code has a bug. While debugging, he printed out the value of `uvpt[vpn]` at the end of the function, and found that the parent environment's user stack was not marked copy-on-write. Explain why this is happening, and describe how to fix the bug. You should assume that his other code is free of bugs.

Answer: After marking its own stack COW with the first `sys_page_map` call, the environment page faults on the stack when it tries to push arguments for the next `sys_page_map`. The page fault handler for fork allocates a new page, copies the data over, and marks it not-COW. The same physical page is then installed (COW) in the child environment by the second `sys_page_map`, but because it is not COW in the parent, the child sees all updates to the parent's stack until it tries to write to its own stack and switches to a copy. This puts the stack in an invalid state by the time the child environment is run. To fix the bug, Ben should map a page COW in the child before in the parent.

VI Exokernel `exec`

Ben Bitdiddle has chosen to implement UNIX-style `exec` as his challenge exercise for lab 5. `exec` replaces the current environment's memory with the desired executable, sets up a stack, and jumps to the entry point of the executable.

Alyssa P. Hacker proposes the following design for `exec` in the user-space library operating system:

1. Unmap all pages in the current environment except for the stack and the code for `exec`.
2. Map in the pages (loaded from the ELF file on disk) for the new environment.
3. Unmap the code page for `exec`.
4. Jump to the new environment's entry point.

12. [8 points]: Ben Bitdiddle sees a problem with Alyssa's first and second steps. What might go wrong? Assume the code for `exec` fits in a single page and doesn't make any function calls (other than system calls).

Answer: If the ELF file has a section whose mapping overlaps the code for `exec`, this will not work. A workaround is to scan the ELF file for conflicts and remap the `exec` code to a new location if necessary.

13. [8 points]: There are a few other problems with Alyssa's plan. Please describe a new plan for a working `exec` in user space. Your solution shouldn't modify the kernel.

Answer: One issue is that after unmapping the code for `exec`, the environment will pagefault. A solution involves setting up and jumping to a small piece of trampoline code on the stack to issue the final unmapping syscall, perform final initialization, and jump to the entry point.

VII 6.828

We'd like to hear your opinions about 6.828, so please answer the following questions. (Any answer, except no answer, will receive full credit.)

14. [2 points]: This year we posted an improved draft of the xv6 commentary at the beginning of the semester. Did you find the chapters useful? What should we do to improve them?

Answer: Yes, they were very useful. They could use more figures.

15. [2 points]: This year we continued code reviews and added rebuttals. Did you find them useful? What should we do to improve them?

Answer: Yes. Reading other people's code was as helpful as getting reviews.

16. [1 points]: Did you find the in-class exercises useful? How would you suggest improving them?

End of Quiz

MIT OpenCourseWare
http://ocw.mit.edu

6.828 Operating System Engineering

Fall 2012

For information about citing these materials or our Terms of Use, visit: http://ocw.mit.edu/terms.



Department of Electrical Engineering and Computer Science

MASSACHUSETTS INSTITUTE OF TECHNOLOGY

6.828 Operating System Engineering: Fall 2003

Quiz II Solutions

All problems are open-ended questions. In order to receive credit you must answer the question as precisely as possible. You have 80 minutes to answer this quiz.

Write your name on this cover sheet AND at the bottom of each page of this booklet.

Some questions may be much harder than others. Read them all through first and attack them in the order that allows you to make the most progress. If you find a question ambiguous, be sure to write down any assumptions you make. Be neat. If we can't understand your answer, we can't give you credit!

THIS IS AN OPEN BOOK, OPEN NOTES QUIZ.

1 (xx/15)	2 (xx/25)	3 (xx/20)	4 (xx/25)	5 (xx/15)	Total (xx/100)

Name:

I Control-C and the shell

1. [15 points]: In UNIX when a user is running a program from the shell, the user can terminate the program by typing ctrl-C. How would you change the 6.828 kernel and its shell to support this feature? Be careful, make sure when a user types the name of the program and hits ctrl-C before the program runs that the right thing happens (i.e., don't kill the shell itself or the file system). (Sketch an implementation and define "what the right thing" is.)

There are many possible solutions; here are three:

- Add a system call to the kernel that allows the shell to indicate at any given point what environment, if any, should be killed when the user types ctrl-C. Modify the console code in the kernel appropriately so that the kernel intercepts ctrl-C (instead of passing it on as a control character to whatever environment is reading the console), and kills the indicated environment. With this solution, environments can never prevent themselves from getting killed: thus, CTRL-C acts like a "hard" or "uncatchable" Unix signal.
- Modify the kernel's console code so that the shell can intercept ctrl-C input characters even when other, "normal" input characters may be going to some other (e.g., child) environment that happens to be reading from the console. Then upon receiving this special control character from the kernel in the console input stream, the shell can decide which of its child environments should be killed. With this solution, environments once again cannot prevent themselves from getting killed, but only the *immediate* child environments of the shell can be killed this way: a (killable) child environment can easily create another, "unkillable" environment simply by forking or spawning a child of its own (a grandchild of the shell).
- Add some form of asynchronous event handling support to the kernel's console code so that environments can asynchronously accept and handle ctrl-C input characters themselves. For example, each environment might be able to register a user-mode "ctrl-C handler," along the lines of the current user-mode "page fault handler." Whenever the user hits ctrl-C, the kernel scans through the list of environments and, for each environment that has registered a ctrl-C handler (and perhaps has set a flag indicating that ctrl-C "signals" should be accepted), the kernel sets up the environment's registers and user-mode exception stack to cause the environment's registered ctrl-C handler to be invoked. Each environment can then perform whatever ctrl-C handling it deems appropriate, such as by killing itself immediately, by killing itself after "gracefully" shutting down (such as after closing files and `sync`ing outstanding data to disk), or even by ignoring the ctrl-C signal entirely. With this solution, CTRL-C is effectively a "soft" signal, which is not guaranteed to work on environments that may be either accidentally or maliciously ignoring it. (This is like the Unix behavior for ctrl-C, since Unix processes can intercept the `SIGINT` signal.)

II CPU scheduling

2. [10 points]: `primespipe` spawns many environments until it has generated a prime greater than some specified number. If you modify your shell for the 6.828 kernel to run `primespipe` in the background and then type `ls` at your shell, it can happen that the output of `ls` won't show before `primespipe` completes. Explain why.

(Keep it brief)

The kernel's scheduler uses a round-robin scheduling policy, always choosing the next runnable environment *in environment ID-space* whenever the current environment yields or runs out of its timeslice. Since newly created environments are assigned environment IDs in ascending sequence, `primespipe` creates a pathological unfairness situation. The "root" `primespipe` environment creates the first child environment, fills the pipe to that child with numbers, then yields the processor in order to wait for more space in the pipe. Since the first child environment typically has the environment ID immediately following the parent's, the child is the next to run. The child in turn creates its own grandchild environment, then reads numbers from its parent's pipe buffer and stuffs the ones that "pass" into the grandchild's pipe buffer. Upon running out of numbers, the child yields, causing the grandchild to run next, and so on. Even if the shell managed to start the `ls` environment before the `primespipe` cascade takes over, `ls` requires a few back-and-forth IPCs with the file system, and so it is very unlikely to get much of anything done before the entire `primespipe` cascade has completed. The key point is that this unfairness is created by the interaction of the *round-robin scheduling policy* with the kernel's "first-free" environment ID assignment.

3. [5 points]: Describe a solution to the problem described in the previous question.

(Keep it brief; no pseudocode required)

- A quick-and-dirty solution that would make this unfairness problem go away but would probably leave many others: Simply make the round-robin scheduling policy look for runnable environments in *descending* rather than ascending order of environment ID. Then `ls` and the file server will each typically get one chance to run for each new `primespipe` instance.
- A more general solution: maintain an explicit *queue of runnable environments* from which the scheduler picks the next environment to run instead of just scanning by environment ID. As long as newly-runnable (including newly-created) environments are always added to the *tail* of the queue and the scheduler always picks the environment at the *head* as the next environment to run, environment creation cannot be used as a way to hog the CPU.
- Implement a Unix-style interactive scheduling policy in which processes that have not run for a while or have not used much CPU time receive an elevated priority.
- Implement a hierarchical or group-based scheduling policy where an environment's children receive a share of the CPU time originally allotted to its parent, so that the entire group of `primespipe` environments collectively get only as much CPU time as the `ls` or file server environments do individually.

Name:

Let's assume we modify the 6.828 kernel to build a high-performance Web server. We add a driver for ethernet card in in the Web server environment. Interrupts from the card are directly delivered to the Web server's environment, and the interrupt handler executes on a separate stack in the Web server's environment (in a similar style as the 6.828 kernel sends page faults to environments.) The Web server's interrupt handler runs with interrupts disabled, processes the received interrupt completely (including any TCP/IP and HTTP processing), returns all resources associated with handling the interrupt, and re-enables interrupts. Web pages in the cache are served straight from the interrupt handler. CGI scripts and pages not in the cache are handled by the Web server on the main stack (not in the interrupt handler).

4. [10 points]: Does this implementation suffer from receive livelock? If so, sketch a sequence of events that will result in receive livelock. If not, explain why. (If you need to make any more assumptions about the architecture of the system, be sure to state them explicitly.)

Yes, the implementation does suffer from receive livelock. Suppose HTTP requests arrive at a sufficient rate that by the time the web server's interrupt handler is finished with the interrupt processing for one request, the Ethernet card has already received another request, causing the interrupt to be triggered again immediately as soon as the web server re-enables interrupts. Since cached pages are served straight from the interrupt handler, the web server will be able to continue serving them. Pages that miss in the cache and dynamic CGI pages must be handled by the web server *outside* of the interrupt handler, however. As long as the web server keeps getting a continuous load of interrupts it will *never* get out of its network interrupt handler and get around to processing any of these cache misses or CGI requests. Thus, the web server is livelocked, making no forward progress at all on many of the requests it is supposed to be serving.

III File systems and reliability

Modern Unixes support the `rename(char *from, char *to)` system call, which causes the link named "from" to be renamed as "to". Unix v6 does not have a system call for rename; instead, rename is an application that makes use of the `link` and `unlink` system calls.

5. [5 points]: Give an implementation of rename using the `link` and `unlink` system calls, and briefly justify your implementation.

```
int rename(char *from, char *to) {
     /* 1 */ unlink(to);
     /* 2 */ link(from, to);
     /* 3 */ unlink(from);
}
```

Some editors use `rename` to make a new version of a file visible to user atomically. For example, an editor may copy "x.c" to "#x.c", make all changes to "#x.c", and when the user hits save, the editor calls `sync()` followed by `rename("#x.c", "x.c")`.

6. [5 points]: What are the possible outcomes of running `rename("#x.c", "x.c")` if the computer fails during the rename library call? Assume that both "#x.c" and "x.c" exist in the same directory but in different directory blocks before the call to `rename`.

Because the effects of the calls in our rename implementation are not persistent unless the corresponding block writes go out to disk, it is *not* correct simply to consider the failure happening before or after each call. Instead, we need to think about the possible block writes that can happen before the failure.

The rename implementation above causes three block writes, all possibly delayed (written with `bdwrite`). First, the directory entry for "to" is removed from its block. Second, the directory entry for "to" is added back to its block, but with the i-number for "from". Third, the directory entry for "from" is removed from its block.

The possible outcomes are:

- Nothing done: no changes written to disk yet.
- The first write went to disk: "from" remains as before, but "to" doesn't exist at all. (Looks like rename failed just after line 1 above.)
- The first and second writes went to disk: "from" remains as before, but now "to" points at the same file as "from". (Looks like rename failed just after line 2 above.)
- The first, second, and third writes went to disk: "from" is gone, and "to" points at the new file. (Looks like rename succeeded.)
- The first and third writes went to disk, *but not the second*: both "from" and "to" are gone! (Cannot be explained by examination of the rename implementation!)

Name:

Modern BSD UNIX implements `rename` as system call. The pseudocode for `rename` is as follows (assuming "to" and "from" are in different directory blocks):

```
int rename (char *from, char *to) {
   update dir block for ''to'' to point to ''from'''s inode // write block
   update dir block for ''from'' to free entry // write block
}
```

BSD's Fast File System (FFS) performs the two writes in `rename` synchronously (i.e., using `bwrite`).

7. [5 points]: What are the possible outcomes of running `rename("#x.c", "x.c")` if the computer fails during the rename system call? Assume that both "#x.c" and "x.c" exist in the same directory but in different directory blocks before the call to `rename`.

- Nothing done: state remains as-is before the `rename`.
- Only first ("to") update done: "from" and "to" both refer to the same file.
- Everything done: `rename` completed successfully.

Because of the atomic change of the "to" entry in this implementation, we never get into a state where "to" does not point to anything. Because the first write must go out before the second, we cannot lose the new copy of the file. Many common Unix applications today depend on this level of `rename` atomicity in the file system in order to ensure reasonable robustness against failures.

Assume the same scenario but now using FFS with soft updates.

8. [5 points]: How does using FFS with soft updates change this scenario?

In terms of robustness, soft updates do not change the file system's semantics at all: exactly the same situations are possible as above for FFS with synchronous writes, because the writes must go out in the same order. The difference is in *performance*: with soft updates, the file system does not have to perform all meta-data writes synchronously, and therefore can achieve much greater performance benefits from caching without compromising robustness.

Just to be precise, it's worthy of note that soft updates *can* potentially change the file system's failure semantics in ways that are theoretically *observable* to applications; just not in ways that create *file system metadata inconsistencies*. For example, suppose an application creates a file A somewhere in the file system, then creates a file B *somewhere else* in the file system that happens to involve completely unrelated directory and inode blocks. If a failure occurs during or shortly after these files are created, with soft updates the application could (on system restart) observe that file B was successfully created but A wasn't, because the soft updates algorithm imposes no ordering dependency between them. With synchronous writes, however the application would never observe that B as having been successfully created unless A had also been successfully created.

Name:

IV Virtual machines

A virtual machine monitor can run a guest OS without requiring changes to the guest OS. To do so, virtual machine monitors must virtualize the instruction set of a processor. Consider implementing a monitor for the x86 architecture that runs directly on the physical hardware. This monitor must virtualize the x86 processor. Unfortunately, virtualizing the x86 instruction set is a challenge.

9. [5 points]: Using the instruction "mov %cs, %ax", give a code fragment that shows how a guest operating system could tell the difference between whether it is running in a virtual machine or directly on the processor, if the virtual machine monitor is not sufficiently careful about how it virtualizes the x86 architecture.

(List a sequence of x86 assembly instructions)

```
movw    %cs, %ax
andw    $3, %ax
jnz     running_on_vm
```

The bottom two bits of the CS register on the x86 represent the processor's current privilege level, 0 meaning kernel mode and 3 meaning user mode. The guest operating system can therefore tell whether it is running in user or kernel mode simply by testing those two bits of CS. Since the "mov %cs, %ax" instruction is *not* privileged, the virtual machine monitor has no easy way to trap that instruction and emulate the *correct* behavior—i.e., to return a CS register value with 0 in the low two bits, indicating that the guest OS is running in "virtualized" kernel mode, even though the guest OS code is *actually* running in user mode as far as the physical hardware is concerned.

10. [5 points]: Describe a solution for how to virtualize the instruction "mov %cs, %ax" correctly. What changes do you need to make to the monitor?

(Give a brief description; no pseudocode required)

The virtual machine monitor first scans through all code in the guest OS before allowing it to run, replacing each non-virtualizable instruction such as the one above with a one-byte "`int $3`" instruction, and remembering the original instruction replaced at each point in the code using a table maintained elsewhere by the monitor. When the guest OS hits one of these "`int $3`" instructions, the virtual machine monitor intercepts the interrupt, scans its table of replaced instructions for that EIP address, emulates the correct behavior for the replaced instruction, and increments EIP past the space reserved for it.

Name:

The monitor might emulate a load of CR3 as follows:

```
// addr is a physical address
void
emulate_lcr3(thiscpu, addr)
{
  Pte *fakepdir;

  thiscpu->cr3 = addr;
  fakepdir = lookup(addr, oldcr3cache);
  if (!fakepdir) {
    fakedir = page_alloc();
    store(oldcr3cache, addr, fakedir);
    // CODE MISSING:
    // May wish to scan through supplied page directory to see if
    // we have to fix up anything in particular.
    // Exact settings will depend on how we want to handle
    // problem cases.
  }
  asm("movl fakepdir,%cr3");
  // Must make sure our page fault handler is in sync with what we do here.
}
```

11. [15 points]: Describe a solution for handling a guest OS that executes instructions stored in its data segment (e.g., `user_icode` in the 6.828 kernel).

The virtual machine monitor must use some protection mechanism to ensure that:

- Any attempt by the guest OS to "jump into" and run newly loaded or modified code will trap into the monitor, giving the monitor a chance to scan the code and replace non-virtualizable instructions as described for question 10.
- *After* a page of physical memory containing code has been scanned and modified, any attempts by the guest OS to *read or write* that page of memory (treating it once again as "data" instead of jumping into it as "code") will once again trap into the monitor, allowing the monitor to undo its modifications or simulate the correct read or write to the code page as if the page contained the guest OS's original, unmodified code.

There are different ways of achieving the above protection characteristics on the x86, but none of them are simple due to such practical complications as the lack of support for "execute-only" page mappings in the x86 architecture. Clever use of segment registers, privilege levels, and/or more intelligent guest OS code analysis and rewriting can help.

V Feedback

Since 6.828 is a new subject, we would appreciate receiving some feedback on how we are doing so that we can make corrections next year. (Any answer, except no answer, will receive full credit!)

12. [1 points]: Did you learn anything in 6.828, on a scale of 0 (nothing) to 10 (more than any other class)?

13. [1 points]: What was the best aspect of 6.828?

14. [1 points]: What was the worst aspect of 6.828?

15. [2 points]: If there is one thing that you would like to see changed in 6.828, what would it be?

16. [2 points]: How should the lab be changed?

17. [1 points]: How useful was the v6 case study, 0 (bad) to 10 (good)?

18. [1 points]: How useful were the papers, 0 (bad) to 10 (good)?

19. [2 points]: Which paper(s) should we definitely delete?

20. [2 points]: Which paper(s) should we definitely keep?

Name:

21. [1 points]: Rank TAs on a scale of 0 (bad) to 10 (good)?

22. [1 points]: Rank the professor on a scale of 0 (bad) to 10 (good)?

End of Quiz II

Name:

MIT OpenCourseWare
http://ocw.mit.edu

6.828 Operating System Engineering
Fall 2012

For information about citing these materials or our Terms of Use, visit: http://ocw.mit.edu/terms.



Department of Electrical Engineering and Computer Science

MASSACHUSETTS INSTITUTE OF TECHNOLOGY

6.828 Operating System Engineering: Fall 2004

Quiz 2 Solutions

(Thanks to Eddie Kohler and David Mazières for contributing many of the questions on this quiz.)

All problems are open-ended questions. In order to receive credit you must answer the question as precisely as possible. You have 80 minutes to answer this quiz.

Write your name on this cover sheet AND at the bottom of each page of this booklet.

Some questions may be much harder than others. Read them all through first and attack them in the order that allows you to make the most progress. If you find a question ambiguous, be sure to write down any assumptions you make. Be neat. If we can't understand your answer, we can't give you credit!

THIS IS AN OPEN BOOK, OPEN NOTES QUIZ.

1-4 (xx/35)	5-6 (xx/15)	7-8 (xx/20)	9-11 (xx/15)	12 (xx/10)	13-14 (xx/5)	Total (xx/100)

Name:

I Labs

Ursula Unsafe is working on Lab 4, System Calls for Environment Creation. Here is her first version of the sys_mem_map system call:

```
//
// Map the page of memory at 'srcva' in srcenvid's address space
// at 'dstva' in dstenvid's address space with permission 'perm'.
// Perm has the same restrictions as in sys_mem_alloc, except
// that it also must not grant write access to a read-only
// page.
//
// Return 0 on success, < 0 on error.  Errors are:
// -E_BAD_ENV if srcenvid and/or dstenvid doesn't currently exist,
// or the caller doesn't have permission to change one of them.
// -E_INVAL if srcva >= UTOP or srcva is not page-aligned,
// or dstva >= UTOP or dstva is not page-aligned.
// -E_INVAL is srcva is not mapped in srcenvid's address space.
// -E_INVAL if perm is inappropriate (see sys_page_alloc).
// -E_INVAL if (perm & PTE_W), but srcva is read-only in srcenvid's
// address space.
// -E_NO_MEM if there's no memory to allocate the new page,
// or to allocate any necessary page tables.
static int
sys_mem_map(uint_t srcenvid, u_int srcva,
            uint_t dstenvid, u_int dstva,
            u_int perm)
{
        struct Env* srcenv, *dstenv;
        struct Page* pg;
        Pte* pte;
        int retval;

        if ((retval = envid2env(srcenvid, &srcenv, 1)) < 0
            || (retval = envid2env(dstenvid, &dstenv, 1)) < 0)
                return retval;
        if ((srcva & (BY2PG - 1)) || (dstva & (BY2PG - 1)))
                return -E_INVAL;
        if (!(pg = page_lookup(srcenv->env_pgdir, srcva, &pte)))
                return -E_INVAL;
        return page_insert(dstenv->env_pgdir, pg, dstva, perm);
}
```

Name:

1. [10 points]: Describe two different ways that Ursula's `sys_mem_map` would let a user environment inject arbitrary code into the kernel. In particular, give two code fragments, including calls to `sys_mem_map`, after which the statement "`strcpy((char*) UTEMP, "Ha ha!");`" would write the string "`Ha ha!`" at kernel virtual address 0xF0003000.

Since the above implementation of `sys_mem_map` *does not check either srcva or dstva against* `UTOP`*, some of the possible ways are:*

- *Copy the kernel's mapping for the page at virtual address 0xF0003000 into a writeable, user-accessible mapping at* `UTEMP`*:*
  ```
  sys_mem_map(0, 0xF0003000, 0, UTEMP, PTE_U|PTE_W|PTE_P);
  ```
- *Allocate a new page at* `UTEMP` *and map this page into the kernel at virtual address 0xF0003000:*
  ```
  sys_mem_alloc(0, UTEMP, PTE_U|PTE_W|PTE_P);
  sys_mem_map(0, UTEMP, 0, 0xF0003000, PTE_U|PTE_W|PTE_P);
  ```
- *Use* `sys_mem_map` *to add* `PTE_U` *permission to the KVPD mapping (the current env's page directory), or similarly add* `PTE_W` *permission to the UVPD mapping, allowing the user env free write access to all of its own page tables. The user env can then directly map the kernel's page 0xF0003000 into its own space at* `UTEMP` *with simple memory writes.*

2. [10 points]: Describe one way that Ursula's `sys_mem_map` would let a user environment change arbitrary pages in *other* user environments. In particular, sketch out code steps (possibly using comments instead of actual C), including a call to `sys_mem_map`, after which the statement "`strcpy((char*) UTEMP, "Ha ha!");`" would write the string "`Ha ha!`" at environment `e`'s virtual address 0x80000. (Hint: Consider using `envs[e].env_cr3`.)

A few of the many possible ways:

- *Since the kernel maps all (usable) physical memory at* `KERNBASE`*, the user can use* `sys_mem_map` *to obtain access to its own page directory by copying the kernel's mapping for address* `KERNBASE+envs[e].env_cr3`*. It can then similarly find and map the page table for address 0x80000 in* `e`*, then finally find and map the page for address 0x80000 in* `e` *itself at* `UTEMP` *writeable into its own space.*
- *A quicker but more subtle approach: Just use* `sys_mem_map` *to make the* `envs[]` *table page containing* `e`*'s* `Env` *structure writeable in our address space, then change* `envs[e].env_parent_id` *to our own envid. We can then use* `sys_mem_map` *to transfer arbitrary mappings between our space and* `e`*'s because* `e` *is now our "child" and so the* `env2envid` *parent/child permission check will succeed.*
- *A more practically complex but totally general approach: patch some code into the kernel itself, e.g., replacing a frequently-called kernel routine such as* `env_run`*, with code that does whatever we want to do from within privileged mode. This uploaded "kernel" code could then just* `lcr3()` *e's page directory and write into* `e`*'s address space using ordinary memory writes, for example.*

Name:

Here's a version of the user-level page fault handler you wrote in Lab 4.

```
// Page fault upcall entrypoint.
// This is where we ask the kernel
// to redirect us to whenever we cause a page fault in user space
// (see the call to sys_set_pgfault_handler in pgfault.c).
//
// When a page fault actually occurs,
// the kernel switches our ESP to point to the user exception stack
// if we're not already on the user exception stack,
// and then it pushes the following minimal trap frame
// onto our user exception stack:
//
// [ 5 spare words ]
// trap-time eip
// trap-time eflags
// trap-time esp
// tf_err (error code)
// fault_va <-- %esp
//
// We then have to save additional caller-saved registers
// and call up to the appropriate page fault handler in C code,
// pointed to by the global variable '_pgfault_handler' declared above.

.text
.globl _pgfault_upcall
_pgfault_upcall:
        // Save the caller-saved registers.
        movl %eax, 28(%esp)
        movl %ecx, 24(%esp)
        movl %edx, 20(%esp)

        // Call the C page fault handler.
        movl _pgfault_handler, %eax
        call *%eax

        // Push trap-time %eip and %eflags onto the trap-time stack.
        //
        // Explanation:
        //   We must prepare the trap-time stack for our eventual return to
        //   re-execute the instruction that faulted.
        //   Unfortunately, we can't return directly from this stack
        //   (the exception stack).  Why not?
        //   We can't call 'jmp', since that requires that we load the address
        //   into a register, and all registers must have their trap-time
        //   values after the return.
        //   We can't call 'ret' from the exception stack either, since if we
        //   did, %esp would have the wrong value.
        //   So instead, we push the trap-time %eip onto the *trap-time* stack!
        //   Below we'll switch to that stack and call 'ret', which will
        //   restore %esp to its pre-fault value.
        //   We'll also restore %eflags from the trap-time stack, in case an
        //   intervening instruction changes the flags.  ('ret' does not.)
        //
        //   In the case of a recursive fault on the exception stack,
        //   note that the two words we're pushing now will overlap with
        //   the current exception frame!
        //
        movl 8(%esp), %eax      // trap-time esp in eax
        subl $8, %eax           // add space for eip and eflags
        movl %eax, 8(%esp)

        movl 16(%esp), %ecx     // eip
        movl %ecx, 4(%eax)                      <---- CIRCLE HERE
        movl 12(%esp), %ecx     // eflags
        movl %ecx, 0(%eax)                      <---- CIRCLE HERE
```

Name:

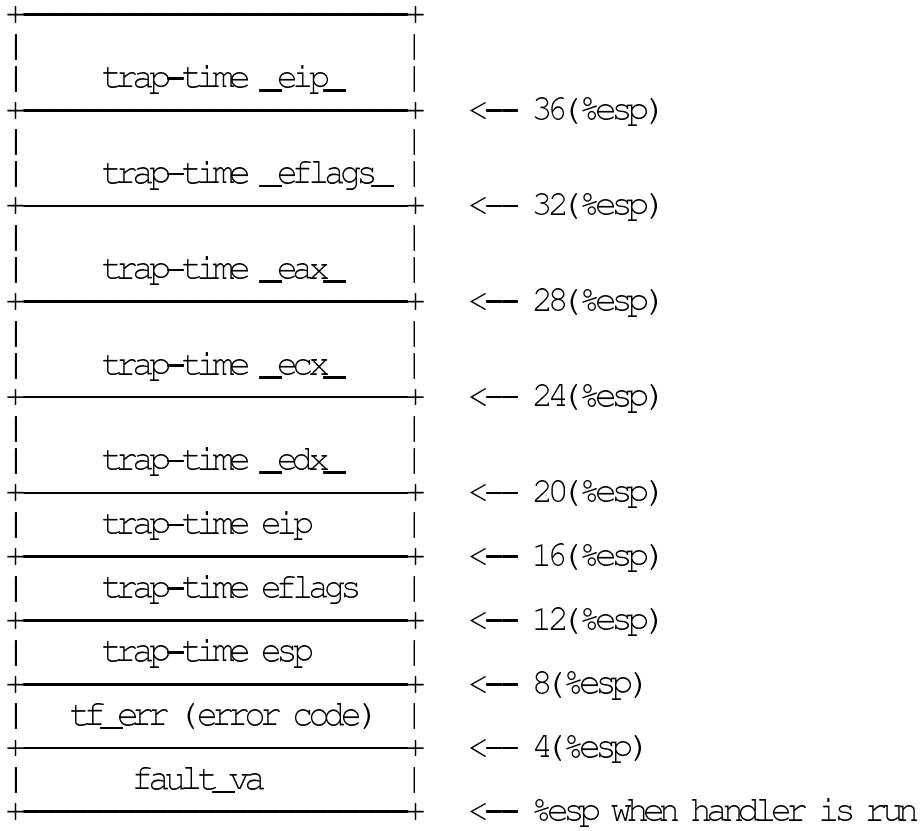
```
// Restore the caller-saved registers.
movl 20(%esp), %edx
movl 24(%esp), %ecx
movl 28(%esp), %eax

// Switch back to the adjusted trap-time stack.
movl 8(%esp), %esp

// Restore eflags from the stack.
popf                                    <---- SQUARE HERE

// Return to re-execute the instruction that faulted.
ret                                     <---- SQUARE HERE
```

3. [5 points]: Fill in the blanks in the following exception stack layout, referring to the code above. The top two words are used only in case of a recursive fault (i.e., a fault in the page fault handler itself); additionally circle the two instructions above that will set these two words, and draw a box around the two instructions that will use them.



Name:

4. [10 points]: Fill in the following function, which implements simple shared memory.

```
// Allocate a page of memory at 'va' that is shared with environment 'e'.
// Do not return until environment 'e' has also called shmget() with
// similar arguments.
// Don't worry about race conditions.
// Assume that 'e' calls sys_ipc_recv only from shmget.
void shmget(void* va, envid_t e, int perm)
{
        assert(!((u_int) va & (BY2PG - 1)));

        // Is 'e' waiting in ipc_recv?
        if (envs[e].env_ipc_recving) {


                // allocate a page to share.
                sys_mem_alloc(0, va, perm);

                // Send a mapping of this page to env e.
                ipc_send(e, va, va, perm);


        } else {
                // Receive a page from 'e' mapped at 'va'.
                u_int va_from_e = ipc_recv(0, va, 0);
                assert(va_from_e == (u_int) va);
        }
}
```

Here are some header comments for reference.

```
// Allocate a page of memory and map it at 'va' with permission
// 'perm' in the address space of 'envid'.
// The page's contents are set to 0.
// If a page is already mapped at 'va', that page is unmapped as a
// side effect.
//
// perm -- PTE_U | PTE_P must be set, PTE_AVAIL | PTE_W may or may not be set,
//         but no other bits may be set.
//
// Return 0 on success, < 0 on error.  ...
int sys_page_alloc(u_int envid, u_int va, int perm);

// Try to send 'value' to the target env 'envid'.
// If va != 0, then also send page currently mapped at va,
// so that receiver gets a duplicate mapping of the same page.
//
// The send fails with a return value of -E_IPC_NOT_RECV if the
// target has not requested IPC with sys_ipc_recv.
//
// Otherwise, the send succeeds, and the target's ipc fields are
// updated as follows:
//    env_ipc_recving is set to 0 to block future sends
//    env_ipc_from is set to the sending envid
//    env_ipc_value is set to the 'value' parameter
// The target environment is marked runnable again.
//
// Return 0 on success, < 0 on error.
```

Name:

```
//
// If the sender sends a page but the receiver isn't asking for one,
// then no page mapping is transferred but no error occurs.
//
// srcva and perm should have the same restrictions as they had
// in sys_mem_map.
//
// Hint: you will find envid2env() useful.
int sys_ipc_can_send(u_int envid, u_int value,
                     u_int srcva, unsigned perm);

// Block until a value is ready.  Record that you want to receive,
// mark yourself not runnable, and then give up the CPU.
//
// Again, dstva should have the same restrictions as it had in
// sys_mem_map.  If it violates these restrictions, assume that it is
// zero.
static int sys_ipc_recv(u_int dstva)
```

Name:

II Brief paper questions

5. [10 points]: shell Answer the following question with respect to "*Es*: A shell with higher-order functions," by Haahr and Rakitzis. What will the following code print in the *es* shell?

```
fn xxx first rest {
    if {~ $#rest 0} {
        return $first
    }
    return <>{ xxx $rest } $first
}

echo <>{ xxx a b c }
```

Answer: `c b a`

6. [5 points]: Synchronization primitives Professor Dumbo wants to demonstrate a variant of the "ticket lock" to his class. Here's what he comes up with:

```
int nwaiting;

void acquire_lock() {
        atomic_inc(&nwaiting);
        while (nwaiting != 1)
                // wait for the other (nwaiting - 1) processes to finish
                pause(nwaiting - 1);
}

void release_lock() {
        atomic_dec(&nwaiting);
}
```

Give a specific scenario demonstrating that this implementaiton of the ticket lock is wrong.

There is more than one way this code can go wrong, but the simplest is probably:

1. `nwaiting` *starts at 0 (lock is not held).*
2. *Process 1 atomically increments* `nwaiting` *to 1.*
3. *Process 2 atomically increments* `nwaiting` *to 2.*
4. *Process 1 tests* `nwaiting`*, and goes into a loop because* `nwaiting = 2`*.*
5. *Process 2 tests* `nwaiting` *and similarly goes into a loop waiting for the lock.*
6. *The two processes are now deadlocked: neither of them can acquire the lock because each is waiting for the other to release it.*

Name:

III Microkernels

The paper "Improving IPC by Kernel Design," by Jochen Liedtke, describes a number of optimizations the author made to achieve good IPC performance in the L3 kernel. One such optimization involved the addition of new system calls. L3 contained two traditional calls for implementing IPC:

- **send –** send a message (asynchronously) to another process
- **receive** – wait for and return a message sent to a process

In addition, Liedtke added two more calls each of which combines the functionality of **send** and **receive**:

- **call**
- **reply & receive next**

7. [10 points]: Which of the following statements is true for the new **call/reply & receive next** interface, compared to equivalent code that issues a **receive** immediately following a **send** system call?

(Circle all that apply; there may be more than one answer.)

A. The new interface reduces the number of user-kernel crossings required for an IPC.

Yes: the total number of system calls for a typical client/server interaction is reduced from four to two.

B. The new interface reduces the number of TLB flushes required during an IPC.

No: Either way, the TLB must be flushed once when switching from client to server, then again on the switch from the server to client. With the old interface, a TLB flush isn't required on `send` *(which doesn't block or transfer control), only on* `receive` *(which does).*

C. The new interface lets L3 reduce the number of scheduler queue manipulations required during an IPC.

Yes: with the old interface, on a `send` *the kernel would have to add the receiving thread to the scheduler queue, and then on the sender's subsequent* `receive` *call the kernel would have to go back and find the receiving thread on the scheduler queue (which it put there on the previous system call), remove it, and transfer control to the receiving thread. With the new interface, the kernel can simply transfer control atomically from the sender to the receiver during the single* **call** *or* **reply & receive next** *system call, and avoid touching the scheduler queue at all.*

Name:

D. When no processes are swapped out to disk, the new interface reduces the number of times the kernel must copy IPC arguments and results that don't fit in registers.

No: Either way the kernel copies messages directly from the sender's address space to the receiver's, resulting in the same total amount of data copying. The receiver window trick makes this direct cross-space data copying possible, but that's a separate optimization not directly related to the combined control transfer interface for IPC.

E. The new interface makes the scheduler less fair, and could even lead to starvation were it not for the "long time wakeup list."

No: The new interface doesn't change the scheduler's normal behavior in any way. The optimization merely avoids the need to frequently put a thread onto the scheduling queue (during a `send`*) only to take it off again immediately (during a subsequent* `receive`*), effectively reducing the total number of scheduling operations that need to be done in the first place.*

To optimize data transfers during IPC, the kernel copies straight from the sender's address space into the receiver's, as follows. The kernel reserves a portion of virtual memory known as the communication window. The kernel also restricts each IPC argument to a maximum of 4 Megabytes. To copy into the recipient's address space, the kernel takes two page directory entries from the recipient's page directory and places them into the sender's at the virtual address of the communication window. (The kernel also clears the `PTE_U` bit, so that user code in the sender cannot access the memory mapped in the communications window.)

One concern is that, on the processor used by L3, it is expensive to flush TLB entries for a large virtual address range. (You can only flush one page translation at a time, or else the whole TLB.) Thus, Liedtke introduces the term *window clean*, to mean the TLB is guaranteed not to be caching any mappings for virtual addresses in the communication window.

8. [10 points]: Describe a scenario in which, when the kernel must copy data between address spaces, the TLB is *not* window clean. (Hint: feel free to use the old or the new system calls in your scenario.)

There are at least two such possible scenarios, and probably others:

- *Suppose that a thread performs several non-blocking* `send`*s in succession (perhaps to several different receivers) without intervening* `receive` *calls. The first* `send` *will use the window trick to copy the message into the receiver's address space, leaving the sender's TLB "dirty" with the temporary page mappings used to perform this copy. However, since* `send` *does not block the sending thread and change address spaces, it does not automatically flush the TLB. Thus the thread will* not *be window clean at the start of the next* `send` *operation, and the kernel will have to flush the TLB explicitly before re-using the window to copy the next message.*
- *Suppose that one thread sends a message and then blocks waiting for a reply. The TLB is now "dirty" because the kernel mapped in memory for the send. If another thread in the same address space is scheduled next, the TLB will not be flushed because the kernel does not need to change address spaces. Now if this second thread calls* `send`*, the kernel will need to manually flush the window to clean the TLB.*

Name:

IV Disco

Answer questions in this section with respect to the paper "Disco: Running Commodity Operating Systems on Scalable Multiprocessors" by Bugnion, Devine, and Rosenblum.

9. [5 points]: On page 11, the authors write, "The execution of the operating system in general and critical section in particular is slower on disco, which increases the contention for semaphores and spinlocks." What is the intuition for why contention increases, and what experiment demonstrates it?

Executing operating system code is much slower under Disco than without because of the virtualization overhead. In particular, the slowdown of OS code due to virtualization is disproportionate to the slowdown of application code because OS code tends to execute many privileged instructions and perform many low-level operations that require special handling by the VMM, which is where the main performance penalty lies. For this reason, when one processor holds a semaphore or spinlock while within the OS kernel, it will tend to take much longer to release it under Disco, so other processors will be more likely to come along during that time and create contention for the same semaphore or spinlock.

10. [5 points]: In Figure 5, why does "kernel time" decrease when running on Disco?

The Disco VMM performs some important tasks that the "stand-alone" IRIX kernel would normally have to do itself. For example, Disco allocates and initializes machine pages, thereby absorbing much of the memory access overhead that the IRIX kernel would normally incur performing these operations.

11. [5 points]: In Figure 6, why does Irix data grow? Why does buffer cache stay the same size for the M bar?

IRIX data grows because the kernel datastructures are now replicated in each instance of the kernel. The buffer cache is shared among all instances and therefore stays the same size.

Name:

V File systems

12. [5 points]: After finishing a Window systems for JOS, Austin Ports is evaluating whether to use soft-updates to improve the performance of the JOS file system. He keeps the layout and the basic file structures the same, but he would like to arrange that the JOS file server writes as many disk blocks asynchronously to the disk as possible, while maintaining file system consistency. Is there a scenario in which the file server must undo file system modifications when writing blocks out to disk to ensure file system consistency? If so, draw a picture with a dependency. (The next page shows the JOS file system structures.)

Yes. For example, suppose we have a file `foo` *in some directory, and* `foo` *has one data block attached to it. We delete* `foo`*, causing its one data block to be returned to the free block bitmap. We then create a new file* `bar` *in the same block of the same directory, then write a few bytes to it, causing the data block previously attached to* `foo` *to be re-allocated from the free block bitmap and attached to the new file* `bar`*. We now have a dependency chain as follows:*

Write directory block to remove `foo`

⇓

Write bitmap block to mark `foo`*'s old data block free*

⇓

Write directory block to create `bar`

⇓

Write bitmap block to mark `bar`*'s new data block in-use*

⇓

Write directory block to attach `bar`*'s new data block to* `bar`

Since the above logical writes alternate between two physical disk blocks, this sequence creates circular dependencies requiring the later logical writes to be rolled back in order to commit the earlier logical writes to disk. For example, the file system could roll back all but the first two writes, commit those two in order, then roll forward the next two writes and commit those in order, and finally roll forward the final write and commit that one.

13. [5 points]: What file system calls would add more dependencies? (Explain briefly why.)

The Unix `rename` *operation, for example, removes a file from one directory block and inserts it in another while ensuring that the file can never be "lost" if the system crashes in the middle of the operation. Ensuring this property creates a dependency between the insertion of the file in the new directory (which must happen first) and the removal of the file from the old directory (which must happen afterwards).*

Name:

```
// File nodes (both in-memory and on-disk)

// Bytes per file system block - same as page size
#define BY2BLK          BY2PG
#define BIT2BLK         (BY2BLK*8)


// Maximum size of a filename (a single path component), including null
#define MAXNAMELEN      128

// Maximum size of a complete pathname, including null
#define MAXPATHLEN      1024

// Number of (direct) block pointers in a File descriptor
#define NDIRECT         10
#define NINDIRECT       (BY2BLK/4)

#define MAXFILESIZE     (NINDIRECT*BY2BLK)

#define BY2FILE         256
struct File
  union
    struct
        uint8_t f_name[MAXNAMELEN];     // filename
        uint32_t f_size;                // file size in bytes
        uint32_t f_type;                // file type
        uint32_t f_direct[NDIRECT];
        uint32_t f_indirect;

        struct File *f_dir;             // valid only in memory
    ;
    uint8_t f_pad[BY2FILE];             // make sizeof(struct File) == BY2FILE
  ;
;

#define FILE2BLK        (BY2BLK/sizeof(struct File))

// File types
#define FTYPE_REG               0       // Regular file
#define FTYPE_DIR               1       // Directory


// File system super-block (both in-memory and on-disk)

#define FS_MAGIC        0x68286097      // Everyone's favorite OS class

struct Super
        uint32_t s_magic;       // Magic number: FS_MAGIC
        uint32_t s_nblocks;     // Total number of blocks on disk
        struct File s_root;     // Root directory node
;
```

Name:

```
/* from fs/fs.h */
int file_open(char *path, struct File **pfile);
int file_get_block(struct File *f, u_int blockno, void **pblk);
int file_set_size(struct File *f, u_int newsize);
void file_close(struct File *f);
int file_remove(char *path);
void fs_init(void);
int file_dirty(struct File *f, u_int offset);
void fs_sync(void);
void file_flush(struct File*);

/* from inc/fd.h */
struct Dev

        int dev_id;
        char *dev_name;
        int (*dev_read)(struct Fd*, void*, u_int, u_int);
        int (*dev_write)(struct Fd*, const void*, u_int, u_int);
        int (*dev_close)(struct Fd*);
        int (*dev_stat)(struct Fd*, struct Stat*);
        int (*dev_seek)(struct Fd*, u_int);
        int (*dev_trunc)(struct Fd*, u_int);
;

struct Fd

        u_int fd_dev_id;
        u_int fd_offset;
        u_int fd_omode;
;

struct Stat

        char st_name[MAXNAMELEN];
        u_int st_size;
        u_int st_isdir;
        struct Dev *st_dev;
;

struct Filefd

        struct Fd f_fd;
        u_int f_fileid;
        struct File f_file;
;
```

Name:

VI 6.828

We love to have your suggestions for improving 6.828. Please, answer the following question. (Any answer, except no answer, will receive full credit!)

14. [2 points]: If you could change one aspect of 6.828, what would it be?

15. [3 points]: Are there any topics you'd like to see added to the class, or any topics you'd like to see removed?

End of Quiz 2

Name:

MIT OpenCourseWare
http://ocw.mit.edu

6.828 Operating System Engineering
Fall 2012

For information about citing these materials or our Terms of Use, visit: http://ocw.mit.edu/terms.



Department of Electrical Engineering and Computer Science

MASSACHUSETTS INSTITUTE OF TECHNOLOGY

6.828 Operating System Engineering: Fall 2005

Quiz II Solutions

Average 84, median 83, standard deviation 10.

I O/S Bugs

1. [10 points]: The correctness of JOS depends in part on interrupts not occurring while the kernel is executing. Suppose that by mistake your JOS kernel didn't turn off interrupts when entering the kernel. Explain a specific problem that would arise if a clock interrupt occurred while executing the code for a system call in the kernel.

Answer: The interrupt will execute on the kernel stack, and will push its state below the system call's stack frames. `trap()` in the interrupt will overwrite the current environment's `env_tf` with a return point in the system call code in the kernel. There are two possible sequences of events now, both of which lead to errors.

If the interrupt returns without calling `sched_yield()`, it will return back into the system call. When the system call returns, it will use the same `env_tf`, and will return into the middle of the kernel system call code rather than back to user space.

If the interrupt calls `sched_yield()` to switch to a new environment, it will return to the user space in the new environment. The old environment's system call will never be finished: next time it runs, it will run in user space as if the system call had returned.

II Shells

The file `script` contains the following two lines:

```
echo first
echo 2nd
```

You type the following command to a shell:

```
$ sh < script > out
```

2. **[5 points]:** What should file `out` contain after this command?

Answer:

```
first
2nd
```

The JOS library sets the PTE_SHARE bit in the page table entry of the page that contains each `struct Fd`. This bit causes `spawn` and `fork` to share `Fd` pages read/write with child environments, instead of copying them.

Here's what `struct Fd` looks like, from `inc/fd.h`:

```
struct Fd {
  int fd_dev_id;
  off_t fd_offset;
  int fd_omode;
  union {
    // File server files
    struct FdFile {
      int id;
      struct File file;
    } fd_file;
  }
}
```

3. [10 points]: What would file `out` contain in the above example if the JOS library copied `Fd` pages in `spawn` and `fork`, instead of sharing them read/write? Explain your answer.

Answer: In unmodified JOS, the page containing the `struct Fd` is shared between the parent, the children, and the file server. Thus the children see each others' modifications to `Fd.fd_offset` when they write the file, so the second `echo` starts writing where the first left off. Similarly, the children share the `Fd` page with the server, so both children see the server's changes to the `Fd.fd_file.file.f_size` field when either child sends a `set_size` RPC to the server to increase the file size.

If the `Fd` page isn't shared, the file server will see that its `Fd` page has no other references. This will cause it to believe that all clients have closed the file descriptor, and it will reject both clients' `set_size` RPCs, leaving the file size at zero.

If the file server didn't check the reference count of the `Fd` page, it's likely that `out` would end up containing `2nd` followed by a newline. The reason is that the second `echo` would start writing at offset zero (since it doesn't see the first `echo`'s change to the offset), would think that it had appended bytes to the end of the file (since it doesn't see changes to the size field), and would thus send a `set_size` RPC to set the file size to four.

III IPC

Jochen Liedtke's paper *Improving IPC by Kernel Design* suggests, in Section 5.3.5, that IPC performance may be improved if the environment sending a message yields the CPU to the receiving environment. It occurs to you that the following simple modification to JOS might achieve the same effect:

```
static int
sys_ipc_try_send(envid_t envid, uint32_t value, void *srcva, unsigned perm)
{
  struct Env *e;
  int r;

  if((r = envid2env(envid, &e, 0)) != 0)
    return r;

  if(e->env_ipc_recving == 0)
    return -E_IPC_NOT_RECV;

  // we've omitted code to deal with the page mapping...

  e->env_ipc_recving = 0;
  e->env_ipc_from = curenv->env_id;
  e->env_ipc_value = value;
  e->env_status = ENV_RUNNABLE;

  // here's the original code:
  // return pgmap;

  // here's your modification:
  curenv->env_tf.tf_regs.reg_eax = pgmap;
  sched_yield();
}
```

Suppose that most IPCs are sent as part of remote procedure calls (RPCs), in which a client sends a request to a server and the server sends a response. The client environment sends the request message to the server and waits to receive the response; the server then sends the response and waits to receive the next request. You should assume that, at the time the client sends the request, the server is waiting in `sys_ipc_recv()`. As a reminder, the bottom of the page shows `ipc_send()` from `lib/ipc.c` and `sys_ipc_recv()` from `kern/syscall.c`.

4. [10 points]: How would the preceding modification to `sys_ipc_try_send()` affect JOS's IPC performance, where performance is measured as the number of system calls and returns required to perform an RPC? Explain the specific reasons for any change in performance.

Answer: The modification will increase the number of system calls. When the server sends the response, the client will not already be waiting to receive, so the server's first send will fail and it will have to call `sys_yield()`. Without the modification, the client's send returns and the client calls `sys_ipc_recv()` before the server runs.

```
void
ipc_send(envid_t to_env, uint32_t val, void *pg, int perm)
{
  while(1){
    int r = sys_ipc_try_send(to_env, val, pg, perm);
    if(r != -E_IPC_NOT_RECV)
      return r;
    sys_yield();
  }
}

static int
sys_ipc_recv(void *dstva)
{
  if(dstva < (void*)UTOP && ROUNDDOWN(dstva, PGSIZE) != dstva)
    return -E_INVAL;

  curenv->env_ipc_recving = 1;
  curenv->env_ipc_dstva = dstva;
  curenv->env_ipc_value = 0;
  curenv->env_ipc_from = 0;
  curenv->env_ipc_perm = 0;
  curenv->env_status = ENV_NOT_RUNNABLE;
  sched_yield();
}
```

IV Soft Updates

Recall the soft updates technique described in McKusick and Ganger's *Soft Updates: A Technique for Eliminating Most Synchronous Writes in the Fast File System*. You run the following benchmark on a soft updates file system:

```
fd = creat("foo", 0666);
for(i = 0; i < 100; i++){
  write(fd, buf, 8192); /* append a block to the file's end */
  fsync(fd); /* force the data to disk */
}
```

You find that each iteration of the for loop causes four disk writes, in this order:

A. part of the bit-map of free blocks,

B. the new data block,

C. one of the i-node's indirect blocks, and

D. the i-node itself.

The file system uses 8192-byte blocks.

5. [5 points]: Why does the file system write the new data block before the indirect block? Give an example of what could go wrong if the file system performed these two writes in the opposite order.

Answer: If a crash and reboot occurs after the indirect block is written but before the data block is written, the file will contain a block of junk (data from some previously deleted file).

6. [10 points]: Why does the file system write the bit-map of free blocks before the indirect block? Give an example of what could go wrong if the write to the bit-map of free blocks occurred after the write to the indirect block.

Answer: If a crash and reboot occurs after the indirect block is written but before the bit-map is written, the block will be both marked free and in the file. Now the block could be allocated to a second file at the same time it is in use by the first file, which is not correct.

You run the same benchmark on an FFS file system (the predecessor to soft updates). The benchmark is faster on an FFS file system, requiring only three disk writes per loop iteration; FFS does not update the bit-map of free blocks on each iteration. Instead, FFS writes any dirty parts of the bit-map once every 30 seconds.

7. [10 points]: Explain why it's OK for FFS to write the free bit-map after it writes the indirect block, even though it is not OK for soft updates.

Answer: FFS's `fsck` re-builds the free bit-map after each reboot, by looking at all files and collecting the set of all blocks not used by any file.

V SMP JOS

Suppose you would like to modify JOS to run on a symmetric multi-processor (SMP): a machine with eight x86 CPUs, a shared bus, and a single memory system. Based on advice you heard in the 6.828 project presentations, you arrange for at most one CPU at a time to execute in the JOS kernel. Any number of CPUs can be executing in user space (for different environments). You ensure at most one CPU in the kernel by acquiring a spin-lock in `trap()`, and releasing it in `env_pop_tf()`:

```
struct Lock big_lock;
struct Env *cpu_env[8]; // the environment running on each CPU

void
trap(struct Trapframe *tf)
{
  acquire(&big_lock);

  curenv = cpu_env[cpu_num()]; // find the environment running on this CPU
  curenv->env_tf = *tf;

  trap_dispatch(&curenv->env_tf);

  if (curenv && curenv->env_status == ENV_RUNNABLE)
    env_run(curenv);
  else
    sched_yield();
}

void
env_pop_tf(struct Trapframe *tf)
{
  release(&big_lock);

  __asm __volatile("movl %0,%%esp\n"
    "\tpopal\n"
    "\tpopl %%es\n"
    "\tpopl %%ds\n"
    "\taddl $0x8,%%esp\n" /* skip tf_trapno and tf_errcode */
    "\tiret"
    : : "g" (tf) : "memory");
  panic("iret failed");  /* mostly to placate the compiler */
}
```

Each CPU has a separate stack on which it executes when it enters the kernel. The `cpu_env` array holds, for each CPU, a pointer to the environment that is running on that CPU, and `cpu_num()` returns the current CPU's number (0 through 7). Each CPU has its own idle environment that `sched_yield()` runs when there is nothing else to do.

Here's the kernel code to allocate a page of physical memory, from `kern/pmap.c`:

```
int
page_alloc(struct Page **pp_store)
{
  *pp_store = LIST_FIRST(&page_free_list);
  if (*pp_store) {
    LIST_REMOVE(*pp_store, pp_link);
    page_initpp(*pp_store);
    return 0;
  }
  return -E_NO_MEM;
}
```

8. [5 points]: Give an example of a way in which `page_alloc()` would likely behave incorrectly if multiple CPUs were allowed to execute in the JOS kernel at the same time.

Answer: If two CPUs call `page_alloc()` at the same time, they might both allocate the same page.

It turns out that the big lock strategy is not quite sufficient to make JOS work correctly on an SMP. Consider, for example, a simple `sched_yield()` algorithm that runs the next environment marked `ENV_RUNNABLE`, and ignores environments marked `ENV_FREE` or `ENV_NOT_RUNNABLE`.

9. [10 points]: Why is this simple scheduler incorrect on an SMP, and how can it be fixed?

Answer: Two CPUs might both decide to run the same environment. There needs to be a new environment state, `ENV_RUNNING`, to indicate environments that are running.

Suppose that your application consists of dozens of environments that constantly send each other IPC messages; there are always more environments waiting to run than there are CPUs. You notice that your applications spend essentially all of their time in the JOS kernel, specifically the system calls that send and receive IPC messages.

10. [10 points]: Compare the likely performance of your application on an eight-CPU SMP with performance on an ordinary single-CPU machine. Briefly explain your answer.

Answer: About the same total performance. Only one CPU is allowed in the kernel at a time, so the other seven CPUs will waste their time waiting for the big lock.

You'd like to make the send and receive system calls higher performance by letting them run in parallel on different CPUs as long as the environments involved are distinct. You modify `trap()` and `env_pop_tf()` to not acquire/release the big lock for send and receive system calls. Instead, you want to add locks inside the send and receive system calls that preserve correctness and allow two environments executing on different CPUs to send concurrently to different environments.

11. [10 points]: Add `acquire()` and `release()` calls to the copies of `sys_ipc_try_send()` and `sys_ipc_recv()` below so that they are correct and allow concurrent sends to different environments. Explain where you declare the `Lock` variables. Briefly explain why your locking scheme allows concurrent sends.

```
static int
sys_ipc_try_send(envid_t envid, uint32_t value, void *srcva, unsigned perm)
{
  struct Env *e;
  int r;

  if((r = envid2env(envid, &e, 0)) != 0)
    return r;

  acquire(e->env_ipc_lock); // HERE

  if(e->env_ipc_recving == 0){
    release(e->env_ipc_lock); // HERE
    return -E_IPC_NOT_RECV;
  }

  // we've omitted code to deal with the page mapping...

  e->env_ipc_recving = 0;
  e->env_ipc_from = curenv->env_id;
  e->env_ipc_value = value;
  e->env_status = ENV_RUNNABLE;
  release(e->env_ipc_lock); // HERE
  return pgmap;
}

static int
sys_ipc_recv(void *dstva)
{
  acquire(curenv->env_ipc_lock); // HERE
  if (curenv->env_ipc_recving)
    panic("already recving!");

  curenv->env_ipc_recving = 1;
```

```
  curenv->env_ipc_dstva = dstva;
  curenv->env_status = ENV_NOT_RUNNABLE;
  release(curenv->env_ipc_lock); // HERE
  sched_yield();
  return 0;
}
```

Answer: Every environment has a separate `env_ipc_lock` that guards its IPC receive variables. This locking scheme allows concurrent sends to different environments because the two sends require different locks.

VI 6.828

We would like to hear your opinions about 6.828, so please answer the following questions. (Any answer, except no answer, will receive full credit!)

12. [1 points]: On a scale of 1 to 10, please rate how much you learned from 6.828 (1 is low, 10 is high).

Answer: We (the staff) learned a lot from you (the students) this semester!

13. [2 points]: If you could change one aspect of 6.828, what would it be?

Answer: We're planning to write our own simple operating system for the x86, in the spirit of UNIX V6 but more modern. We hope to use this new O/S in place of V6. We love V6 but it might be better not to have to spend time on its quirks and on the PDP 11/40.

14. [2 points]: Are there any topics you would like to see added to or removed from the class?

Answer: Perhaps we'll teach more about locking and concurrency, and read more papers.

End of Quiz II

MIT OpenCourseWare
http://ocw.mit.edu

6.828 Operating System Engineering
Fall 2012

For information about citing these materials or our Terms of Use, visit: http://ocw.mit.edu/terms.



Department of Electrical Engineering and Computer Science

MASSACHUSETTS INSTITUTE OF TECHNOLOGY

6.828 Fall 2007

Quiz II Solutions

The average was 80.

I Virtual Machines

The x86 SIDT and LIDT instructions read and write the interrupt descriptor table register (IDTR). Recall that the IDTR specifies the address of the interrupt descriptor table, which controls where the CPU jumps when a fault or hardware device interrupt occurs. LIDT loads the IDTR (modifies the register) while SIDT just reads the register so the executing code can examine its value. LIDT is privileged: it can only be executed at CPL 0. SIDT is not a privileged instruction: it can be executed at CPL 3.

1. [6 points]: A user program in JOS or xv6 could use SIDT to examine the IDTR. Is this a bad thing? Why or why not?

Answer: It is not bad. The IDTR value is a constant in JOS and xv6, so there's little to be learned from it.

Suppose you want to make a virtual machine monitor (VMM) for the x86, along the same lines as the Disco paper. Remember that, for efficiency, Disco directly executes guest operating system instructions when possible. It executes them with ordinary user privilege, so that the guest kernel traps to the VMM when the guest tries to execute privileged MIPS instructions.

2. [6 points]: The x86 will trap to the VMM if the guest kernel tries to use LIDT to load an address a into the IDTR. Can the VMM just itself execute LIDT a and return to the guest? Why or why not? You can assume that a is a valid linear address that really does refer to the base of the guest kernel's desired IDT, and that the IDT entries all refer to valid code locations within the guest kernel.

Answer: No. Hardware interrupts and traps should enter the VMM, not the guest kernel, so that the VMM can decide whether the guest kernel should see them. Thus the VMM must supply its own IDT.

3. [6 points]: It turns out that SIDT causes problems for an x86 VMM. Explain why.

Answer: SIDT lets the guest see the real contents of the IDTR, which point to the VMM's IDT. The guest is expecting to see a pointer to the guest's IDT. The fact that SIDT does not trap when executed at CPL 3 means the VMM doesn't get a chance to intervene.

II File System Naming

Look at xv6's `_namei()` in fs.c on sheet 43. The job of `_namei()` is to turn a pathname, such as `/usr/rtm/quiz.tex`, into a pointer to an in-core i-node. The while() loop processes each element of the pathname in turn, descending the directory tree. `_namei()` locks each directory while it is looking at it; for example, it locks the inode of `/` while it is searching for `usr`. It unlocks the directory at end of the loop. These are not spin locks; instead these locks set the inode's `I_BUSY` flag, and waiting processes sleep instead of spinning.

Suppose process P1 has called `_namei()` for `/usr/rtm/quiz.tex` and is about to execute `ip = next` at line 4379 after finding `rtm` in `/usr`. Thus the old `ip` refers to `/usr`, and `next` refers to `rtm`. `/usr/rtm` has only one file in it: `quiz.tex`. A clock interrupt forces a switch to a different process, which happens to be a root shell that runs

```
rm /usr/rtm/quiz.tex
rmdir /usr/rtm
mkdir /usr/zzz
echo hello > /usr/zzz/quiz.tex
```

All of these commands complete successfully; there are no errors. Then P1's `_namei()` continues.

4. [6 points]: Could the file system allocate `/usr/zzz` the same disk inode (i.e. same i-number) that `/usr/rtm` was using before the `rmdir`? Why or why not?

Answer: No. `_namei()` still has a pointer to the in-core inode of `/usr/rtm`, and has not called `iput()`, so the `ref` reference count is non-zero. Thus `iput()` will not free the inode when `rmdir` finishes. Thus the `mkdir` won't allocate the same inode.

5. [6 points]: What will P1's `_namei()` do after it continues? Will it find the original quiz.tex? Is there a possibility it could instead find the new quiz.tex? Or will something else happen? Explain why.

Answer: `_namei()` will return 0. It will search the original inode of `/usr/rtm` and, since `rm` deleted `/usr/rtm/quiz.tex`, `_namei()` won't find any file.

III File System Atomicity and Recovery

Suppose a program on xv6 creates and writes a file:

```
fd = open("file", O_CREATE | O_WRONLY);
write(fd, "hello", 5);
close(fd);
```

The following is an abbreviated list of the disk writes that would result:

```
write file inode
write directory entry
write block free bitmap
write data block
write file inode
```

These disk writes are synchronous: xv6 waits for the disk to finish each before it starts the next. xv6 uses synchronous writes to enforce a particular order, much as described in Section 2.1 of the Soft Updates paper.

You can assume that individual disk writes are atomic with respect to crashes: if there's a crash or power failure during a write, the write either completes successfully, or does nothing. You should assume that there is no file system checker (i.e. there is nothing like UNIX's `fsck`).

The above sequence writes the file inode twice, which sounds inefficient. Suppose xv6 delayed the first file inode update, and combined it with the second, to save one write:

```
write directory entry
write block free bitmap
write data block
write file inode
```

> **6. [6 points]:** Suppose there's a power failure during the above sequence. What inconsistency could exist in the on-disk file system after the failure as a result of eliminating the first inode write? How might that inconsistency eventually cause the file system to return the wrong data for a `read()`?

Answer: The directory will contain the i-number of an inode that is marked free (type zero). A future creation of a different file might use the same inode, causing the first file to be the same as the second.

Suppose xv6 were modified to omit updating the block free bitmap right away, and rather only update the bitmap every 30 seconds in order to increase efficiency. The write order would then be:

```
write file inode
write directory entry
write data block
write file inode
```

7. [6 points]: Suppose there's a power failure just after the above sequence. What inconsistency could exist in the on-disk file system after the failure as a result of eliminating the free bitmap write? How might that inconsistency eventually cause the file system to return the wrong data for a `read()`?

Answer: The file will contain a block that is on the free list. A future file write might allocate the same block and change its content, thus incorrectly modifying the first file.

IV XFI Memory Protection

The XFI paper describes a system that ensures that a module can only read, write, and execute inside its own memory. XFI would be useful for Linux loadable kernel modules, which the kernel loads from a file into kernel memory and executes with full privilege (CPL 0). Without something like XFI, a buggy or malicious loadable kernel module could wreck the kernel by writing into its memory. XFI could be used to make sure that a loadable kernel module only used its own memory, and did not read or write any of the kernel's data structures.

Below is a copy of Figure 2 of the XFI paper. XFI has inserted code that checks that the target of the `call` really is the start of a function inside the module.

```
   EAX := 0x12345677
   EAX := EAX + 1
   if Mem[EBX - 4] != EAX, goto CFIERR
   call EBX
   ...
   0x12345678
L: push EBP
```

8. [6 points]: Why is it absolutely necessary for XFI to check that the module is jumping to the start of a function? Why can't XFI just enforce that the module is jumping somewhere inside its own memory – i.e. just prevent the module from jumping into the kernel?

Answer: When XFI checks that the module's code doesn't contain instructions that write outside the module's memory, it only looks at instructions that could be reached from the beginnings of functions. If the module could jump anywhere in its memory, it could jump into the middle of a legitimate instruction, thus executing an instruction that XFI did not check. The module could also jump directly to a load or store instruction, bypassing XFI's checks.

You could view XFI as providing a software version of the memory protection provided by the x86's pagetables. You could even imagine using XFI instead of the x86's pagetables in order to protect the kernel against user processes. That is, you might be able to re-design JOS to run user environments with CPL zero (full privilege), but use XFI to make sure environments only read and wrote their own memory. Environments would use function calls into the kernel instead of INT system calls; XFI is powerful enough to check that environments only call approved system call functions in the kernel.

9. [6 points]: Explain two different ways that JOS uses x86 pagetables to do things that XFI (as described in the paper) does not support.

Answer: Copy on write fork and direct transfer of file pages from the file server. XFI doesn't provide the level of indirection in addressing that pagetables provide.

V L3, Microkernels, and fast IPC

The L3 paper (Improving IPC by Kernel Design) describes a number of techniques to achieve fast inter-process communication in a micro-kernel on an x86.

10. [6 points]: Explain why the L3 paper worries much more about avoiding TLB misses than about avoiding data cache misses.

Answer: For both the TLB and the CPU data cache, you will get expensive misses if you touch too many different areas in memory. Thus much of what the paper is doing is reducing the number of distinct areas of memory that the IPC code uses. The paper concentrates on the TLB because it has fewer entries than the data cache (32 vs 512 on the Intel i486).

An IPC sending process in L3 waits until the receiving process is ready to receive (i.e. in the receive system call) before sending the message (see the start of Section 5.1).

11. [6 points]: Explain why L3's high performance depends on the receiver being ready.

Answer: If the receiver is ready, the sending process can directly enter the receiver's user space, carrying along the IPC message in the registers. This direct entry also eliminates the expense of scheduling (putting the sender to sleep and calling the scheduler).

VI Bugs as Deviant Behavior

Recall Section 3.1 of the Bugs as Deviant Behavior paper, which aims to detect possible dereferences of null pointers by looking for inconsistencies between the programmer's pointer checks and dereferences. The paper's checker flags both the examples in 3.1 as bugs.

Assume there is no inter-procedural information – the checker only looks at the current function.

Consider the following code:

```
fn1(int x, int *p){
  if(x){
    printf("p -> %d\n", *p);
  }
  if(p == 0)
    panic("p is null");
}
```

12. [6 points]: Will a checker like the one in 3.1 flag the above code as having inconsistent use of pointers? Why or why not?

Answer: Yes. There is a path through `fn1()` that dereferences `p` and then checks if `p` is null. `p`'s belief set will be "not null" after the dereference, which contradicts the "null" and "not null" implied by the check.

Now consider this code:

```
fn2(int x, int *p){
  if(x){
    printf("p -> %d\n", *p);
  } else {
    if(p == 0)
      panic("p is null");
  }
}
```

13. [6 points]: Will a checker like the one in 3.1 flag the above code as having inconsistent use of pointers? Why or why not?

Answer: No. There is no path through the code that has both a derefence and a check.

VII OKWS

14. [6 points]: Section 6.1 (4) of the OKWS paper mentions that okld does no message parsing. Why is that important to the security of the system?

Answer: Since attackers cannot arrange to send data to okld (even indirectly), they won't have the opportunity to exploit bugs that might exist in okld's message handling code. okld runs as root (unlike the rest of OKWS), so it's particularly important that it not be exposed to attacks.

VIII Scalable Synchronization

Suppose you are running xv6 on a machine with 10 CPUs and shared memory. After reading the Scalable Synchronization paper you're worried that xv6 might have bad spin-lock performance, and you wonder if it might benefit from the techniques in the paper.

15. [6 points]: Would a more scalable spin-lock be likely to improve the performance of `pipewrite()` on sheet 52? Why or why not?

Answer: Probably not. In most cases pipes have just a single reader and a single writer, so even with many CPUs there is not likely to be much contention for any given pipe's lock.

Here's a copy of the paper's Algorithm 5, the MCS list-based queuing lock:

```
procedure acquire_lock(L : ^lock, I : ^qnode)
  I->next := nil
  predecessor : ^qnode := fetch_and_store(L, I)
  if predecessor != nil
    I->locked := true
    predecessor->next := I
    repeat while I->locked // <-- XXX

procedure release_lock(L : ^lock, I : ^qnode)
  if I->next = nil
    if compare_and_swap(L, I, nil)
      return
    repeat while I->next = nil
  I->next->locked := false
```

Suppose 10 CPUs try to acquire the same MCS lock at the same time. Each releases the lock immediately after it successfully acquires it. The amount of time it takes all 10 CPUs to finish acquiring and releasing the lock is mostly determined by the number of cache misses. A CPU will incur a cache miss if it reads a location that was last written on a different CPU, or if it writes a location that is currently in the cache of a different CPU.

You can assume that each CPU's `qnode` structure starts out locally cached on just that CPU. You can assume that variables and data that are only used by one CPU incur no cache misses. You can assume that `predecessor` is non-nil for 9 of the 10 calls to `acquire_lock()`.

16. [6 points]: How many cache misses will be incurred by the line marked XXX, summed over all the CPUs, in the process of all 10 CPUs each acquiring and releasing the lock? Explain your answer.

Answer: Nine. Each waiting CPU will see one cache miss on its `I->locked`, when its predecessor sets `locked` to false at the end of `release()`.

End of Quiz

MIT OpenCourseWare
http://ocw.mit.edu

6.828 Operating System Engineering

Fall 2012

For information about citing these materials or our Terms of Use, visit: http://ocw.mit.edu/terms.



Department of Electrical Engineering and Computer Science

MASSACHUSETTS INSTITUTE OF TECHNOLOGY

6.828 Fall 2008

Quiz II Solutions

I File System Consistency

Ben is writing software that stores data in an xsyncfs file system (see the paper *Rethink the Sync* by Nightingale *et al*). He's nervous about crash recovery. To help himself test, he adds a new system call to his operating system called crash(), which powers off his computer without doing anything else. He also writes a simple "crash" command which calls his system call. His general test strategy is to perform some operations, call crash(), restart the computer and let the file system finish recovering, and then observe what files and data are on the disk.

1. **[7 points]:** First, Ben types the following commands to his shell:

```
% echo hello > foo
% crash
```

After the restart, is Ben guaranteed to see a file foo with contents "hello"? Why or why not?

Answer: Yes. xsyncfs guarantees that any file system writes that causally precede user output will be stable on disk before the user sees the output. echo's write() to foo causally precedes the second "%" prompt in the above output: echo exit()s after it write()s, and the shell wait()s for echo's exit() before it displays the prompt. Thus xsyncfs will have written echo's output to foo before it allows the second % to be displayed.

2. [7 points]: Now Ben runs the following UNIX program which he believes is equivalent to the above commands:

```
pid = fork();
if(pid == 0){
  fd = creat("foo", 0666); // create a file foo
  write(fd, "hello\n", 6);
  close(fd);
  exit(0);
}
wait(&status); // wait for pid to exit
crash();
```

After the restart, is Ben guaranteed to see "hello" in foo? Why or why not?

Answer: No. There is no user-visible output, so xsyncfs does not guarantee to send file writes to the disk. Much of the point of xsyncfs is to avoid immediately writing the disk in cases like this, to increase performance.

3. [7 points]: What would the answers be for the xv6 file system? Why?

Answer: If you used xv6, you'd see "hello" in foo after the restart. That's because the xv6 file system is synchronous: all file system operations (such as creat() and write()) wait for their modifications to be written to disk before they return.

II Virtualization

The paper *A Comparison of Software and Hardware Techniques for x86 Virtualization* states that one of the goals of virtualization is "Fidelity: Software on the VMM executes identically to its execution on hardware, barring timing effects."

The paper also says, at the end of Section 3.2, that the software virtualization technique outlined in Section 3 does not translate guest user-mode code. That is, when the guest operating system executes instructions that would cause a switch to CPL=3 on real hardware, the VMM stops translating code and allows the original guest instructions to execute directly on the hardware.

> **4. [8 points]:** Explain a way that a carefully constructed guest kernel and/or user-mode program could exploit direct execution of user-mode code to discover whether it was executing on a virtual machine. You should assume the system outlined in Section 3, running on a 32-bit x86.

Answer: One possibility is for the guest kernel to transfer to user space with interrupts turned off (IF clear in EFLAGS). The VMM will leave interrupts turned on in the real hardware, since it needs to see the real hardware interrupts; IF will only be clear in the shadow EFLAGS. User-mode code can use pushf to examine the real hardware EFLAGS. If interrupts are enabled in this situation, the software is running on a virtual machine; if interrupts are disabled, the software is running on a real machine.

III Fault Tolerance

Ben is impressed with the paper *Fault Tolerance Under UNIX*, by Borg *et al.* He builds a system that is identical – with one exception. In order to get better performance, his hardware has two busses, and every machine has a connection to both busses. When a machine needs to send a message, it selects one of the two busses at random, and sends the message on that bus.

> **5. [8 points]:** Explain why this change will cause serious problems to the correctness of the system. Give a specific example of something that will likely go wrong.

Answer: Suppose two different machines, M_1 and M_2, send messages to M_3 at the same time, but use different busses. M_3 might see the messages in one order, and M_3's backup might see them in the other order. If M_3 were to crash, its backup would restore its state incorrectly, since it would replay those two messages in a different order than M_3 saw them.

IV Bugs as Deviant Behavior

Have a look at Figure 3 of the paper *Bugs as Deviant Behavior* by Engler *et al*. Suppose first function in Figure 3 looked like:

```
ssize_t proc_mpc_write(struct file *file,
                       const char *buff){
  retval = parse_qos(buff, incoming);
}
```

That is, suppose the first seven lines of the original function were deleted.

> **6. [8 points]:** Would the checker described in Section 7.1 emit an error for this modified function? Why or why not?

Answer: No. The checker looks for pointers that the code both dereferences and treats as suspect (by passing to a "paranoid" function). There are no such pointers in this function.

It's possible that the checker's comparison of functions that are assigned to the same function pointer would catch the error even in this modified code, if some other abstractly related function passes the buff argument to a paranoid function.

V Livelock

Answer this question in the context of the paper *Eliminating Receive Livelock in an Interrupt-driven Kernel* by Mogul *et al.*

> **7. [8 points]:** Figure 6-3 shows that performance with polling and no quota is quite poor under high input load, but polling with a quota performs much better. Explain what happens without a quota that results in almost zero performance under high input load, and how a quota helps solve this problem.

Answer: Without a quota, the device driver spends 100% of the CPU time reading packets from the device and queuing them in a software output queue. This leaves no CPU time to give the queued packets to the output device hardware, so all packets (after the first queue's worth) are dropped due to overflow of the software output queue. With a quota, the polling system alternates between reading input packets and sending packets to the output device.

VI KeyKOS

Answer this question in the context of the paper *The KeyKOS Nanokernel Architecture* by Bomberger *et al*.

8. [4 points]: Suppose a machine running KeyNIX loses power while creating a new file in a directory. After the system is powered back up and executes to a quiescent state (e.g. so that any recovery code is executed), what are the possible states that the file system could be in? Circle all that apply (-1 point for each wrong answer).

A. The new file is allocated but the directory does not contain the new file.

B. The new file is not allocated but the directory contains a reference to the new (unallocated) file.

C. The new file is allocated and the directory contains a reference to the new file.

D. The new file is not allocated and the directory does not contain a reference to the new file.

Answer: **C** and **D**.

9. [7 points]: Most file system implementations worry a great deal about what happens after a crash, but the KeyNIX file system has no explicit file system recovery code. Explain what strategy KeyNIX and KeyKOS use to recover from crashes such as the one above. In particular, would the KeyNIX file system recover to a consistent state if a new file was created but not added to its parent directory by the time the machine was powered off (and if so, how)?

Answer: KeyKOS periodically checkpoints the entire state of the system to disk, including process memory and the register state of each process. If there's a crash, KeyKOS loads the most recent checkpoint into memory and continues executing from that point. The KeyNIX file system is implemented as a set of processes; these processes only write in-memory representations of files and directories, and do not directly write the disk. If there were to be a crash during the creation of a file, two scenarios are possible, both of which lead to a consistent state after recovery. The last checkpoint might have been taken early enough that the file creation hadn't started; in this case the recovered system will start out with neither file allocated nor directory modified (though perhaps whatever program eventually tried to create the file will re-execute to that point and create it again). Or the last checkpoint might have been taken after the start of creating the file, in which case the file-creation code will continue running after the recovery and create both the file and the directory entry.

VII HiStar

Answer this question in the context of the paper *Making Information Flow Explicit in HiStar* by Zeldovich *et al*.

Recall that KeyNIX maintains a global table mapping process IDs to capabilities (keys) that allow sending a message to that process's Unix keeper. When one process running on top of KeyNIX wants to send a signal to a different process, the sender's Unix keeper must check that the sender is allowed to send a signal to the recipient process (e.g. that both processes belong to the same user), before sending a "kill" message to the recipient's Unix keeper.

HiStar implements signals in a similar fashion: the Unix library locates the signal gate for the recipient process and sends a "kill" message to that gate. (Even though the Unix library has no global process ID table, it can still find the signal gate for a given process ID by enumerating that process's container to find the signal gate, since a process ID is the ID of that process's container object.)

> **10. [7 points]:** The HiStar Unix library is not trusted; anything the Unix library can do, ordinary program code can also do. As a result, the Unix library cannot be trusted to check that signals are only sent to processes owned by the same user. How does HiStar prevent a malicious user program from sending signals to other users' processes?

Answer: HiStar relies on the signal gate's clearance to restrict the set of threads that can send a signal to a process. Each process's signal gate has a clearance of $\{u_w 0, 2\}$, where the u_w category corresponds to the user that owns that process. This means only threads that either own u_w or have a label of $\{u_w 0, ...\}$ can invoke this gate, and one process cannot send signals to processes of other users whose categories it does not own.

11. [7 points]: Can KeyNIX be re-designed to prevent a compromised Unix keeper (controlled by an attacker) from sending signals to other users' processes? Sketch out a design that would run on KeyKOS, or describe why it would be difficult to implement using capabilities.

Answer: One possible design would be to keep the keys to Unix keepers in a per-user process table, rather than in a single global table. Unix keepers would maintain a key to the process table corresponding to the user they're running as, and would be able to send signals to processes in that table. Unix keepers of processes running as root would have a key to a top-level table of keys for every user's process table, allowing root to send signals to anyone's process. When a root process calls setuid to run as a particular user, its Unix keeper would delete its key for the top-level table, and only keep a key to the table for that user. A non-root process would not be able to send signals to processes of other users, since it would have no way to get the relevant key.

An alternative design would be to create a separate key representing each user in the system (the key could refer to anything, e.g. an empty segment), and pass this key as part of the "kill" message sent to any Unix keeper. Each Unix keeper would have a key for the user it's running as, and on receiving a "kill" message, it would use Keybits to compare the key it received with the key for the user it's running as, and only accept the signal if the keys matched. Care would have to be taken for signals sent by root. If a Unix keeper running as root were to include root's key in signals sent to non-root processes, those processes' Unix keepers could save that key and use it to send their own signals to other processes. To avoid this problem, keepers running as root would have to use the correct user's key when sending signals, perhaps using some other table to ensure they don't divulge one user's key to a Unix keeper of another user.

VIII RCU

Answer this question in the context of the paper *Read-Copy Update* by McKenney *et al*.

12. [7 points]: Illustrate a specific problem that could occur if the reference-counted linked list search code in Figure 4 did not obtain a read lock on list_lock. Be sure to indicate what would go wrong as a result.

Answer: `search()` might be about to use `p->next` for some element, while at the same time another thread is calling `delete()` for the same element. A third thread might re-use the freed element's memory for another purpose, overwriting `p->next`. If `search()` now dereferences `p->next`, it may crash or interpret an inappropriate piece of data as a list element.

13. [7 points]: The RCU version of the linked list search code in Figure 8 does not use locks. Describe the steps taken by RCU to ensure that the problem seen in the above question does not occur in this case.

Answer: In Figure 9, RCU's `kfree_rcu()` does not immediately free the list element; it defers freeing it until all threads have voluntarily given up the CPU. Since `search()` in Figure 8 doesn't voluntarily context switch, it can safely use any list element without fear that it will be concurrently freed and reused.

14. [8 points]: Suppose you want to use RCU for a linked list in the xv6 kernel. Explain how to detect quiescence in xv6. To maximize read performance, you may not use any additional code (such as locks) around read accesses to the linked list.

Answer: xv6 has involuntary context switches: if interrupts are enabled (which they usually are in the kernel), a timer interrupt can cause a switch to a different process. This means that any RUNNING or RUNNABLE thread that's in the kernel might hold a reference to an RCU-protected object; let's call them "suspect" threads. Thus, when an object is passed to `kfree_rcu()`, RCU must defer the free until every thread that is suspect at that time has voluntarily given up the CPU. In xv6, this means waiting for each such thread to call `sleep()` or return to user space or exit. One possibility is to associate, with each thread, a count of the number of voluntary context switches it has performed. `kfree_rcu()` would tag the object with each suspect thread's count. It would free the object after all of those threads' counts had changed.

MIT OpenCourseWare
http://ocw.mit.edu

6.828 Operating System Engineering

Fall 2012

For information about citing these materials or our Terms of Use, visit: http://ocw.mit.edu/terms.



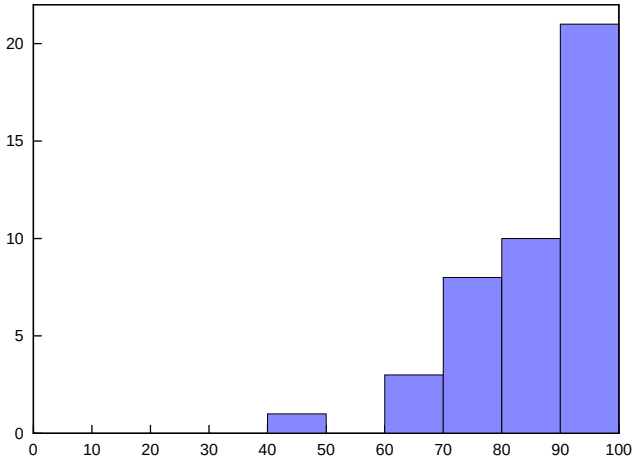
Department of Electrical Engineering and Computer Science

MASSACHUSETTS INSTITUTE OF TECHNOLOGY

6.828 Fall 2010

Quiz II Solutions

Mean 85 Median 89 Std. dev 11.09



I Short Paper Questions

You are using an xsyncfs file system as described in the paper "Rethink the Sync." You run the following program with its standard output connected to your terminal. Initially, the file xyzzy does not exist.

```
int main(void)
{
  // O_CREAT means create the file if it doesn't already exist.
  // O_WRONLY means open for writing.
  int fd = open("xyzzy", O_CREAT | O_WRONLY);
  write(fd, "one", 4);
  write(fd, "two", 4);
  close(fd);
  printf("done");
  return 0;
}
```

1. [8 points]: Sadly a power failure occurs just after you start the program, and you do not see `done` on the console. You restart the system (including the file system's recovery procedure) and look at file xyzzy. Write yes or no after each line, depending on whether it is possible state for xyzzy.

- xyzzy is empty **Yes**
- xyzzy contains just `one` **Yes**
- xyzzy contains just `two` **No**
- xyzzy contains `onetwo` **Yes**

Suppose now that you delete xyzzy and run the same program with its output redirected to file plugh:

```
$ program > plugh
```

2. [8 points]: Again, the power fails just after you start the program, and you don't see any output on your console. After restart you see that file plugh contains `done`. Which states for file xyzzy are possible now?

- xyzzy is empty **No**
- xyzzy contains just `one` **No**
- xyzzy contains just `two` **No**
- xyzzy contains `onetwo` **Yes**

The paper "A Comparison of Software and Hardware Techniques for x86 Virtualization" describes techniques by which a software VMM can detect guest kernel modifications to the guest's page tables. The VMM needs to be aware of such modifications so that the VMM can reflect them in the "shadow" page table that the VMM installs in the real hardware %cr3. Indicate whether each of the following statements are true or false about the the paper's description of how the VMWare software VMM deals with PTE writes:

3. [6 points]:

True /(False): The VMM uses binary translation (BT) to insert instructions before every guest kernel memory-referencing instruction to detect whether the instruction modifies a PTE.

(True)/ False : The VMM write-protects the pages that make up the guest's page tables.

True /(False): Each guest kernel modification of a PTE results in a page fault to the VMM.

True /(False): The shadow page table is identical to the guest kernel's page table, except that some PTE's present or writable bits differ.

Mark each of the following statements according to whether it accurately reflects the content of the paper "Efficient System-Enforced Deterministic Parallelism."

4. [8 points]:

True /(False): The output of a program on Determinator cannot be influenced by any factors other than the program code itself.

True /(False): A given program on Determinator always interleaves the execution of memory read and write instructions from different CPUs in the same order.

True /(False): A program run on Determinator cannot have any bugs relating to concurrency.

True /(False): The Determinator file system is implemented using Puts and Gets between the file system server and its clients.

(True)/ False : Each Get in a parent must be matched by a Ret in the corresponding child.

True /(False): A parent can use Get to observe the contents of a child space's memory at any time.

II Concurrency and RCU

After taking 6.828, Ben is fascinated with concurrency, and decides to implement concurrent stacks. He starts with the following correct serial implementation of a stack:

```
1  elem_t *head;   // the top of the stack
2
3  void push(int key, int value)
4  {
5    elem_t *e = malloc(sizeof(*e));
6    e->next = head;
7    e->key = key;
8    e->value = value;
9    head = e;    // Put it on the stack
10 }
11
12 elem_t *pop(void)
13 {
14   elem_t *e = head;
15   if (e) head = e->next;
16   return e;
17 }
18
19 elem_t *search(int key)
20 {
21   for (elem_t *e = head; e; e = e->next) {
22     if (e->key == key) {
23       return e;
24     }
25   }
26   return NULL;
27 }
```

Ben wants to run this code on his multicore computer. The cores share a cache-coherent memory and Ben places `head`, `freelist`, etc. in shared memory, so that different cores can push and pop from the shared stack.

5. [8 points]: Ben is pretty sure this code cannot run correctly on a multicore computer. That is, if two threads on different cores concurrently invoke `search`, `push`, and `pop`, then there can be races. To refresh Ben's mind, please give an example of a non-benign race in Ben's code.

Answer: Consider the following interleaving of two threads, performing a push and a pop respectively. After this, the popped element will still appear in the stack.

```
6 e->next = head;
7 e->key = key;
8 e->value = value;
                        14 elem_t *e = head;
                        15 if (e) head = e->next;
9 head = e;
```

Ben decides to make the implementation correct using a read-write lock. Here are the locked versions of `push` and `search`:

```
struct rwlock rwlock;

void locked_push(int key, int value)
{
  acquire_write(&rwlock);
  push(key, value);
  release_write(&rwlock);
}

elem_t *locked_search(int key)
{
  acquire_read(&rwlock);
  elem_t *e = search(key);
  release_read(&rwlock);
  return e;
}
```

Ben implements read/write locks correctly: either one writer can acquire the lock in write mode, or several readers can acquire the lock in read mode. The acquire parts of his read/write lock are. You don't need to understand them in detail.

```
struct rwlock {
  spinlock_t l;
  volatile unsigned nreader;
};

void acquire_read(struct rwlock *rl)
{
  spin_lock(&rl->l);
  atomic_increment(&rl->nreader);
  spin_unlock(&rl->l);
}

void acquire_write(struct rwlock *rl)
{
   spin_lock(&rl->l);
   while (rl->nreader) /* spin */ ;
}
```

6. [6 points]: If Ben invokes many `locked_search`'s concurrently, why does the version with a read/write lock perform better than if Ben had used a regular spin lock?

Answer: The read/write lock allows searches to proceed in parallel, whereas a spinlock serializes all search operations.

7. [6 points]: Ben observes that even with a read/write lock and no concurrent `push`'s or `pop`'s, 48 concurrent `locked_search`'s run slower than 48 concurrent `search`'s. Explain briefly why.

Answer: `acquire_read` is itself serialized because it briefly holds a spinlock, which incurs overhead in `locked_search`.

Inspired by the RCU paper ("Read-copy update" by McKenney et al.), Ben decides to create an RCU-like list implementation, but based on time instead of quiescent states. He adds two fields to `struct elem_t` (`time` and `delayed_next`) and introduces the following RCU functions:

```
elem_t *delayed_freelist;
int safe_time[NUMCPU];

void rcu_free(elem_t *e)
{
  e->time = time_since_boot();
  e->delayed_next = delayed_freelist;
  delayed_freelist = e;
  rcu_gc();
}

void rcu_safe(void)
{
  safe_time[mycpu] = time_since_boot();
}

void rcu_gc(void) { /* .. see below .. */ }
```

When a thread wants to free an element, it calls `rcu_free` to inform RCU that it is done with the element. The element cannot be reused yet, because other threads may have a reference to it, so `rcu_free` adds the element to the `delayed_freelist`. When a thread has no references to any element, it informs RCU by calling `rcu_safe`. You can assume that `time_since_boot` is very cheap to execute. The `rcu_gc` function frees elements that are safe to free (i.e., elements that cannot be in use by any thread).

An example of how these functions are intended to be used is as follows:

```
void thread1(void) {
  elem_t *e;
  while (1) {
    acquire_write(&rwlock);
    if ((e = pop())) {
      compute1(e);
      rcu_free(e);
    }
    rcu_safe();
    release_write(&rwlock);
  }
}

void thread2(void) {
 elem_t *e;
 while (1) {
   if ((e = search(random())))
      compute2(e);
   rcu_safe();
  }
}
```

This code assume there is a list with many elements. Thread 1 repeatedly pops an element from this list, computes with it, `rcu_free`'s it, and informs RCU it has no references to any element. Thread 2 searches the list, computes with the found element, and informs RCU it has no references to any element.

8. [8 points]: Why must RCU delay free operations? Give an example scenario and an explanation of what might go wrong if RCU didn't delay free operations.

Answer: A writer thread could free an element and the memory for that element could be reused for something else while a reader thread is in the process of computing with that element. RCU free delays actually freeing the element until no reader thread could possibly be using it.

9. [10 points]: Complete pseudocode for the RCU garbage collector (`rcu_gc`) that frees all elements from the delayed free list that can safely be freed (use `free` to free the individual elements).

```
void rcu_gc(void)
{


  // Find the minimum value in the safe_time array
  min_safe = array_min(safe_time, NUMCPU)

  // Free elements who's free time is less than the minimum safe time
  for each element e in delayed_freelist
    if e->time < min_safe
      remove e from delayed_freelist
      free e









}
```

Consider the following two variations on `thread2` (differences underlined):

```
void thread2_unlocked(void) {
 elem_t *e;
 while (1) {
   int key = random();
   if ((e = search(key))) {
      compute2(e);
   }
   // No rcu_safe();
  }
}

void thread2_locked(void) {
 elem_t *e;
 while (1) {
   int key = random();
   if ((e = locked_search(key))) {
      compute2(e);
   }
  }
}
```

10. [8 points]: Assuming no concurrent `push`'s or `pop`'s, would 48 concurrent `thread2`'s perform as fast as 48 concurrent `thread2_unlocked`'s, as slow as 48 concurrent `thread2_locked`'s, or somewhere between? Explain briefly.

Answer: `thread2` will perform very nearly as fast as `thread2_unlocked` (this is what's cool about RCU). There is slight overhead from maintaining `safe_time` that `thread2` incurs over `thread2_unlocked`.

III Network driver

Late one night, Ben Bitdiddle is tooling away at his E100 receive implementation for the network lab. For reference, the receive frame descriptor (RFD) format is

Command word						Status word			
EL	S	000000000	H	SF	000	C	0	OK	Status bits
Link address									
Reserved									
0	0	Size				EOF	F	Actual count	

He implements receive as follows:

```
// The number of RFD's in the receive ring
#define NUM_RFDS 16
// Pointers to the RFD's in the receive ring
static struct rfd *rfd_ring[NUM_RFDS];
// The index of the next RFD to read a packet from
static unsigned int rfd_tail;

// Receive one packet and store it in to buf.
int e100_receive(char *buf)
{
  struct rfd *rfd = rfd_ring[rfd_tail % NUM_RFDS];
  if (!(rfd->status & RFD_STATUS_C))
    // No packets in receive ring
    return 0;
  // Clear completed ("C") bit
  rfd->status = 0;
  // Copy packet data
  int actualCount = rfd->actualCount & 0x3FFF;
  memmove(buf, rfd->data, actualCount);
  // Move to the next entry in the ring
  rfd_tail++;
  return actualCount;
}
```

Ben tests his implementation using the echo server and finds that it works. Proud of his succinct implementation, he shows it off to Louis Reasoner, who spots a bug.

11. [8 points]: Louis points out to Ben that he needs to manipulate the suspend ("S") bits in the RFD's, so Ben sets the S bit on `rfd_ring[NUM_RFDS-1]` during initialization and adds the following code just after the `memmove` in `e100_receive`:

```
// Set the suspend bit on this RFD
rfd->command = RFD_COMMAND_S;
// Clear the suspend bit on the previous RFD
rfd_ring[(rfd_tail + NUM_RFDS - 1)%NUM_RFDS]->command = 0;
```

Give an example of a problem that Ben's addition prevents.

Answer: Without Ben's addition, if `e100_receive` fails to keep up with the incoming packets, the E100 will overwrite packets in the ring. If it overwrites a packet that `e100_receive` is in the process of copying out of the ring, the driver will return a corrupted packet. (Note that Ben's addition does not prevent *dropped* packets. Before his addition, overwritten packets could be considered dropped; after his addition, the RU will drop packets if the ring overflows.)

12. [8 points]: Ben again tests his code using the echo server. Now confident in his implementation, Ben shows it off to Alyssa P. Hacker, who informs Ben that, while the change he just made was important, it missed something: just after clearing the suspend bit on the previous RFD, he needs to issue an "RU resume" command. Give an example of a problem that Alyssa's addition fixes.

Answer: Without Alyssa's addition, if more than `NUM_RFDS` packets arrive before `e100_receive` is able to process them, the RU will suspend permanently and drop packets for the rest of eternity. Resuming the RU tells it that more RFD's are available.

IV 6.828

We'd like to hear your opinions about 6.828, so please answer the following questions. (Any answer, except no answer, will receive full credit.)

13. [2 points]: In the second half of the term, we read papers instead of xv6 source code. Which papers did you like? (We read ext3 Journaling, Rethink the Sync, Eliminating Receive Livelock, KeyKOS, Singularity, Anderson Locks, RCU, Barrelfish, Determinator, Software vs Hardware Virtualization, SMP-ReVirt, and Ksplice.)

Answer: Rethink the Sync, Anderson locks, Ksplice.

14. [2 points]: With respect to the labs since quiz 1, are those labs too time consuming, too short, or are they about right? (Lab 4 was COW fork, preemptive multitasking, and IPC; lab 5 was the file system; lab 6 was the network; and lab 7 was the shell and project.)

Answer: About right, but more time for lab 7 would be good.

15. [2 points]: Now that you have completed 6.828, what is the best aspect of 6.828?

Answer: Labs. How streamlined the infrastructure was. Watching ls run on JOS. 6.828 staff. Diverse array of material covered.

16. [2 points]: Now that you have completed 6.828, what is the worst aspect of 6.828?

Answer: That I can't take 6.828 again. The pipe buffer size was too small. There should be a tutorial on using gdb. Printing out papers. The relevance of some papers is unclear. Boring papers. Quiz 1.

End of Quiz

MIT OpenCourseWare
http://ocw.mit.edu

6.828 Operating System Engineering

Fall 2012

For information about citing these materials or our Terms of Use, visit: http://ocw.mit.edu/terms.



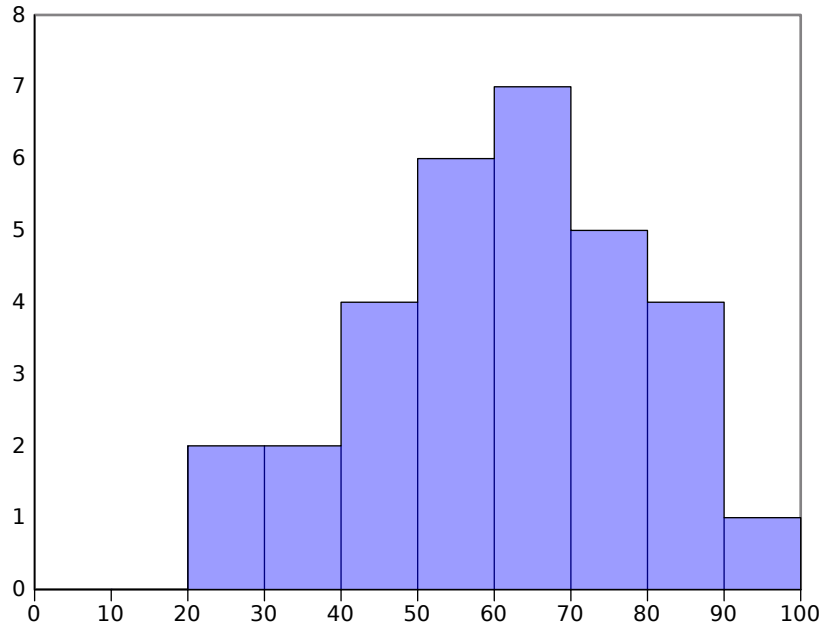
Department of Electrical Engineering and Computer Science

MASSACHUSETTS INSTITUTE OF TECHNOLOGY

6.828 Fall 2009

Quiz II Solutions

Mean 61.3 Median 64 Std. dev. 17.2



I Paper questions

1. [6 points]: Many disks have a cache of disk blocks inside the disk and reorder writes to get better performance. Such reordering could cause problems for the version of ext3 as described by Tweedie in "Journaling the Linux ext2fs Filesystem;" give a scenario that results in inconsistent file system metadata structures.

Answer: A journal flush writes out a sequence of metadata blocks and then updates a header block to point to the new head of the journal. If the header block is flushed to disk before the others and then the power fails, upon recovery the file system will try to replay stale records from the journal.

2. [6 points]: How could you extend ext3 to fix this problem without disabling the disk's cache?

Answer: We could extend the header block to include a hash of the metadata blocks added to the journal. Upon replay, we recompute the hash of the metadata blocks found in the journal and disregard the header block if the hash doesn't match.

3. [6 points]: KeyKOS's use of capabilities is motivated by the "confused deputy" problem. Give an example of the confused deputy problem in Unix. Give an example how a KeyKOS application developer uses capabilities to avoid your example of the confused deputy problem in Unix.

Answer: The paper gives an example of a compiler service that also tracks billing information. If the compiler is passed the path to the billing information as the output file, it can overwrite the billing information because the system can't distinguish between the compiler operating as its own principle and it operating on behalf of the user's principle. In KeyKOS, the compiler must provide a key for each file access, forcing the compiler-writer to specify the capability with the right permissions.

4. [6 points]: To track dependencies introduced by Unix system calls, BackTracker (as described in "Backtracking Intrusions" by King and Chen) maintains objects for processes, files, and filenames. Ben thinks it's redundant to maintain both file objects, which identify files by inode numbers, and filename objects, which identify files by absolute path. He proposes modifying BackTracker to collapse the two into just filename objects, identifying files solely by their absolute path. Alyssa argues that this change will make BackTracker inaccurate (i.e., BackTracker could fail to identify an attack that influences a suspicious file or process). Give an example showing that Alyssa is correct.

Answer: Consider a file with two hard-linked path names. The attacker could write to one of these path names and if a legitimate application then reads from the other path name, Ben's BackTracker will miss this dependency.

5. [6 points]: Louis Reasoner proposes to extend BackTracker to recover from attacks by undoing all the attacker's actions, but not regular user actions. Using BackTracker, he can identify the starting event of an attack. He proposes to first use ReVirt to roll the system back to just before that starting event. For example, for the attack in Figure 1 of the paper, he would roll the system back to just before the "[sh,bash]" process was created. He would then *selectively* roll the system forward using ReVirt, skipping all the events that could have been influenced by the attacker, according to the dependencies that BackTracker has computed. For example, Louis' extension would not replay any operation that depended on the "[sh,bash]" node. Ben points out that this approach may fail to re-execute regular user actions. Give an example showing that Ben is right.

Answer: Just because an event depends on one of the events involved in the attack doesn't mean that the attack depends on that event. Suppose the attacker creates a file in `/tmp` and then a legitimate user runs `ls -l /tmp`. This will create a dependency from the legitimate user's actions to the file involved in the attack. `ls` only observed the attack; it did not partake in it.

6. [6 points]: Figure 8 in the Klee paper ("Unassisted and automatic generation of high-coverage tests for complex systems programs") shows an example of a bug found by Klee in the `pr` utility. Specially, executing `pr -e t2.txt` will result in a buffer overflow if `t2.txt` has the content:

```
\b\b\b\b\b\b\b\t
```

Explain how Klee is able to construct this input.

Answer: Somewhere in `pr`, there must be a conditional in a loop that checks if the next character is `\b`. When KLEE encounters this conditional, it will fork the program and one fork will force the value of the character to be `\b`. Eventually, KLEE will construct a sequence of `\b` characters, encounter the conditional for the `\t`, and discover the buffer overflow in this symbolic execution path.

II Lock-free data structures

Joe Buck is increasing the performance of the the Linux physical page allocator using a lock-free LIFO data structure to track free pages. He reasons that recently freed pages will still be in on-chip hardware caches, so allocating those pages first will improve performance. Joe Buck's code for the lock-free LIFO is below.

```
// Compare the value at addr with oldval.  If equal, store newval
// at addr and return 1, otherwise return 0.  Executes atomically.
int cmpxchg(void *addr, uint32_t oldval, uint32_t newval);

struct page {
    struct page *next;
    /* Other page metadata ... */
};
struct page *head;

void push(struct page *page) {
    while (1) {
        page->next = head;
        if (cmpxchg(&head, (uint32_t)page->next, (uint32_t)page))
            break;
    }
}

struct page * pop(void) {
    while (1) {
        struct page *page = head;
        if (page == NULL)
            return NULL;
        struct page *next = page->next;
        if (cmpxchg(&head, (uint32_t)page, (uint32_t)next))
            return page;
    }
}
```

7. [10 points]: Unfortunately Joe Buck's code has a race condition. Describe a sequence of events for two CPUs that could cause the LIFO to be in an inconsistent state (that is, an allocated page appears in the LIFO or an unallocated page does not appear in the LIFO). For your convenience, we provided the first two events of a sequence; your job is to add the others.

1. Initially, *head* → page A → page B → page C → *NULL*.
2. CPU 1 starts to `pop`; it sets `page = A` and `next = B`.

Answer:

3. CPU 2 pops A, now *head* → page B → page C → *NULL*
4. CPU 2 pops B, now *head* → page C → *NULL*
5. CPU 2 pushes A, now *head* → page A → page C → *NULL*
6. CPU 1 sees that `head == A` and sets `head = B`, now page B, which is allocated, appears in the LIFO.

8. [10 points]: Some CPUs provide atomic instructions that make it easier to write lock-free code. For example, PowerPC CPUs provide the Load-Linked and Store-Conditional instructions. LL must be paired with a SC. LL loads a value from memory and the CPU remembers that the address is load-linked. SC stores a value to an address if the address has not been written to by any other CPU since the corresponding LL. Write a correct lock-free LIFO implementation that uses LL and SC.

```
// Return the 32-bit value at addr and remember addr is load-linked.
uint32_t ll(void *addr);
// Store val at addr if *addr has not been written to since the last ll from addr.
// Return 0 on failure, 1 on success.
int sc(uint32_t val, void *addr);

void push(struct page *page) {
```

Answer:

```
while (1) {
  page->next = (struct page*)ll(&head);
  if (sc((uint32_t)page, &head))
    break;
}
```

```
}

struct page * pop(void) {
```

Answer:

```
while (1) {
  struct page *page = (struct page*)ll(&head);
  if (p == NULL)
    return NULL;
  if (sc((uint32_t)page->next, &head))
    return page;
}
```

```
}
```

III VMM

Many x86 CPUs now have support in hardware for CPU, MMU, and IO port virtualization. The CPU virtualization features provide each guest VM with its own segment/interrupt descriptor tables, control registers, and segment registers. A virtual machine monitor (VMM) can use CPU virtualization to trap a VM when it executes an instruction or causes an event that could affect the VMM or other VMs. IO port virtualization allows VMMs to control VM access to IO ports, trapping into the VMM when the VM accesses certain IO ports.

9. [6 points]: AMD virtualization hardware allows the VMM to intercept certain CPU instructions and events by setting bits in a bitmask. As discussed in Section 4.4 of "A Comparison of Software and Hardware for x86 Virtualization" the performance of a VMM depends on the frequency of exits from the VM. To help avoid frequent exits, AMD hardware copies (or shadows) CPU state into a Virtual Machine Control Block (VMCB). Guest instructions that manipulate CPU state (e.g. `lcr0`) operate on the shadow state instead of the real CPU state. Circle the instructions that the VMM *must* intercept, because it is not possible to shadow the state they manipulate.

- Instructions that read or write control registers (e.g. `cr3`)
- (−) Instructions that cause the computer to shutdown
- Instructions that load descriptor tables (e.g. `lgdt`)
- (−) Instructions that clears cache contents without writing back to memory
- `pushf` and `popf`
- `iret`

10. [6 points]: You are unhappy with the performance of networking in JOS. On x86 hardware, port-based IO is not optimized and JOS's e100 driver is executing many `in` and `out` instructions, which hurt performance. The e100 exposes all of the same control registers using memory-mapped IO, which is much faster than port-based IO. Suppose we're running JOS in a virtual machine, but giving it direct access to our host's e100 card. Describe how you might use dynamic binary translation in the VMM to transparently convert JOS's e100 driver from using `in` and `out` port-based IO to using memory-mapped IO.

Answer: When the binary translator encounters an `in` or `out` instruction, it should replace the instruction with the appropriate memory load or store from the corresponding memory-mapped register.

11. [10 points]: Modern MMU virtualization hardware works as described in Section 7.4 of "A Comparison of Software and Hardware for x86 Virtualization." As usual, guest VMs create *guest page tables* that translate from guest virtual addresses to guest physical addresses. However, with MMU virtualization, guest physical addresses are then further translated through a *nested page table*, created by the VMM, which maps guest physical addresses to host physical addresses.

When preparing a VM to run, the VMM must allocate physical pages for the guest's memory and initialize this nested page table. Write the code to do this below. Your implementation should allocate `npages` host physical pages and map them in the page table `pgdir` starting at guest physical address 0. Return 0 from the function on success, or return a negative value on an error.

```
/* Helper functions from JOS  */
int     page_insert(pde_t *pgdir, struct Page *pp, void *va, int perm);
int     page_alloc(struct Page **pp_store);
physaddr_t page2pa(struct Page *pp);

/* Guest permission bits */
#define PTE_GUEST       (PTE_P|PTE_W|PTE_U)

int
vm_init_mem(unsigned int npages, pde_t *pgdir)
{









}
```

Answer:

```
int i, r;
for (i = 0; i < npages; i++) {
  if ((r = page_alloc(&p)) < 0)
    return r;
  if ((r = page_insert(pgdir, p, (void*)(i*PGSIZE), PTE_GUEST)) < 0)
    return r;
}
return 0;
```

IV High-performance JOS

12. [10 points]: Ben, a little sleep-deprived from the network lab, dreams of turning JOS into the next big, high-performance web serving platform. He took the easy route in his network lab and made his receive syscall always return immediately, even if no packet is available, which is less than ideal for CPU utilization. He starts by making his receive syscall block, much like `sys_ipc_recv`.

Ben keeps around his old non-blocking receive function for use in the new implementation. Now that he's no longer polling, he configures the e100 to generate an interrupt whenever it puts a packet in the receive ring and installs `e100_rx_intr` as the interrupt handler. Finally, since only one environment can receive at a time, he adds a few global variables to track the currently receiving environment.

```
// Copy the next packet into dst and return the length of the packet.
// If the receive ring is empty, return 0.
int e100_recv_nonblocking(void *dst);

// Environment currently blocked on receive, or NULL if none.
struct Env *e100_receiver;
// Packet buffer of the currently receiving environment.
void       *e100_receiver_dst;
```

Give pseudo-code for the new blocking syscall implementation and for `e100_rx_intr`. You can assume `dst` is a valid pointer to a sufficiently large buffer.

```
int sys_e100_recv(void *dst) {
```

Answer:

```
if (e100_receiver)
  return -EINVAL;
if ((r = e100_recv_nonblocking(dst)) > 0)
  return r;
e100_receiver = curenv;
e100_receiver_dst = dst;
curenv->env_status = ENV_NOT_RUNNABLE;
sched_yield();
}

void e100_rx_intr(void) {
    // Acknowledge the interrupt.  Otherwise, the e100 blocks further interrupts.
    e100_ack_rx();
```

Answer:

```
if (!e100_receiver) return;
lcr3(e100_receiver->cr3);
e100_receiver->env_tf.tf_regs.tf_eax =
   e100_recv_nonblocking(e100_receiver_dst);
e100_receiver->env_status = ENV_RUNNABLE;
e100_receiver = NULL;
lcr3(curenv->cr3);


}
```

13. [6 points]: Is your solution susceptible to receive livelock? If so, give a scenario that demonstrates the livelock. If not, explain why not.

Answer: No. If the user environment isn't keeping up with the incoming packets, the receive ring will fill up, the e100 will start dropping packets, and will stop generating interrupts until space is freed up in the ring. Packets are only removed from the receive ring when the user environment is ready to process them, which puts back-pressure on card. At most one interrupt will be generated per packet fully processed by the user environment.

V 6.828

We'd like to hear your opinions about 6.828, so please answer the following questions. (Any answer, except no answer, will receive full credit.)

14. [2 points]: The network lab took more time on average than we had intended. What would have saved you the most time? If it's test cases, which test cases would you add? If it's documentation, what would you like documentation on?

15. [2 points]: What is the best aspect of 6.828?

16. [2 points]: What is the worst aspect of 6.828?

End of Quiz

MIT OpenCourseWare
http://ocw.mit.edu

6.828 Operating System Engineering

Fall 2012

For information about citing these materials or our Terms of Use, visit: http://ocw.mit.edu/terms.



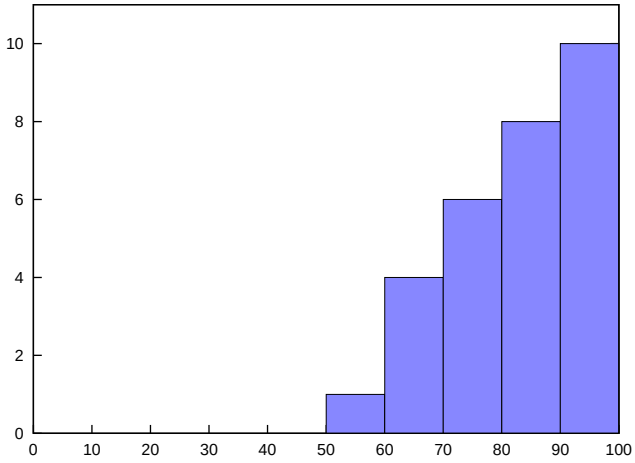
Department of Electrical Engineering and Computer Science

MASSACHUSETTS INSTITUTE OF TECHNOLOGY

6.828 Fall 2011

Quiz II Solutions

Mean 82 Median 85 Std. dev. 10.9



I Short paper questions

1. [8 points]: In Singularity ("Singularity: rethinking the software stack" by Hunt and Larus) a system call is run on the same stack as user code. As the authors point out, this has the potential to entangle user and kernel garbage collection. Why can a malicious SIP *not* trick the kernel into examining a user stack frame when the kernel performs garbage collection? (The paper doesn't give a precise answer; you'll have to deduce it as best as you can.)

Answer: Crossing the user/kernel boundary pushes a barrier on the stack, which the garbage collector knows not to pass when scanning the stack. This, combined with the fact that object references cannot be passed through the kernel ABI and the language is typesafe, means that the kernel garbage collector will never traverse into a user stack frame.

2. [8 points]: The paper on Linux scalability uses a standard trick with generation numbers to avoid a scalability bottleneck in the `dentry` cache (section 4.4). The pseudo code is as follows:

```
lookup(inode, name):
    dentry = hash(inode, name)
    g = dentry.gen
    if g == 0:
        # some core is modifying dentry
        fall back to locking protocol
    # copy relevant fields
    i = dentry.inode
    n = dentry.name
    g1 = dentry.gen
    if g == g1 and inode == i and name == n:
        ok = atomic_inc_if_not_zero(&dentry.refcnt);
        if (!ok) fall back to locking protocol
    else:
        fall back to locking protocol
    return dentry
```

```
modify(dentry):
    spin lock dentry
    g = dentry.gen
    dentry.gen = 0 // DELETE THIS LINE
    dentry.inode = ...
    dentry.name = ...
    dentry.gen = g + 1
    unlock dentry
```

(continued on next page)

Give an example of something that could go wrong if you deleted the line marked "DELETE THIS LINE".

Answer: If `lookup` were to run while a dentry is being modified, lookup could return a dentry that isn't what the caller asked for. (Even with the indicated line included in the code, this is a problem because `modify` may alter a dentry with live references. This can be fixed in `modify` by checking that refcnt is zero after setting gen to 0. With this modification, if the indicated line is deleted and `lookup` runs between the assignment of `inmode` and `name` in `modify`, `lookup` can return a dentry the caller didn't request.)

3. [8 points]: The ballooning technique described in "Memory Resource Management in VMware ESX Server" by Waldspurger avoids double paging by monitor and guests. Why does ESX use ballooning instead of a scheme in which the guest kernel explicitly tells the VMM about how it uses physical pages?

Answer: This requires minimal guest modifications and keeps the page eviction policy in the guest memory manager, which has the most accurate information about memory usage.

MCS locks don't suffer performance collapse like ticket spin locks do, as described in unpublished document assigned for the lecture 16 (paper #243). For reference in the following two questions, here is an implementation of MCS locks in C (which follows the description of MCS locks in the unpublished document):

```
1  // Each core has a qnode for each lock
2  struct qnode {
3      volatile struct qnode *next;
4      volatile char waiting;
5      char __pad[0] __attribute__((aligned(CACHELINE)));
6  };
7
8  typedef struct {
9      struct qnode *tail __attribute__((aligned(CACHELINE)));
10 } mcslock;
11
12 void
13 mcs_lock(mcslock *L, struct qnode *mynode)
14 {
15     mynode->next = NULL;
16     struct qnode *predecessor = fetch_and_store(&L->tail, mynode);
17     if (predecessor) {
18         mynode->waiting = 1;
19         predecessor->next = mynode;
20         while (mynode->waiting)
21             /* spin */;
22     }
23 }
24
25 void
26 mcs_unlock(mcslock *L, struct qnode *mynode)
27 {
28     if (!mynode->next) {
29         if (compare_and_swap(&L->tail, mynode, NULL))
30             return;
31         while (!mynode->next)
32             /* spin */;
33     }
34     mynode->next->waiting = 0;
35 }
```

Each core invokes lock and unlock with its own private `qnode`.

4. [8 points]: Use the above code to explain why MCS locks don't suffer performance collapse as ticket locks do when several cores are contending for the same lock.

Answer: Each core waiting on a contended MCS lock spins on a separate cache line (line 21). Hence, when the lock is released, only the cache line belonging to the next waiter needs to be transferred (line 36). In contrast, for a ticket lock, every core spins on the same cache line, so release must transfer $O(n)$ cache lines, where n in the number of waiting cores.

5. [8 points]: As the paper reports, a disadvantage of MCS locks is that acquiring and releasing an uncontended MCS lock takes more time than acquiring and releasing an uncontended ticket lock (see Figure 11). Using the above MCS code and the ticket code from the paper (Figure 1), explain why MCS locks are more expensive in the non-contended case.

Answer: An uncontended MCS lock requires an atomic operation in both acquire and release, while a ticket lock only requires an atomic operation in acquire.

II Rethink the Sync

Tom the TA writes the following grading program. The idea is that per-assignment scores are stored in a set of input files, and the program's job is to compute a final course score for each student and write it to file *studentname*`.final`.

```
main():
    remove all files ending with ".final"
    for each student name s in alphabetical order:
        read assignment scores for student s
        calculate final grade
        filename = s + ".final"
        fd = creat(filename)
        write(fd, final grade, ...)
        close(fd)
        printf("done with %s\n", s)
```

Tom uses a Linux laptop with an EXT3 journaling file system in "journaled data" mode (so that the journal contains file content, in addition to metadata). He runs the following:

```
program | cat
```

Tom sees "done with x" for all students with names up through "m", and then his laptop crashes. Tom restarts his laptop.

6. [6 points]: Tom thinks he may have to re-run the grading program for some or all students. Explain what guarantees he can count on about which final grades will be on disk after the restart.

Answer: If x.final exists on the disk, then all files that precede x in alphabetical order are guaranteed to be on disk and complete. The last file might be before or after m and might be truncated.

The next year, Tom does the same thing, but this time he uses the xsyncfs file system described in the Rethink the Sync paper. Again, Tom's laptop crashes just after he sees the "done with x" for all students with names up through "m", and he restarts the laptop.

7. [6 points]: Explain what guarantees Tom can count on about which final grades will be on disk after the restart.

Answer: All final grade files before the one starting with m will be on disk and complete. Other grade files might also be on disk and those files may be truncated.

The third year, Tom types these commands on his xsyncfs laptop:

```
program > out
cat out
```

Again, he sees "done with x" for all students with names up through "m", his laptop crashes, and he restarts it.

8. [6 points]: Explain what guarantees Tom can count on about which final grades will be on disk after the restart.

Answer: All grade files will be on disk and complete.

III KLEE

Consider this C program:

```
int
square(int x)
{
    int table[] = { 0, 1, 4, 9, 0xb00b00, 25, 36 };
    return table[x];
}

void
main(void)
{
    printf("%d\n", square(4));
}
```

You run KLEE (see the paper by Cadar *et al.*) on this program. Assume that the C libraries have no bugs.

9. [6 points]: Would KLEE find any bugs in the above program? If yes, describe the full set of test cases that KLEE could generate to illustrate the bug(s).

Answer: No, KLEE would not find any bugs.

Now suppose you run KLEE on this program; `square()` is the same but `main()` is different:

```
int
square(int x)
{
    int table[] = { 0, 1, 4, 9, 0xb00b00, 25, 36 };
    return table[x];
}

void
main(void)
{
    int x = 0;
    scanf("%d", &x);
    printf("%d\n", square(x));
}
```

The call to `scanf()` reads an integer from the standard input and places it in `x`.

10. [6 points]: Would KLEE find any bugs in the above program? If yes, describe the full set of test cases that KLEE could generate to illustrate the bug(s).

Answer: Yes. Stdin would consist of any number < 0 or > 6, optionally followed by anything starting with a non-digit (because scanf stops reading at the first non-digit).

Now this program; again, `square()` is the same but `main()` is different:

```
int
square(int x)
{
    int table[] = { 0, 1, 4, 9, 0xb00b00, 25, 36 };
    return table[x];
}

void
main(void)
{
    int x = 0, y;
    scanf("%d", &x);
    y = square(x);
    assert(y == x*x);
}
```

11. [6 points]: Would KLEE find any bugs in the above program? If yes, describe the full set of test cases that KLEE could generate to illustrate the bug(s).

Answer: Yes. Stdin would consist of any number < 0, 4, or > 6, optionally followed by anything starting with a non-digit.

IV E1000

Ben Bitdiddle has just finished implementing his E1000 receive function, shown below.

```
int
e1000_receive(char *buf)
{
    int tail = read_reg(RDT);

    // Check for and clear "Descriptor Done" bit
    if (!(rx_descriptors[tail].status & E1000_RXD_STAT_DD))
        return 0;
    rx_descriptors[tail].status = 0;

    // Receive the packet
    write_reg(RDT, (tail + 1) % RX_RING_SIZE);
    memmove(buf, rx_buffers[tail], rx_descriptors[tail].length);
    return rx_descriptors[tail].length;
}
```

Ben tests his receive function by sending JOS a hundred packets and, between each packet, checking that it received the right data. Confident in his work, he shows it off to Alyssa P. Hacker. When Alyssa tries sending a hundred packets in a short burst, everything falls apart; Ben's JOS drops several packets, duplicates some, and corrupts others.

12. [8 points]: Why does Ben's implementation drop, duplicate, and corrupt packets and how can he fix it?

Before copying the packet buffer, he updates the receive tail pointer, which allows the E1000 to reuse that buffer immediately. He should change the end of his function to

```
memmove(buf, rx_buffers[tail], rx_descriptors[tail].length);
int len = rx_descriptors[tail].length;
write_reg(RDT, (tail + 1) % RX_RING_SIZE);
return len;
```

(Note it's just as important to read the length before updating the tail pointer as it is to copy the packet contents.)

Louis Reasoner is having similar problems receiving packets. However, he knows that his E1000 driver is correctly receiving all packets. For Louis, packets are getting dropped or duplicated before they are processed by the network server environment. He narrows the problem to his input environment, which looks like

```
void
input(envid_t ns_envid)
{
    while (1) {
        // Receive a packet
        nsipcbuf.pkt.jp_len = sys_net_rx(nsipcbuf.pkt.jp_data, 2048);
        if (nsipcbuf.pkt.jp_len > 0) {
            // Pass packet to network server environment
            ipc_send(ns_envid, NSREQ_INPUT, &nsipcbuf, PTE_P|PTE_U);
            // Wait for NS environment to process the packet
            for (int i = 0; i < 8; i++)
                sys_yield();
        } else
            sys_yield();
    }
}
```

13. [4 points]: Explain the race in Louis' implementation that causes the network server environment to see dropped and duplicated packets.

Answer: Louis is reusing the page passed to the NS environment. Since the NS environment is running in parallel with the input environment, it may still be using that page when Louis reads the next packet into it. Delaying between receiving packets doesn't help, since Louis can't guarantee a bound on how long it takes the NS environment to process a packet.

14. [4 points]: Modify Louis' code to fix this race. (You can mark up the code above if you find it easier.)

Answer: Replace the sys_yield loop with sys_page_alloc(0, &nsipcbuf, PTE_P|PTE_U|PTE_W);

V 6.828

We'd like to hear your opinions about 6.828, so please answer the following questions. (Any answer, except no answer, will receive full credit.)

15. [2 points]: Grade 6.828 on a scale of 0 (worst) to 10 (best)?

Answer: 11

16. [2 points]: Any suggestions for how to improve 6.828?

Answer: More time/emphasis on the project. Make a debugging cheat sheet and hold a GDB tutorial. Post homework solutions. More final project suggestions. More coherent lectures and varied lecture styles. Let students write the shell. Slower introduction to xv6 and C. Make a list of common OS C programming pitfalls. More pictures. More memory checking. Choose final project earlier. Cover x86-64 or ARM. Fewer readings but more depth. More familiarization with provided code.

17. [2 points]: What is the best aspect of 6.828?

Answer: Labs.

18. [2 points]: What is the worst aspect of 6.828?

Answer: Bugs. Quizzes.

End of Quiz

MIT OpenCourseWare
http://ocw.mit.edu

6.828 Operating System Engineering

Fall 2012

For information about citing these materials or our Terms of Use, visit: http://ocw.mit.edu/terms.